MÓDULO 1: INTRODUCCIÓN A AMAZON BEDROCK

INTRODUCCIÓN

APLICACIONES Y CASOS DE USO

"El auge de la inteligencia artificial generativa (IA generativa) revolucionó el trabajo de los científicos de datos, desarrolladores e ingenieros de machine learning (ML). Pueden usarla para trabajar en proyectos innovadores y de gran impacto a través de la automatización de tareas de implementación repetitivas".

Los científicos de datos pueden operar a un nivel más alto y estratégico mediante el diseño de soluciones innovadoras de IA generativa para abordar problemas empresariales reales que afectan de forma directa a los usuarios finales. Pueden enfocarse en elaborar soluciones, como asistentes de IA, optimizadores de la cadena de suministro y sistemas de recomendación personalizada.

Exploración del poder de la IA generativa.

Amazon Bedrock ofrece varias capacidades de procesamiento de lenguaje natural (NLP) que pueden ayudar a los científicos de datos en su trabajo.

- **Resumen de texto:**



El resumen de texto con modelos fundacionales (FM) de Amazon Bedrock ayuda a los científicos de datos a comprender información clave con rapidez en grandes cantidades de texto para una exploración y limpieza eficientes de los datos. Los resúmenes ayudan a explicar los comportamientos de los modelos, acelerar la escritura de informes y mejorar el análisis de los datos de texto.

- **Generación de texto:**



La generación de texto a partir de modelos de lenguaje ayuda a los científicos de datos, ya que aumenta los datos de entrenamiento, genera código, explica modelos y crea un borrador del contenido. Los asistentes de IA funcionan como interfaces de lenguaje natural para consultar datos y modelos de manera interactiva. En este curso, estudiamos las arquitecturas para generar resúmenes de mayor calidad y pertinencia. Estas técnicas incluyen Amazon Bedrock, LangChain y generación aumentada por recuperación (RAG) con incrustaciones persistentes para el reconocimiento contextual y la retención de información clave.

- **Sistemas de respuesta a preguntas:**



Los sistemas de respuesta a preguntas automatizan tareas de datos tediosas, como la lectura de documentación. Proporcionan información respondiendo preguntas analíticas, generan fragmentos de código y resumen documentos. Los asistentes de IA de RAG pueden consultar bases de conocimientos de forma interactiva y generar respuestas contextuales bajo demanda. En este curso, se enseña cómo crear sistemas de respuesta a preguntas y asistentes de IA de RAG.

- **Agentes:**



Los agentes para Amazon Bedrock comprenden las solicitudes de los usuarios en lenguaje natural, dividen las tareas complejas en llamadas a la API y búsquedas de datos, mantienen el contexto de la conversación y realizan acciones para cumplir las solicitudes. El servicio orquesta la ingeniería de peticiones con información específica de la empresa o del dominio, y proporciona respuestas en lenguaje natural. Los agentes para Amazon Bedrock gestionan la infraestructura, la supervisión, el cifrado, los permisos y la administración de invocaciones sin código personalizado. Los agentes para Amazon Bedrock se integran con las barreras de protección de Amazon Bedrock para evitar comportamientos no deseados en las respuestas de los modelos o en los mensajes de los usuarios.

En este curso, se explica cómo puede utilizar los agentes para Amazon Bedrock con el fin de sintetizar y administrar flujos de trabajo de IA generativa. También se explica cómo usar los agentes para Amazon Bedrock con el objetivo de acelerar el desarrollo de aplicaciones de IA generativa.

MÓDULO 2: MODELOS FUNDACIONALES

MODELOS FUNDACIONALES DE AMAZON BEDROCK

INTRODUCCIÓN A LOS MODELOS FUNDACIONALES DE AMAZON BEDROCK

FM de Amazon Bedrock.

Amazon Bedrock ofrece una amplia selección de modelos fundacionales (FM) de alto rendimiento de startups líderes en inteligencia artificial (IA) y de Amazon. Cada uno de estos FM se adapta a distintos casos de uso de inteligencia artificial generativa (IA generativa), como resumen, traducción de idiomas, codificación y generación de imágenes.

Empresa	Modelo fundacional	Descripción
Amazon	Amazon Titan	Se trata de una familia de modelos creada por Amazon que se preentrena con conjuntos de datos de gran tamaño, lo que los convierte en modelos potentes y de uso general.
AI2I Labs	Jurassic-2 Jumba	Se trata de modelos de lenguaje de gran tamaño (LLM) multilingües para la generación de texto en español, francés, alemán, portugués, italiano y holandés.
Anthropic	Claude 3	Los modelos Claude 3 ofrecen un rendimiento cada vez más potente, lo que permite a los usuarios seleccionar el equilibrio óptimo entre inteligencia, velocidad y costo para su aplicación específica.
Cohere	Command y Embed	Se trata de un modelo de generación de texto para aplicaciones empresariales y de un modelo de incrustaciones con el fin de efectuar búsquedas, agrupar en clústeres o clasificar en más de cien idiomas.

Meta	Llama	Los modelos Llama son especialmente útiles para aplicaciones como interfaces de chat, asistentes virtuales y traducción de idiomas.
Mistral AI	Mistral/Mixtral	Modelos de generación de texto sintético, generación de código, RAG o agentes
Stability AI	Stable Diffusion	Se trata de un modelo de conversión de texto a imagen para la generación de imágenes, obras artísticas, logotipos y diseños únicos, realistas y de alta calidad. Ya está disponible Stable Diffusion XL (SDXL) 1.0.

Parámetros de inferencia.

Al interactuar con un FM, puede configurar los parámetros de inferencia a fin de personalizar la respuesta del FM. En general, solo debería ajustar un parámetro por vez, y los resultados pueden variar según el FM.

Los parámetros siguientes pueden usarse para modificar la salida de los modelos de lenguaje de gran tamaño (LLM). No todos los parámetros están disponibles con todos los LLM.

La aleatoriedad y la diversidad:

Los modelos fundacionales en general admiten los siguientes parámetros para controlar la aleatoriedad y la diversidad en la respuesta.

- **Temperatura:**

  El parámetro de **temperatura** controla la aleatoriedad en la elección de palabras. Los valores más bajos llevan a respuestas más predecibles. En la tabla siguiente, se mencionan los valores mínimos, máximos y predeterminados del parámetro de temperatura.

Parámetro	Formato del campo en JSON	Mínimo	Máximo	Predeterminado
Temperatura	temperature	0	1	0

- **K superior:**

  K superior limita las elecciones de palabras a las K opciones más probables. Los valores más bajos disminuyen las respuestas inusuales.

- **P superior:**

  P superior limita las elecciones de palabras de baja probabilidad según la probabilidad acumulada. Ajusta la distribución de respuesta general. En la siguiente tabla, se mencionan los valores mínimos, máximos y predeterminados del parámetro P superior.

Parámetro	Formato del campo en JSON	Mínimo	Máximo	Predeterminado
P superior	topP	0	1	1

Longitud:

Los modelos fundacionales en general admiten los siguientes parámetros para controlar la longitud de la respuesta generada.

- **Longitud de respuesta:**

  La **longitud de respuesta** establece los recuentos de tokens mínimos y máximos. Establece un límite estricto al tamaño de la respuesta. En la siguiente tabla, se mencionan los valores mínimos, máximos y predeterminados del parámetro de longitud de respuesta. La longitud máxima de respuesta depende del FM específico.

Parámetro	Formato del campo en JSON	Mínimo	Máximo	Predeterminado
Longitud de respuesta	maxTokenCount	0	8000	512

- **Penalización por longitud:**

  Penalización por longitud fomenta respuestas más concisas penalizando las más extensas. Establece un límite flexible al tamaño.

- **Secuencias de detención:**

  Secuencias de detención incluyen combinaciones de caracteres específicas que indican al modelo que deje de generar tokens cuando las encuentre. Se utilizan para la terminación anticipada de las respuestas.

USO DE LOS FM DE AMAZON BEDROCK PARA INFERENCIA

Trabajo con FM de Amazon Bedrock.

Algunos parámetros de inferencia son comunes a la mayoría de los modelos, como la temperatura, P superior, K superior y longitud de respuesta. Profundizará en los parámetros únicos específicos del modelo y la configuración de E/S que puede ajustar para lograr la salida deseada basada en el caso de uso.

Modelos fundacionales de Amazon Titan:

Los modelos de Amazon Titan son modelos fundacionales de Amazon. Amazon ofrece el modelo Amazon Titan Text y el modelo Amazon Titan Embeddings a través de Amazon Bedrock.

Los modelos Amazon Titan ofrecen soporte para los siguientes parámetros únicos de inferencia, además de temperatura, P superior y longitud de respuesta, que son parámetros comunes en varios modelos.

- **Secuencias de detención:**

  Con las secuencias de detención **(stopSequences)**, puede especificar secuencias de caracteres para indicar dónde se debería detener el modelo. Use el símbolo de barra vertical **(|)** para separar diferentes secuencias (máximo de veinte caracteres).

- **Amazon Titan Text:**

  Amazon Titan Text es un LLM generativo para tareas como el resumen, la generación de texto, la clasificación, la respuesta a preguntas abiertas y la extracción de información. El modelo de generación de texto se entrena con muchos lenguajes de programación diferentes y formato de texto enriquecido (Rich Text Format, RTF), como tablas, JSON, valores separados por coma (comma-separated values, CSV), etc.

  En el ejemplo siguiente, se muestra una configuración de entrada que se usa para invocar una respuesta de Amazon Titan Text por medio de Amazon Bedrock. Puede pasar parámetros de configuración de entrada junto con una petición de entrada al modelo.

Entrada:

```
{
   "inputText": "<prompt>",
   "textGenerationConfig" : {
       "maxTokenCount": 512,
       "stopSequences": [],
       "temperature": 0.1,
       "topP": 0.9
   }
}
```

En el siguiente ejemplo, se muestra la salida de Amazon Titan Text para la entrada proporcionada en el bloque de código anterior. El modelo devuelve la salida junto con los parámetros, como la cantidad de tokens de entrada y salida generados, junto con el motivo por el que se terminó de generar la respuesta.

Salida:

```
{
   "inputTextTokenCount": 613,
   "results": [{
       "tokenCount": 219,
       "outputText": "<output>",
"completionReason" : "string"
   }]
}
```

- **Amazon Titan Text Embeddings:**

  En el modelo Amazon Titan Text Embeddings, se traducen las entradas de texto (palabras y frases) en representaciones numéricas (incrustaciones). Las aplicaciones de este modelo incluyen la personalización y la búsqueda. La comparación de las incrustaciones produce respuestas más relevantes y contextuales que la coincidencia de palabras.

  En el siguiente ejemplo, se muestra cómo puede crear vectores de incrustación a partir de peticiones mediante la V2 del modelo Amazon Titan Embeddings.

Entrada:

```
{
   body = json.dumps({"inputText": <prompt>,
"dimensions": <256   512   1024>,
"normalize": True   False,
})
   model_id = 'amazon.titan-embed-text-v2:0'
   accept = 'application/json'
   content_type = 'application/json'

   response = bedrock_runtime.invoke_model(
       body=body,
       modelId=model_id,
       accept=accept,
       contentType=content_type )

   response_body = json.loads(response['body'].read())
   embedding = response_body.get('embedding')
}
```

Esto generará un vector de incrustaciones que consiste en números con un aspecto similar a la siguiente salida.

Salida:

```
[0.82421875, -0.6953125, -0.115722656, 0.87890625, 0.05883789, -
0.020385742,
0.32421875, -0.00078201294, -0.40234375, 0.44140625, …]
```

- **Amazon Titan Multimodal Embeddings:**

  El modelo Multimodal Embeddings G1 se usa para casos de uso como la búsqueda de imágenes por texto, por imagen para similitudes o por una combinación de texto e imagen. Traduce la imagen o el texto de entrada en una incrustación que contiene el significado semántico tanto de la imagen como del texto en el mismo espacio semántico.

En el siguiente ejemplo, se muestra cómo puede crear vectores de incrustación a partir de peticiones mediante el modelo Amazon Titan Multimodal Embeddings G1.

Entrada:

```
{
  body = json.dumps({"inputText": <prompt>,
      "inputImage": <image>,
      "embeddingConfig":  { "outputEmbeddingLength":  <output_embedding_length> }
})
  model_id = 'amazon.titan-embed-image-v1'
  accept = 'application/json'
  content_type = 'application/json'

  response = bedrock_runtime.invoke_model(
      body=body,
      modelId=model_id,
      accept=accept,
      contentType=content_type )

  response_body = json.loads(response.get('body').read())
  embedding = response_body.get('message')
}
```

- **Amazon Titan Image Generator:**

  Amazon Titan Image Generator G1 es un modelo de generación de imágenes. Viene en dos versiones v1 y v2.

  Con Amazon Titan Image Generator v1, los usuarios pueden crear imágenes que coincidan con sus descripciones basadas en texto mediante la introducción de peticiones en lenguaje natural. Pueden cargar y editar imágenes existentes, aplicar peticiones basadas en texto o editar partes específicas de una imagen por medio de una máscara de imagen. El modelo también admite la **expansión de imágenes**, que amplía

los límites de una imagen. La **restauración de imágenes** se usa para rellenar las áreas faltantes de la imagen.

El **Amazon Titan Image Generator v2** admite todas las características existentes de Titan Image Generator v1 con capacidades adicionales. Permite a los usuarios aprovechar las imágenes de referencia para guiar la generación de imágenes. La imagen de salida se alineará con el diseño y la composición de la imagen de referencia, sin dejar de seguir la petición textual. También incluye una característica de eliminación automática de fondos para eliminar fondos de imágenes que contengan varios objetos.

AI21 Jurassic – 2 (Mid y Ultra).

Los parámetros comunes de los modelos Jurassic-2 incluyen temperatura, P superior y secuencias de detención. Los modelos Jurassic-2 admiten los siguientes parámetros únicos para controlar la aleatoriedad, la diversidad, la longitud o la repetición en la respuesta:

Longitud

1. **Longitud máxima de finalización (maxTokens):** especifique la cantidad máxima de tokens que se van a utilizar en la respuesta generada.
2. **Secuencias de detención (stopSequences):** configure las secuencias de detención que el modelo reconoce y después de las cuales deja de generar más tokens.

Repeticiones

- **Penalización por presencia (presencePenalty):** utilice un valor más alto para reducir la probabilidad de generar nuevos tokens que ya aparezcan al menos una vez en la petición o en la finalización.
- **Penalización por recuento (countPenalty):** utilice un valor más alto para reducir la probabilidad de generar nuevos tokens que ya aparezcan al menos una vez en la petición o en la finalización. El valor es proporcional al número de apariciones del token.
- **Penalización por frecuencia (frequencyPenalty):** utilice un valor más alto para reducir la probabilidad de generar nuevos tokens que ya aparezcan al menos una vez en la petición o en la finalización. El valor es proporcional a la frecuencia de aparición del token (normalizado a la longitud del texto).
- **Penalización de tokens especiales:** reduzca la probabilidad de repetición de caracteres especiales. Los valores predeterminados son true de la siguiente manera.

- ✓ **Espacios en blanco (applyToWhitespaces):** el valor **true** aplica la penalización a los espacios en blanco y a las líneas nuevas.
- ✓ **Puntuaciones (applyToPunctuation):** el valor **true** aplica la penalización a la puntuación.
- ✓ **Números (applyToNumbers):** el valor **true** aplica la penalización a los números.
- ✓ **Palabras clave (applyToStopwords):** el valor **true** aplica la penalización a las palabras clave.
- ✓ **Emojis (applyToEmojis)**: el valor **true** excluye los emojis de la penalización.

- **Jurassic-2 Mid:**

  Se trata de un modelo de tamaño mediano que está optimizado para seguir las instrucciones y el contexto del lenguaje natural, por lo que no es necesario proporcionarle ningún ejemplo. Es ideal para componer texto similar al producido por un humano y resolver tareas de lenguaje complejas, como responder a preguntas y resumir.

- **Jurassic-2 Ultra:**

  Ultra es un modelo de gran tamaño que puede aplicar a tareas de comprensión o generación de lenguaje. Entre los casos de uso, se incluyen generación de textos de marketing, respaldo de asistentes de IA, asistencia en la escritura creativa, realización de resúmenes y extracción de información.

  Entrada:

```
{
  "prompt": "<prompt>",
  "maxTokens": 200,
  "temperature": 0.5,
  "topP": 0.5,
  "stopSequences": [],
  "countPenalty": {"scale": 0.0},
  "presencePenalty": {"scale": 0.0},
  "frequencyPenalty": {"scale": 0.0}
}
```

Salida:


```
{
    "id": 1234,
    "prompt": {
        "text": "<prompt>",
        "tokens": [
            {
                "generatedToken": {
                    "token": "\u2581who\u2581is",
                    "logprob": -12.980147361755371,
                    "raw_logprob": -12.980147361755371
                },
                "topTokens": null,
                "textRange": {"start": 0, "end": 6}
            },
            //...
        ]
    },
    "completions": [
        {
            "data": {
                "text": "<output>",
                "tokens": [
                    {
                        "generatedToken": {
                            "token": "<|newline|>",
                            "logprob": 0.0,
                            "raw_logprob": -0.01293118204921484
                        },
                        "topTokens": null,
                        "textRange": {"start": 0, "end": 1}
                    },
                    //...
```

```
            ]
        },
        "finishReason": {"reason": "endoftext"}
    }
  ]
}
```

- **Jamba – Instruct:**

  Jamba-Instruct ofrece una ventana de contexto de 256 K para tareas de generación de texto, resumen y respuesta a preguntas para la empresa.

  Para llamar al modelo Jamba-Instruct de AI21 Labs, puede usar ***invoke_model*** o la API de ***converse***.

  Entrada con *invoke_model:*

```
response = bedrock.invoke_model(
modelId='ai21.jamba-instruct-v1:0',
body=json.dumps({
'messages': [
{ 'role': 'user',
'content': <prompt>
}
],
})
)
```

  Entada con *converse:*

```
response = bedrock.converse(
modelId='ai21.jamba-instruct-v1:0',
'messages': [
{ 'role': 'user',
'content': [{'text':<prompt>}]
}
```

```
],
)
```

Anthropic Claude.

Anthropic Claude 2 y Claude 3 son modelos adicionales disponibles para la generación de textos en Amazon Bedrock. Claude es un modelo de IA generativa de Anthropic. Se creó específicamente para mantener conversaciones, hacer resúmenes, responder preguntas, automatizar flujos de trabajo, escribir código y más. Admite desde diálogos sofisticados y la generación de contenido creativo hasta el seguimiento detallado de instrucciones.

Puede usar Amazon Bedrock para enviar solicitudes de inferencia a la API de finalizaciones de texto de Anthropic Claude o a la API de mensajes de Anthropic Claude. La API de mensajes se utiliza para crear aplicaciones conversacionales, como un asistente virtual o una aplicación de capacitación. Use la API de finalización de texto para aplicaciones de generación de texto de un solo turno. Por ejemplo, generar texto para una entrada de blog o resumir el texto que proporciona un usuario.

Claude utiliza parámetros comunes, como temperatura, P superior, K superior y secuencias de detención. Además, los modelos Claude utilizan el siguiente parámetro único para ajustar aún más la salida de la respuesta.

- **Longitud máxima (max_tokens_to_sample)**: especifique la cantidad máxima de tokens que se utilizarán en la respuesta generada.

Uso de la API de mensajes:

Puede usar la **API de mensajes** para crear aplicaciones de asistente de IA. La API administra los intercambios conversacionales entre un usuario y un modelo Anthropic Claude (asistente).

En la API de mensajes de Anthropic Claude, cada mensaje de entrada debe ser un objeto con un rol y un contenido.

Este es un ejemplo con un mensaje de un solo usuario:

```
[{"role": "user", "content": "Hello, Claude"}]
```

En el ejemplo siguiente, se muestra una configuración de entrada que se utiliza para invocar una respuesta de Anthropic Claude 3 por medio de Amazon Bedrock con varios turnos de conversación:

Entrada:

```
[ {"role": "user", "content": "Hello there."},
 {"role": "assistant", "content": "Hi, I'm Claude. How can I help you?"},
 {"role": "user", "content": "Can you explain LLMs in plain English?"},
]
```

Salida:

```
{
  "id": "<identifier>",
  "type": "message",
  "role": "assistant",
  "model": "claude-3-sonnet-20240229",
  "content": [
     {
        "type": "text",
        "text": " Sure, I'll try to explain …"
     }
  ],
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "usage": {
     "input_tokens": <token count>,
     "output_tokens": <token count>
  }
}
```

Uso de la API de Converse:

La API de Converse proporciona un conjunto unificado de parámetros que funcionan en todos los modelos que admiten mensajes.

Stability AI (SDXL).

Es un modelo de conversión de texto a imagen usado para generar imágenes detalladas. SDXL admite los siguientes tipos de creación de imágenes:

- **Peticiones de imagen a imagen:** consisten en introducir una imagen para obtener variaciones de ella.
- **Restauración de imágenes:** consiste en reconstruir las partes faltantes de una imagen.
- **Expansión de imágenes:** consiste en crear una extensión perfecta de una imagen existente.

Los modelos Stability AI Diffusion admiten los siguientes controles.

- **Potencia de la petición (cfg_scale):** este control determina en qué medida la imagen final refleja la petición. Utilice un número más bajo para aumentar la aleatoriedad de la generación.
- **Paso de generación (steps):** este control determina cuántas veces se realiza el muestreo de la imagen. Una mayor cantidad de pasos puede dar lugar a un resultado más preciso.
- **Semilla (seed):** este control define la configuración de ruido inicial. Utilice la misma semilla y la misma configuración que en una puesta en marcha anterior, de modo que la inferencia pueda crear una imagen similar. Si no configura este valor, se establece como un número aleatorio.

En el ejemplo siguiente, se muestra una configuración de entrada que se utiliza para invocar una respuesta de SDXL por medio de Amazon Bedrock.

Entrada:

```
{       "text_prompts": [
        {
            "text": string,
            "weight": float
        }
    ],
    "height": int,
    "width": int,
    "cfg_scale": float,
    "clip_guidance_preset": string,
    "sampler": string,
    "samples",
    "seed": int,
```

```
    "steps": int,
    "style_preset": string,
    "extras" :JSON object

}
```

Salida:

```
{
   "result": "success",
   "artifacts": [
      {
         "seed": 123,
         "base64": "<image in base64>",
         "finishReason": "SUCCESS"
      },
      //...
   ]
}
```

Command, de Cohere.

Command es el modelo de generación de texto insignia de Cohere. Está entrenado para seguir los comandos de los usuarios y ser útil de forma instantánea en aplicaciones empresariales prácticas, como resúmenes, redacción de textos, diálogo, extracción y respuesta a preguntas. Cohere, que se optimizó para prioridades empresariales, cumple con System and Organizations Control (SOC) 2 y hace hincapié en la seguridad, la privacidad y la IA responsable.

Además de la temperatura, P superior, K superior, longitud máxima y secuencias de detención, el modelo Command, de Cohere, admite los siguientes controles únicos:

- **Devolución de probabilidades (return_likelihoods):** especifique si las probabilidades del token se devuelven en la respuesta y de qué modo. Puede especificar las siguientes opciones:
  - ✓ **GENERATION:** esta opción solo devuelve las probabilidades de los tokens generados.
  - ✓ **ALL:** esta opción devuelve las probabilidades de todos los tokens.
  - ✓ **NONE:** esta opción no devuelve ninguna probabilidad. Esta es la opción predeterminada.

- **Transmisión (stream):** especifique *true* para devolver la respuesta parte por parte en tiempo real y *false* a fin de devolver la respuesta completa una vez que finalice el proceso.

En el ejemplo siguiente, se muestra una configuración de entrada que se usa para invocar una respuesta de Command, de Cohere, por medio de Amazon Bedrock.

Entrada:

```
{
"prompt": "string",
"temperature": float,
"p": float,
"k": float,
"max_tokens": int,
"stop_sequences": ["string"],
"return_likelihoods": "GENERATION|ALL|NONE",
"stream": boolean,
"num_generations": int
}
```

REFERENCIA DE API DE AMAZON BEDROCK

MÉTODOS DE AMAZON BEDROCK

"Amazon Bedrock proporciona una lista de API a las que puede acceder en sus cuadernos respectivos y funciones de AWS Lambda para acceder a Amazon Bedrock. Existen API relacionadas con la configuración de Amazon Bedrock y API relacionadas con el tiempo de ejecución que explorará en esta sección".

API relacionadas con la instalación y configuración de Amazon Bedrock.

ListFoundationModels:

Este método se usa para proporcionar una lista de modelos fundacionales de Amazon Bedrock que puede usar. En el siguiente ejemplo, se muestra cómo enumerar los modelos base usando Python.

Entrada:

```
%pip install --upgrade boto3
import boto3
import json
bedrock = boto3.client(service_name='bedrock')
model_list=bedrock.list_foundation_models()
for x in range(len(model_list.get('modelSummaries'))):
    print(model_list.get('modelSummaries')[x]['modelId'])
```

Obtendrá una lista de todos los modelos fundacionales disponibles en Amazon Bedrock y la información de sus metadatos respectivos. A continuación, encontrará un ejemplo de la salida filtrada con la ID de modelo.

Salida:

```
amazon.titan-tg1-large
amazon.titan-image-generator-v1:0
amazon.titan-image-generator-v1
amazon.titan-image-generator-v2:0
amazon.titan-text-premier-v1:0
amazon.titan-embed-g1-text-02
amazon.titan-text-lite-v1:0:4k
amazon.titan-text-lite-v1
amazon.titan-text-express-v1:0:8k
amazon.titan-text-express-v1
amazon.titan-text-agile-v1
amazon.titan-embed-text-v1:2:8k
amazon.titan-embed-text-v1
amazon.titan-embed-text-v2:0:8k
amazon.titan-embed-text-v2:0
amazon.titan-embed-image-v1:0
amazon.titan-embed-image-v1
stability.stable-diffusion-xl-v1:0
stability.stable-diffusion-xl-v1
ai21.j2-grande-instruct
```

ai21.j2-jumbo-instruct

ai21.j2-mid

ai21.j2-mid-v1

ai21.j2-ultra

ai21.j2-ultra-v1:0:8k

ai21.j2-ultra-v1

ai21.jamba-instruct-v1:0

ai21.jamba-1-5-large-v1:0

ai21.jamba-1-5-mini-v1:0

anthropic.claude-instant-v1:2:100k

anthropic.claude-instant-v1

anthropic.claude-v2:0:18k

anthropic.claude-v2:0:100k

anthropic.claude-v2:1:18k

anthropic.claude-v2:1:200k

anthropic.claude-v2:1

anthropic.claude-v2

anthropic.claude-3-sonnet-20240229-v1:0:28k

anthropic.claude-3-sonnet-20240229-v1:0:200k

anthropic.claude-3-sonnet-20240229-v1:0

anthropic.claude-3-haiku-20240307-v1:0:48k

anthropic.claude-3-haiku-20240307-v1:0:200k

anthropic.claude-3-haiku-20240307-v1:0

anthropic.claude-3-opus-20240229-v1:0:12k

anthropic.claude-3-opus-20240229-v1:0:28k

anthropic.claude-3-opus-20240229-v1:0:200k

anthropic.claude-3-opus-20240229-v1:0

anthropic.claude-3-5-sonnet-20240620-v1:0:18k

anthropic.claude-3-5-sonnet-20240620-v1:0:51k

anthropic.claude-3-5-sonnet-20240620-v1:0:200k

anthropic.claude-3-5-sonnet-20240620-v1:0

cohere.command-text-v14:7:4k

cohere.command-text-v14

cohere.command-r-v1:0
cohere.command-r-plus-v1:0
cohere.command-light-text-v14:7:4k
cohere.command-light-text-v14
cohere.embed-english-v3:0:512
cohere.embed-english-v3
cohere.embed-multilingual-v3:0:512
cohere.embed-multilingual-v3
meta.llama2-13b-chat-v1:0:4k
meta.llama2-13b-chat-v1
meta.llama2-70b-chat-v1:0:4k
meta.llama2-70b-chat-v1
meta.llama2-13b-v1:0:4k
meta.llama2-13b-v1
meta.llama2-70b-v1:0:4k
meta.llama2-70b-v1
meta.llama3-8b-instruct-v1:0
meta.llama3-70b-instruct-v1:0
mistral.mistral-7b-instruct-v0:2
mistral.mixtral-8x7b-instruct-v0:1
mistral.mistral-large-2402-v1:0
mistral.mistral-small-2402-v1:0

API relacionadas con el tiempo de ejecución de Amazon Bedrock.

InvokeModel:

Esta API invoca el modelo de Amazon Bedrock especificado para poner en marcha la inferencia usando la entrada proporcionada en el cuerpo de la solicitud. Use **InvokeModel** a fin de poner en marcha la inferencia para los modelos de texto, de imágenes y de incrustación.

Ejecutar la inferencia en un modelo de texto

Puede enviar una solicitud de invocación para ejecutar la inferencia en un modelo. Configure el parámetro **accept** para aceptar cualquier tipo de contenido en la respuesta.

En el siguiente ejemplo, se detalla cómo generar texto con Python mediante la petición "**What is Amazon Bedrock?**" (¿Qué es Amazon Bedrock?).

Entrada:

```
bedrock_rt = boto3.client(service_name='bedrock-runtime')
prompt = "What is Amazon Bedrock?"
configs= {
"inputText": prompt,
"textGenerationConfig": {
"maxTokenCount": 4096,
"stopSequences": [],
"temperature":0,
"topP":1
}
}
body=json.dumps(configs)
modelId = 'amazon.titan-tg1-large'
accept = 'application/json'
contentType = 'application/json'
response = bedrock_rt.invoke_model(
    body=body,
    modelId=modelId,
    accept=accept,
    contentType=contentType
)
response_body = json.loads(response.get('body').read())
print(response_body.get('results')[0].get('outputText'))
```

Salida:

Amazon Bedrock es un servicio administrado que hace que los modelos fundacionales de los principales startups de IA y los propios modelos de Amazon Titan estén disponibles mediante una API. Para obtener información actualizada sobre Amazon Bedrock y cómo se aprueban los modelos 3P, el proveedor aprobado es responsable de actualizar la documentación de Amazon Bedrock. Para obtener más información sobre Amazon Bedrock y cómo se aprueban, respaldan o seleccionan los modelos 3P, consulte la documentación proporcionada y las preguntas frecuentes pertinentes.

InvokeModelWithResponseStream:

Esta API invoca el modelo de Amazon Bedrock especificado para poner en marcha la inferencia usando la entrada proporcionada. Devuelve la respuesta en una transmisión.

Para el streaming, puede configurar **x-amzn-bedrock-accept-type** en el encabezado con el objetivo de que contenga el tipo de contenido requerido de la respuesta. En el ejemplo siguiente, el encabezado está configurado para aceptar cualquier tipo de contenido. El valor predeterminado es **application/json**.

En el ejemplo, se detalla cómo generar texto de streaming con Python. Aquí, se utiliza el modelo Amazon titan-tg1-large y la petición "**Write an essay for living on Mars using 10 sentences**" (Escriba un ensayo relacionado con la vida en Marte en 10 oraciones).

Entrada:

```
prompt = "Write an essay for living on Mars using 10 sentences."

configs= {
    "inputText": prompt,
    "textGenerationConfig": {
        "temperature":0
    }
}

body=json.dumps(configs)

accept = 'application/json'
```

```
contentType = 'application/json'
modelId = 'amazon.titan-tg1-large'

response = bedrock_rt.invoke_model_with_response_stream(
    modelId=modelId,
    body=body,
    accept=accept,
    contentType=contentType
)

stream = response.get('body')
if stream:
    for event in stream:
        chunk = event.get('chunk')
        if chunk:
            print((json.loads(chunk.get('bytes').decode())))
```

En la siguiente respuesta, se incluye la salida de la transmisión en fragmentos de datos cuando la invocación se realiza a través de la API de streaming de Amazon Bedrock.

Salida:

```
{'outputText': '\nLiving on Mars would be an incredible adventure. The planet is known for its red deserts, vast canyons, and unique geological features. The atmosphere is thin, but with the right technology, it could', 'index': 0, 'totalOutputTextTokenCount': None, 'completionReason': None, 'inputTextTokenCount': 13}
```

Converse:

Esta API proporciona una interfaz coherente que funciona con todos los modelos que admiten mensajes. Esto le permite escribir código una vez y usarlo con diferentes modelos. Si un modelo tiene parámetros de inferencia únicos, también puede pasar esos parámetros únicos al modelo.

Entrada:

```
import boto3
client = boto3.client("bedrock-runtime")
model_id = "amazon.titan-tg1-large"
# Parámetros de inferencia que se deben utilizar.
temperature = 0.5
top_p = 0.9
inference_config={"temperature": temperature,"topP": top_p}
# Configurar las peticiones del sistema y los mensajes que se enviarán al modelo.
system_prompts = [{"text": "You are a helpful assistance. Please answer the query politely."}]
conversation = [
  {
    "role": "user",
    "content": [{"text": "Hello, what is the capital of France?"}]
  }
]
# Usar la API de Converse.
response = client.converse(
  modelId=model_id,
  messages=conversation,
  inferenceConfig=inference_config
)
print(response['output']['message']["content"][0]["text"])
```

Salida:

```
Hola, la capital de Francia es París. También es la ciudad más grande del país.
```

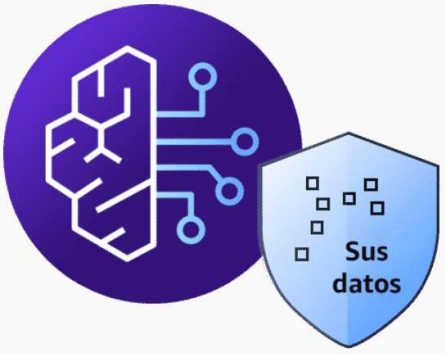
SEGURIDAD Y GOBERNANZA

PROTECCIÓN DE DATOS Y AUDITABILIDAD

Protección y privacidad integral de los datos.

Amazon Bedrock proporciona protección y privacidad integral de sus datos. Los datos que utiliza en Amazon Bedrock no se usan para mejorar el servicio ni se comparten con proveedores de modelos de terceros. Puede usar AWS PrivateLink con Amazon Bedrock para establecer una conexión privada entre sus FM y su nube virtual privada (VPC) sin exponer el tráfico a internet.

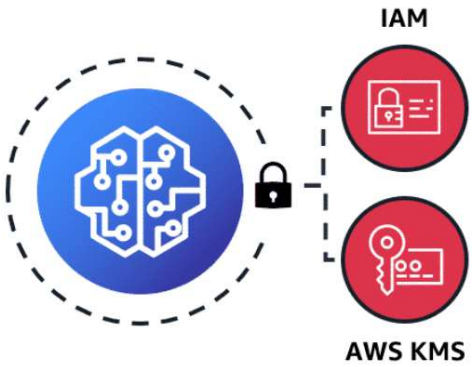
Sus datos se cifran en tránsito y en reposo. Puede personalizar sus FM de manera privada para controlar cómo se utilizan y se cifran sus datos. Amazon Bedrock hace una copia aparte del modelo fundacional base y entrena la copia privada del modelo.



Protección de sus aplicaciones de IA generativa.

Para proteger sus FM personalizados, puede utilizar los servicios de seguridad de AWS a fin de dar forma a su estrategia de seguridad en profundidad para defensa. Cuando sus FM personalizados se cifran con claves AWS Key Management Service (AWS KMS) y se almacenan cifrados.

Si usa AWS Identity and Access Management (AWS IAM), puede controlar el acceso a sus FM personalizados. Puede permitir o denegar el acceso a FM específicos, decidir qué servicios pueden obtener inferencias y elegir quién puede iniciar sesión en la consola de Amazon Bedrock.

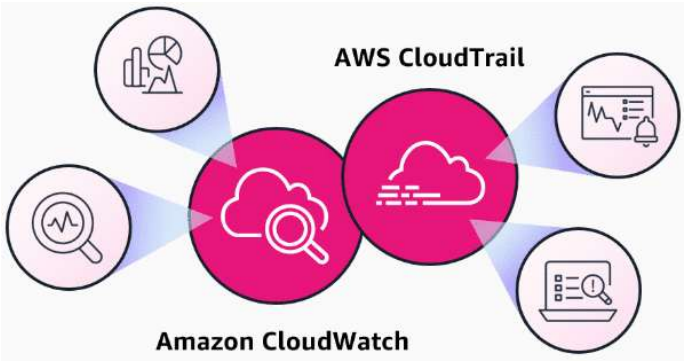


Protección de FM con IAM y AWS KMS

Soporte para gobernanza y auditabilidad.

Amazon Bedrock ofrece capacidades y herramientas de supervisión y registro integrales que puede usar para cumplir con los requisitos de gobernanza y auditoría.

Puede usar Amazon CloudWatch para rastrear las métricas de uso y crear paneles personalizados con métricas que necesita para fines de auditoría. Use AWS CloudTrail para supervisar la actividad de las API y resolver problemas a medida que integra otros sistemas en sus aplicaciones de IA generativa.



Registros de Amazon CloudWatch y AWS CloudTrail

MÓDULO 3: COMPONENTES DE LAS APLICACIONES

INTRODUCCIÓN

INFORMACIÓN GENERAL SOBRE LOS COMPONENTES DE LAS APLICACIONES DE IA GENERATIVA

Componentes típicos de una aplicación de IA generativa.

El diagrama a continuación incluye los componentes típicos involucrados en la creación de una aplicación de inteligencia artificial generativa (IA generativa).



Un modelo fundacional (FM) se encuentra en el centro, rodeado por componentes. Estos incluyen aplicación móvil o aplicación web de frontend, interfaz de FM, entorno de machine learning (ML), entrenamiento de modelos, conjuntos de datos empresariales, bases de datos vectoriales, incrustaciones de texto e imágenes, y

almacenamiento en memoria a largo plazo. La gobernanza y la seguridad están integradas en todos los componentes.

COMPONENTES DE UNA APLICACIÓN DE IA GENERATIVA

MODELOS FUNDACIONALES Y LA INTERFAZ DEL FM

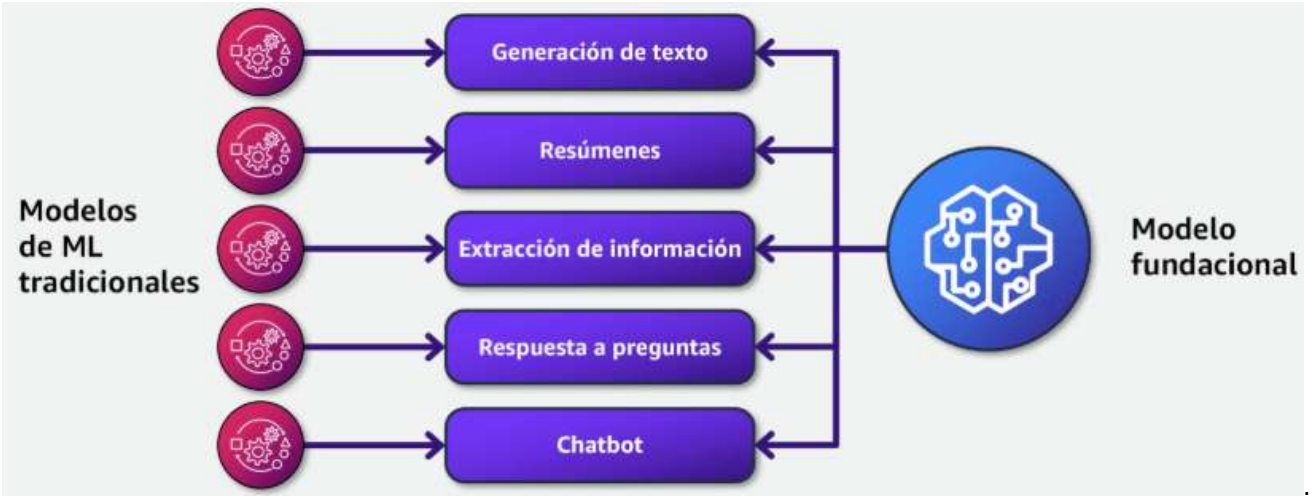
Interfaz del FM.

En el centro de la aplicación de IA generativa, se encuentra el modelo fundacional (FM) que la impulsa. Los FM son modelos entrenados con datos a escala que pueden adaptarse a varias tareas descendentes.



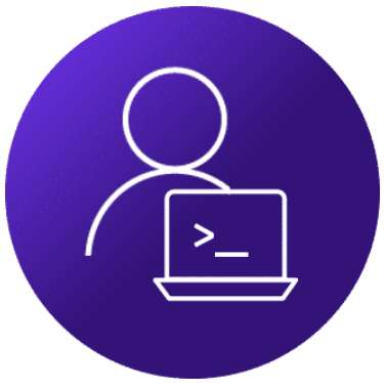
Estos modelos ingieren enormes cantidades de conjuntos de datos que abarcan diferentes temas, asuntos y modalidades. Cuando los modelos se entrenan con estos datos, adquieren una comprensión matizada del lenguaje, el audio y las imágenes. Por lo tanto, puede implementar los FM para realizar una gran variedad de tareas, mientras que los modelos de ML tradicionales solo son adecuados para las tareas en las que se han entrenado.

Su amplia aplicabilidad hace que los FM sean muy potentes. Los FM proporcionan la base sobre la cual se pueden crear varias aplicaciones de IA generativa. Los modelos de lenguaje de gran tamaño (LLM) son un subgrupo de los FM que se entrenan con un gran conjunto de datos de texto



Interfaz y peticiones.

Para usar un FM, necesita una interfaz que le proporcione acceso a este. La interfaz suele ser una API implementada en una infraestructura que se administra en su nombre, o puede alojarse por sí misma junto con el FM. Con el autoalojamiento, es necesario adquirir un entorno de ML que admita instancias de computación acelerada creadas específicamente. Con las llamadas a la API, puede pasar peticiones al FM y recibir respuestas de inferencia.

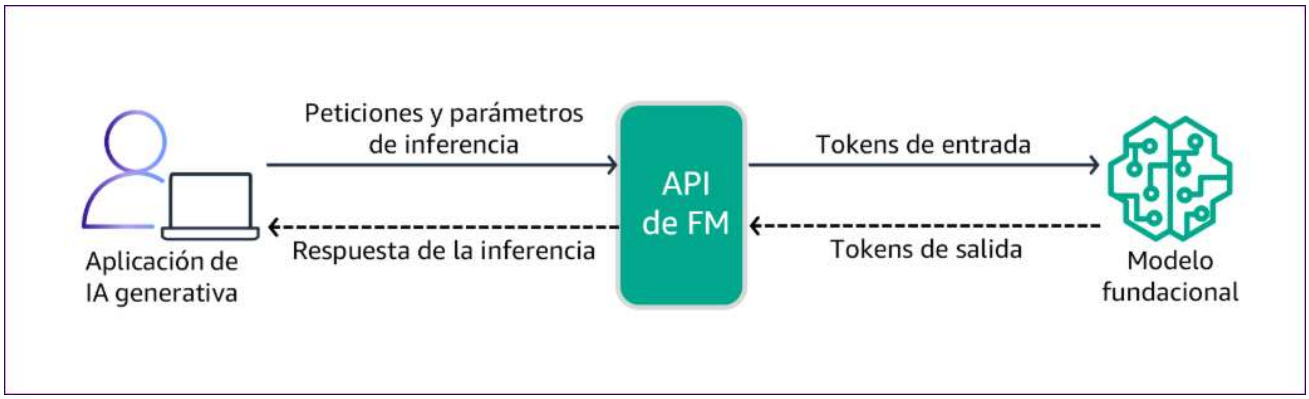


Parámetros de inferencia.

Además de proporcionar peticiones, usar parámetros de inferencia efectivos puede influenciar en gran medida la salida del FM. Puede pasar los parámetros junto con las peticiones a las API de la interfaz del FM. Como ya aprendió en un módulo anterior, cada FM tiene un conjunto específico de parámetros de inferencia, como P superior, K superior y temperatura.

Los LLM funcionan con tokens, que pueden ser palabras, letras o unidades de palabras parciales. (Mil tokens equivalen a aproximadamente 750 palabras).

El LLM toma una secuencia de tokens de entrada y predice el token siguiente. Los parámetros de inferencia ayudan a proporcionar orientación al LLM a fin de producir la salida o una secuencia de tokens que sean relevantes para su caso de uso.



Aplicación API generativa que envía peticiones y parámetros de inferencia a FM mediante la API de FM.

TRABAJAR CON CONJUNTOS DE DATOS E INCRUSTACIONES

Conjuntos de datos empresariales.

Los datos públicos, y no los patentados, potencian a los FM por diseño. De ahí que los FM sean capaces de generar texto en lenguaje natural, imágenes y audio para tareas generalizadas. Sin embargo, para admitir aplicaciones empresariales personalizadas, los FM necesitarán acceder a los datos de los **conjuntos de datos empresariales**.



Las empresas acumulan enormes volúmenes de datos internos, como documentos, presentaciones, manuales de usuario, informes y resúmenes de transacciones. Puede proporcionar los datos de la empresa al FM como contexto junto con la petición. Esto ayudará al modelo a devolver salidas más precisas y adaptadas a la empresa.

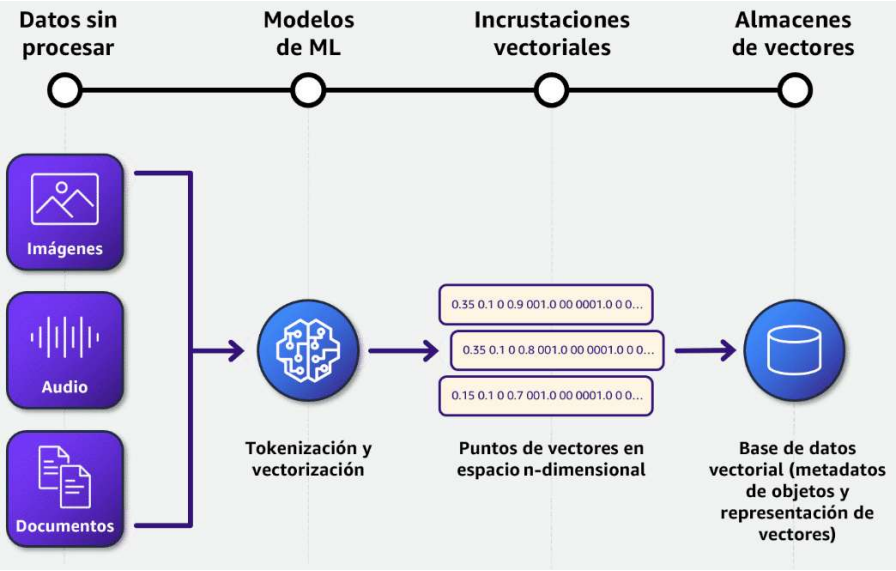
Sin embargo, si ofrece demasiado contexto, su LLM puede confundirse, lo que daría lugar a alucinaciones de LLM, peticiones más largas y costos más elevados. Para evitar las alucinaciones y mantener la latencia y los costos bajos, debe pasar la cantidad adecuada de contenido junto con la petición.

Hay dos maneras de proporcionar el contexto. Si desea consultar el contenido de un documento concreto, puede proporcionar dicho documento como contexto. Sin embargo, si desea seleccionar uno o varios documentos basados en la consulta de un almacén de datos de la empresa, necesita una forma eficaz de hacer búsquedas en los conjuntos de datos empresariales mediante el texto de la petición de entrada. La **incrustación vectorial** es un mecanismo utilizado para encontrar datos en el conjunto de datos empresariales que sean similares a la información de la petición.

Incrustaciones vectoriales.

La incrustación es el proceso mediante el cual se asigna una representación numérica a textos, imágenes y audio en un espacio vectorial. Hay varios FM de incrustación que generan incrustaciones vectoriales.

Los conjuntos de datos empresariales, como los documentos, las imágenes y el audio, se pasan a un modelo de incrustación en forma de tokens y se vectorizan. Estos vectores, junto con los metadatos de los vectores, se indexan y almacenan en una base de datos de vectores creada específicamente para su rápida recuperación. En el siguiente diagrama, se proporcionan más detalles sobre las incrustaciones.



Los conjuntos de datos empresariales se vectorizan mediante un FM de incrustación y se almacenan en una base de datos vectorial para facilitar la búsqueda de documentos similares a una consulta del usuario.

Por ejemplo, considere la modalidad de texto. El objetivo de generar incrustaciones es capturar similitudes semánticas entre textos. Esto significa que el texto con significados similares se asigna a puntos cercanos en el espacio vectorial. La incrustación ayuda a encontrar información relevante en función de las peticiones del usuario.

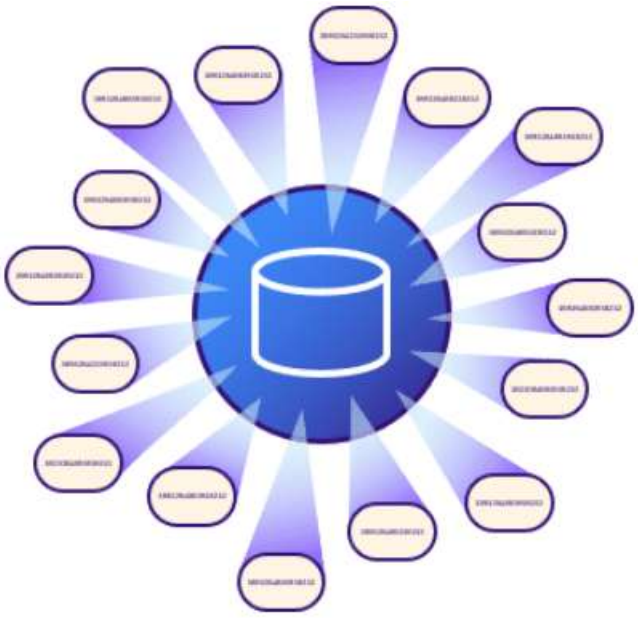
Amazon Bedrock proporciona modelos de incrustación como los siguientes:

- **Amazon Titan Embeddings G1:** modelo de texto que puede convertir texto en incrustaciones.
- **Cohere Embed:** (inglés y multimodal) modelos de incrustación de texto para inglés y varios idiomas.
- **Titan Embeddings multimodal:** incrustación multimodal de imágenes y texto.

Bases de datos vectoriales.

La función principal de las **bases de datos vectoriales** es almacenar de forma compacta mil millones de vectores de alta dimensionalidad que representan palabras y entidades. Las bases de datos vectoriales ofrecen búsquedas eficaces de similitudes entre mil millones de vectores en tiempo real.

Los algoritmos más comunes usados para realizar la búsqueda de similitudes son los k vecinos más cercanos (k-nearest neighbors, k-NN) o la similitud del coseno.



Amazon Web Services (AWS) ofrece las siguientes opciones de bases de datos vectoriales viables:

- Amazon OpenSearch Service (aprovisionada)
- Amazon OpenSearch sin servidor

- Extensión pgvector en Amazon Relational Database Service (Amazon RDS) para PostgreSQL
- Extensión pgvector en Amazon Aurora PostgreSQL: edición compatible

AWS también ofrece Pinecone en AWS Marketplace. Además, hay opciones de código abierto en memoria, como Facebook AI Similarity Search (FAISS) y Chroma.

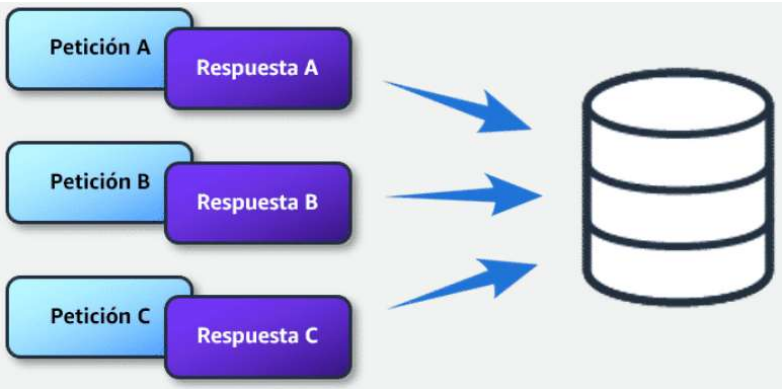
Datos empresariales vectorizados.

Una vez que se vectorizan los datos empresariales, puede buscar la petición dada en una base de datos vectorial. Los resultados se suministran al FM como contexto para mejorar la salida del modelo de IA generativa. Esto puede disminuir las alucinaciones, un fenómeno en el que un LLM genera con confianza información verosímil pero falsa. El contexto y las bases de datos vectoriales se usan en la generación aumentada por recuperación (RAG).

COMPONENTES ADICIONALES DE LAS APLICACIONES

Almacén de historial de peticiones.

Un asistente de IA no puede admitir conversaciones de varios turnos sin acceso al historial de peticiones. Un almacén de historial de peticiones permite mantener conversaciones conscientes del contexto, pertinentes y coherentes. Muchos FM tienen una ventana de contexto limitada, lo que significa que solo se puede pasar una longitud fija de datos como entrada. El almacenamiento de información de estado en una conversación de varios turnos se convierte en un problema cuando la conversación supera la ventana de contexto. Para resolver este problema, puede implementar un almacén de historial de peticiones. Puede persistir el estado de la conversación, lo que permite tener un historial a largo plazo de la conversación.



Petición con respuestas coincidentes almacenadas en un almacén de historial de peticiones.

Si almacena las peticiones y el historial de respuestas, puede buscar las peticiones de una conversación anterior para evitar solicitudes repetitivas al FM.

El almacén de historial de peticiones tiene dos finalidades. Proporciona datos a los equipos de auditoría y cumplimiento para garantizar el cumplimiento de las políticas de la empresa y la normativa gubernamental. Un desarrollador de aplicaciones también puede utilizar el almacén de historial de peticiones para depurar errores y advertencias de la aplicación.

Aplicaciones web y móviles de frontend.

Para hacer uso de las capacidades de IA generativa de un FM, es necesario crear una aplicación frontend que actúe como interfaz para los usuarios de la aplicación. La aplicación frontend es la responsable de construir las peticiones y llamar al FM de la API. También puede realizar tanto el procesamiento previo de los datos como el procesamiento posterior de las respuestas antes de compartirlas con el usuario. Además, la aplicación debe gestionar los errores de forma fluida para ofrecer la mejor experiencia posible al usuario.



PERSONALIZACIÓN DE UN FM

RAG Y AJUSTE FINO DEL MODELO

RAG.

La generación aumentada por recuperación (RAG) es una técnica para desarrollar aplicaciones de IA generativa que utiliza orígenes de datos empresariales y bases de datos vectoriales para superar limitaciones de conocimiento. La RAG funciona usando un módulo recuperador para encontrar información relevante en un almacén de datos externos como respuesta a la petición de un usuario. Estos datos recuperados se usan como

contexto, en combinación con la petición original, para crear una petición ampliada que se pasa al modelo de lenguaje. El modelo de lenguaje luego genera una finalización que incorpora el conocimiento empresarial.

Con la RAG, los modelos de lenguaje pueden ir más allá de los datos de entrenamiento originales y usar información actualizada del mundo real. La RAG aborda el desafío que suponen los cambios frecuentes en los datos dado que recupera información actualizada y pertinente, en lugar de depender de datos potencialmente desactualizados.

Limitaciones de la RAG y cómo puede abordarlas el ajuste fino del modelo.

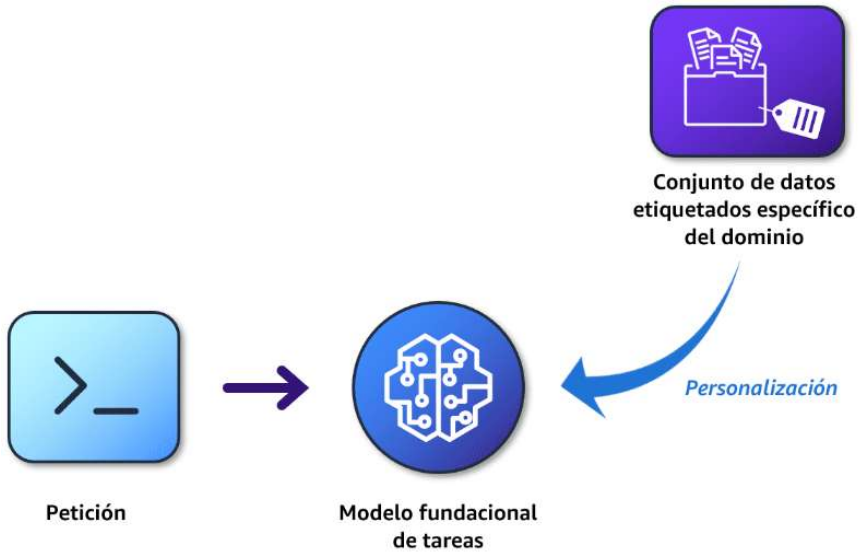
La RAG es útil para los casos de uso empresariales, pero depender solamente de esta trae aparejadas ciertas limitaciones. Por ejemplo, la recuperación se limita a los conjuntos de datos empresariales incrustados en los almacenes vectoriales en el momento de la recuperación. El modelo permanece estático. Con ventanas de contexto grandes, la latencia de LLM puede ser un problema.

El ajuste fino del modelo puede cambiar el FM subyacente tanto o tan poco como desee. El modelo puede aprender a rendir mejor en una tarea específica con la ayuda de un conjunto de datos etiquetados. Piense en esto como un cambio permanente en el modelo subyacente. En comparación, la RAG hace que el modelo sea inteligente solo de manera temporal por medio de proporcionarle contexto de fragmentos de documentos pertinentes.

Hay dos grandes categorías de ajuste fino: la personalización (también conocida como aprendizaje basado en peticiones) y el entrenamiento previo continuo (también llamado adaptación al dominio).

- **Personalización (aprendizaje basado en peticiones):**

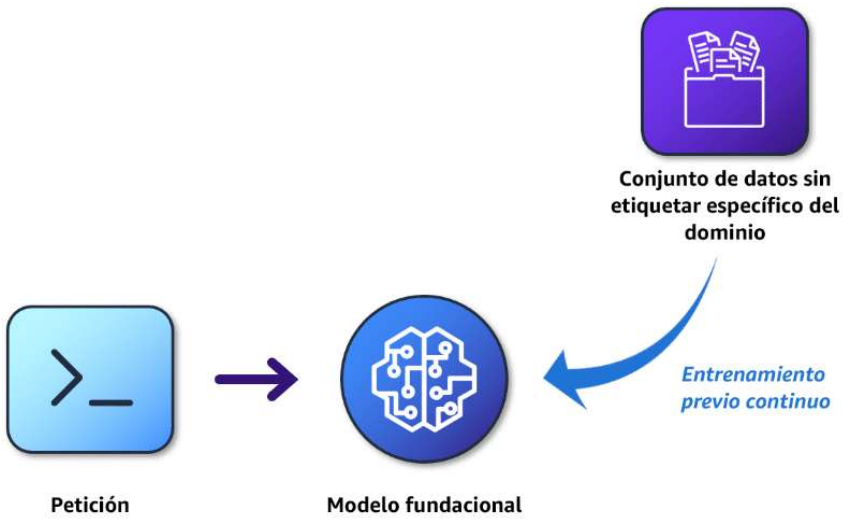
  El ajuste fino del FM subyacente para una tarea específica se logra mediante el aprendizaje basado en peticiones. Esto involucra direccionar el modelo hacia un conjunto de datos de ejemplos etiquetados a partir del cual desea que el modelo aprenda. Los ejemplos etiquetados tienen el formato de pares de petición y respuesta, y están enunciados como instrucciones. El proceso de ajuste fino basado en peticiones modifica las ponderaciones del modelo. Suele ser ligero y requiere pocos ciclos de entrenamiento. Como el ajuste fino es específico para cada tarea en particular, no puede generalizarse en varias tareas.



- **Entrenamiento previo continuo (adaptación del dominio):**

  Con el ajuste fino mediante la adaptación de dominios, puede usar FM entrenados previamente y adaptarlos a varias tareas usando datos limitados específicos del dominio. Puede ajustar el FM con la cantidad que desee de datos sin etiquetar específicos de su dominio. Según la cantidad de datos utilizados para el ajuste fino, usará el lenguaje de su empresa, como los términos técnicos específicos de su dominio.

  Para realizar el ajuste fino, necesita un entorno de ML que pueda ocuparse de todo este proceso. También se requiere acceso a las instancias de computación adecuadas para el ajuste fino.



Comparación de la RAG y el ajuste fino.

Criterios	RAG	Ajuste fino
Cuándo utilizarlo	Cuando cuenta con datos personalizados y el FM no tiene suficiente conocimiento del dominio del caso de uso.	Cuando los datos son muy personalizados o exclusivos de su dominio y el FM tiene un conocimiento limitado sobre este. También se puede usar para reducir la latencia porque el FM requerirá menos contexto en la consulta.
Esfuerzo de implementación	Intermedio	Alto
Costo	Suele ser entre bajo e intermedio	El costo de un ajuste fino puede variar de bajo a alto según el tamaño del modelo.

ASEGURAR LAS APLICACIONES DE IA GENERATIVA

Gobernanza y seguridad.

La gobernanza y la seguridad son de suma importancia al desarrollar aplicaciones de IA generativa. Son fundamentales para cada paso y componente de la aplicación de IA generativa. La implementación de medidas de gobernanza y seguridad permite asegurar la mitigación de riesgos, el mantenimiento de la supervisión, el aseguramiento de la toma de responsabilidad y la confianza de los clientes.



Tenga en cuenta las siguientes prácticas recomendadas al desarrollar aplicaciones de IA generativa.

- Administre y audite quién puede acceder a cada parte de la aplicación de IA generativa, como los FM o los métodos de API.
- Supervise y registre el acceso al FM subyacente, e informe al respecto, ya sea de forma directa o mediante enfoques personalizados.
- Registre las solicitudes y las respuestas del FM para mantener la conformidad con las regulaciones y garantizar la explicabilidad de sus acciones.

- Audite periódicamente los FM con datos de prueba y simule ataques de inyección de peticiones para garantizar que no haya consecuencias imprevistas.
- Documente el proceso completo de las diversas facetas de la aplicación y manténgalas actualizadas.

PATRONES DE ARQUITECTURA DE APLICACIONES DE IA GENERATIVA

Gobernanza y seguridad.

A continuación, aprenderá sobre diversos patrones de arquitectura que se pueden implementar con Amazon Bedrock o con Amazon Bedrock y LangChain para desarrollar aplicaciones de IA generativa en los siguientes casos de uso.

- Generación de texto o código
- Resumen de texto
- Respuesta a preguntas
- Asistentes de IA

 LangChain es un marco de trabajo de código abierto para crear aplicaciones basadas en modelos de lenguaje de gran tamaño (LLM). LangChain proporciona herramientas y abstracciones para mejorar la personalización, exactitud y relevancia de la información que generan los modelos.

Patrón de generación de texto a código con Amazon Bedrock.

Generación de texto es un término que se usa para cualquier caso de uso en el que la salida del modelo es texto recién generado. Puede usarse para escribir artículos, poemas, blogs, libros, correos electrónicos, etc. En Amazon Bedrock, puede usar varios FM para las tareas de generación de texto.

El patrón de arquitectura para la generación de texto con Amazon Bedrock se ilustra en la imagen siguiente. Puede pasar al modelo fundacional de Amazon Bedrock una petición usando un área de pruebas de Amazon Bedrock o una llamada a la API de Amazon Bedrock. El modelo generará una respuesta de texto basada en la petición de entrada que usted proporcione.



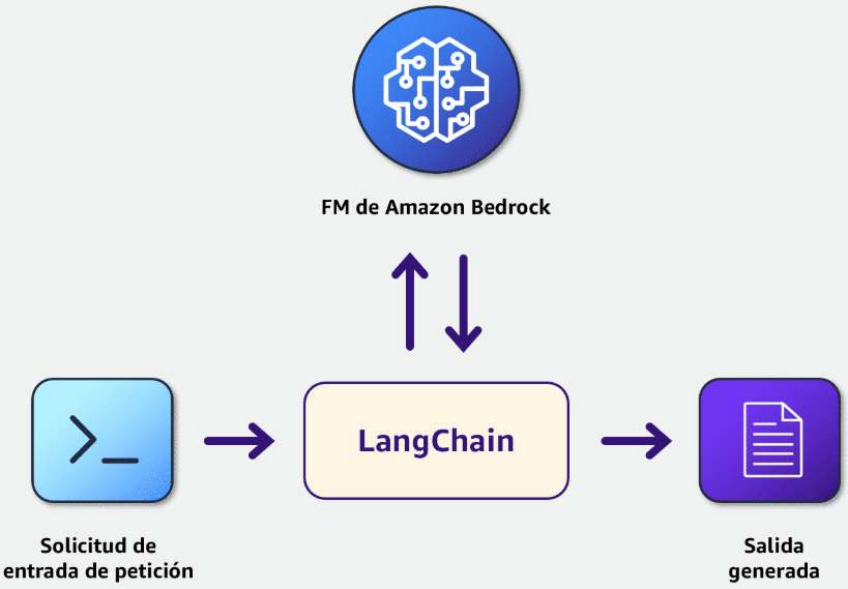
Solicitud de entrada de petición enviada a un FM de Amazon Bedrock que produce una salida generada.

También puede utilizar este patrón para diversas tareas relacionadas con la codificación y la programación. Algunos ejemplos son la generación de consultas SQL, la explicación y traducción de código, la corrección de errores y la optimización de código. El uso de FM para tareas relacionadas con la codificación ayuda a los desarrolladores y científicos de datos a crear rápidamente prototipos de sus ideas y casos de uso. La salida generada por FM contendrá fragmentos de código.

Patrón de generación de texto mediante LangChain.

Para la generación de texto, también puede utilizar LangChain, una biblioteca de código abierto, para casos de uso específicos de generación de texto. Puede combinar LangChain con los FM de generación de texto en Amazon Bedrock para admitir tanto la generación de texto como el desarrollo de asistentes conversacionales de IA.

LangChain actúa como una capa de orquestación entre la solicitud de entrada de petición y la salida generada desde el FM de Amazon Bedrock. El patrón de arquitectura para la generación de texto con LangChain se ilustra en el diagrama siguiente.



Solicitud de entrada de petición enviada a LangChain para administrar la interacción del FM de Amazon Bedrock con la salida generada.

Patrón de resumen de texto.

Cuando trabaje con resúmenes de documentos, la selección del patrón de arquitectura de la aplicación depende del tamaño del documento en relación con el tamaño del contexto del modelo. Otros desafíos que presentan los documentos grandes son los errores por falta de memoria y las alucinaciones. Puede usar los siguientes dos enfoques de arquitectura para crear una aplicación de resumen basada en el tamaño del documento.

- **Resumen de textos para documentos pequeños:**

  Puede aplicar este patrón de arquitectura cuando el texto del documento quepa en el tamaño del contexto del FM. Un texto pequeño (o un archivo pequeño) se pasa directamente al FM, que responde con un resumen del texto. El patrón de arquitectura de esta situación se ilustra en la figura siguiente.

  Nota: Este enfoque *no* funciona cuando el tamaño del documento o del texto es grande y no cabe en la ventana de contexto del FM.

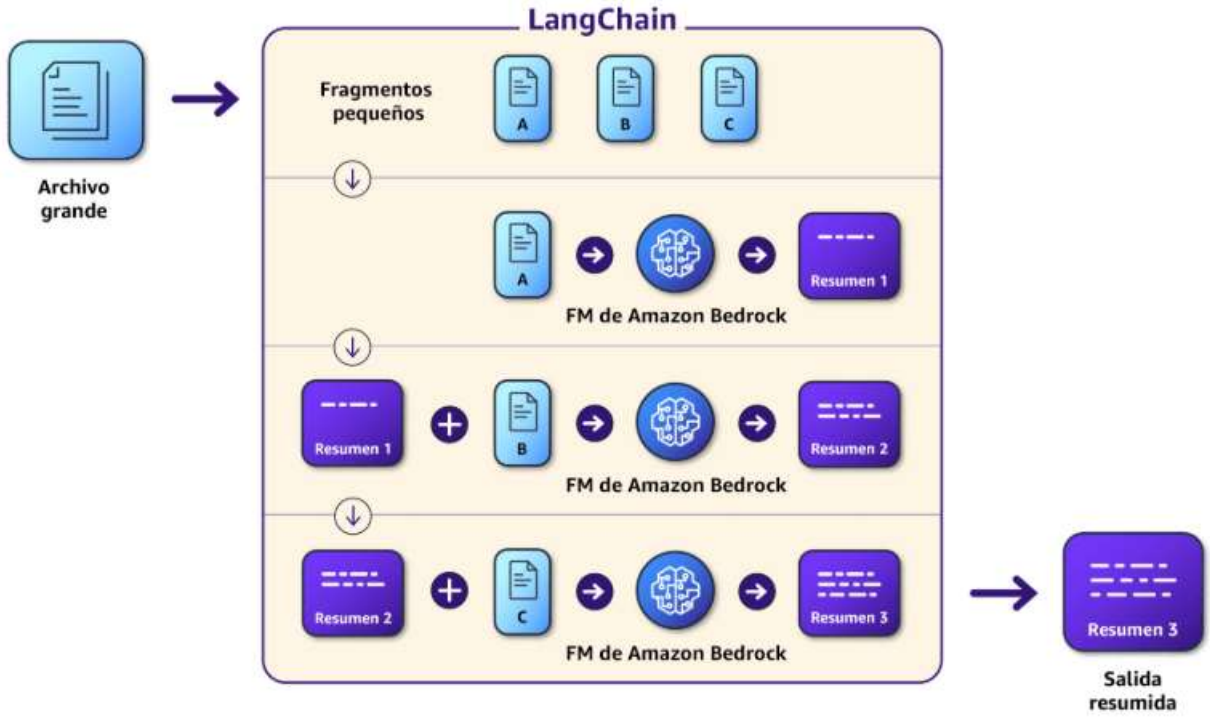


- **Resumen de textos para documentos grandes:**

  Para los documentos o textos grandes, no es posible pasar el texto completo al FM debido al tamaño limitado del contexto. Para resolver este problema, puede usar una arquitectura map-reduce y aplicar los conceptos de fragmentación y encadenado de peticiones.

  La arquitectura de resumen de texto para documentos de gran tamaño incluye los siguientes pasos:

  1. Se divide un documento grande en varios fragmentos de números (n) pequeños con herramientas como LangChain.
  2. Se envía cada fragmento al FM para generar el resumen correspondiente.
  3. Se agrega el siguiente fragmento al primer resumen generado y se vuelve a resumir.
  4. Se itera sobre cada fragmento para crear una salida resumida final.



Patrón de respuestas a preguntas.

La respuesta a preguntas implica la extracción de respuestas de consultas objetivas planteadas en lenguaje natural. En general, un sistema de respuesta a preguntas procesa una consulta frente a una base de conocimientos que contiene datos estructurados o no estructurados y genera una respuesta con información precisa. Poder garantizar una alta exactitud es clave para desarrollar un sistema de respuesta a preguntas que sea útil, fiable y de confianza, en especial en lo que respecta a los casos de uso de la empresa. A continuación, revisará dos patrones de arquitectura para aplicaciones de respuesta a preguntas.

- **Caso de uso de base genérica:**

  Con el caso de uso de base genérica, el usuario puede enviar una petición al FM en Amazon Bedrock y obtener respuestas basadas en información genérica que el público entienda. El caso práctico de base genérica no tiene el elemento de personalización.

  Hay un documento pequeño que un usuario puede usar para formular preguntas. Dicho documento se somete a un procesamiento más detallado en el FM en Amazon Bedrock para obtener una respuesta generada. Esta respuesta puede surgir de forma natural del modelo base sin ninguna recuperación específica. Es posible que le falte personalización para casos de uso que requieran información específica y minuciosidad, como los casos de uso médicos o los documentos legales y financieros. Este patrón de arquitectura se ilustra en la imagen siguiente.



- **Casos de uso específicos y personalizados:**

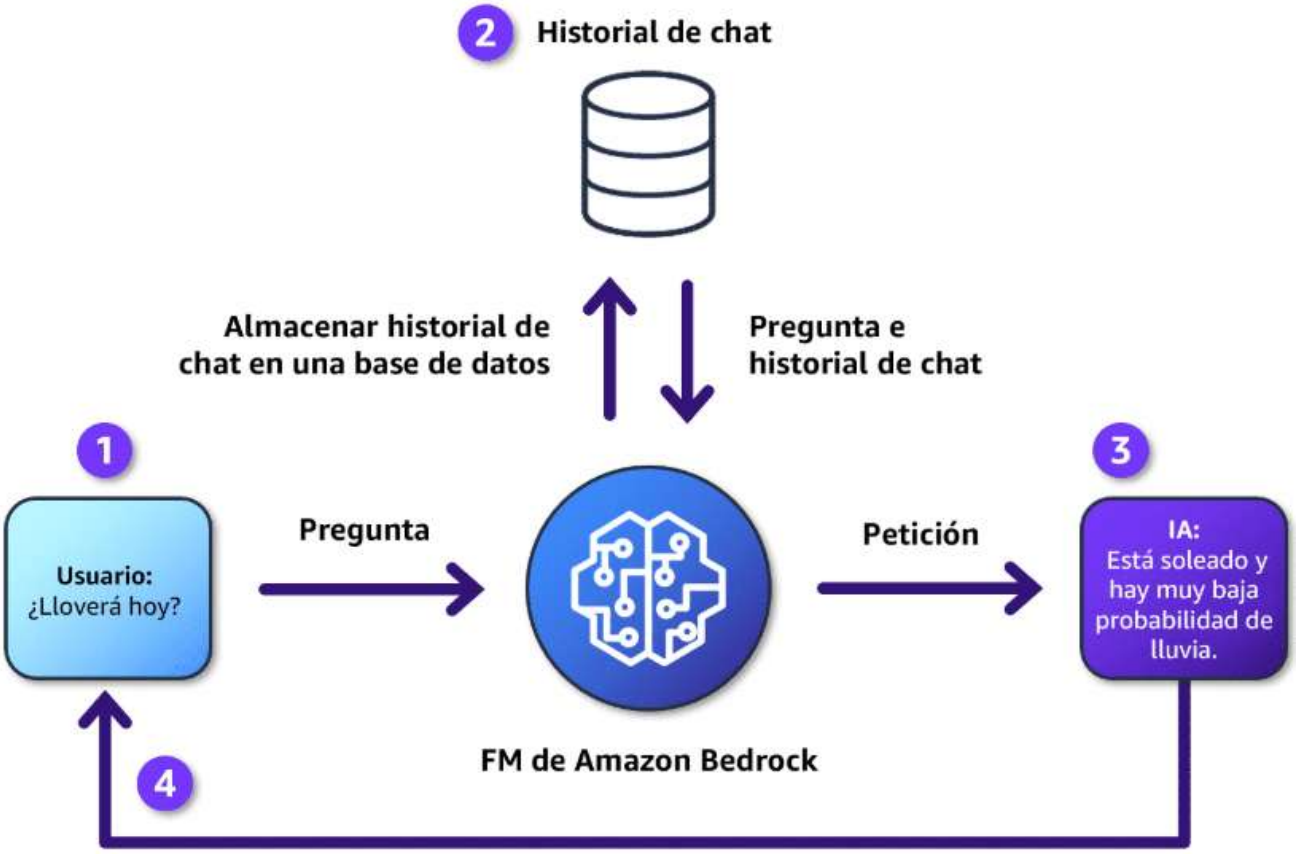
  Uno de los desafíos de la respuesta a preguntas es el número limitado de tokens que permite el contexto. Este problema se soluciona usando la RAG. El uso del patrón de arquitectura para casos de uso personalizados y específicos dará lugar a respuestas más fiables y precisas.

  Los casos de uso personalizados requieren respuestas específicas y minuciosidad en distintos dominios, por lo que no es suficiente utilizar un modelo base. Puede usar la técnica de RAG con el fin de ubicar los fragmentos relevantes de texto que mejor se adapten para responder la pregunta del usuario. A continuación, la RAG concatena el mejor conjunto de cadenas de los fragmentos relevantes recuperados y lo suministra como contexto. Luego, el contexto se pasa como entrada adicional al FM.

Patrón de asistente de IA.

Puede usar interfaces conversacionales, como asistentes de IA, a fin de mejorar la experiencia del usuario para los clientes. Los asistentes de IA usan algoritmos de NLP y ML para comprender y responder las consultas de los usuarios. Puede usar asistentes de IA en varias aplicaciones, como servicio al cliente, ventas y comercio electrónico, para ofrecer respuestas rápidas y eficientes a los usuarios. Puede acceder a los asistentes de IA desde diferentes canales, como sitios web, plataformas de redes sociales y aplicaciones de mensajería.

En el diagrama siguiente, se ilustra un patrón de arquitectura básico de un caso de uso de asistente de IA con Amazon Bedrock.



Conversación de usuario almacenada en un almacén de historial de chat.

Esta arquitectura incluye los pasos siguientes:

1. El usuario consulta al asistente de IA.
2. El historial de chat (si lo hay) se pasa al modelo de Amazon Bedrock junto con la consulta actual del usuario.
3. Luego, el modelo genera una respuesta.

4. El modelo devuelve la respuesta al usuario.

En este caso de uso básico, el usuario puede introducir una petición específica, como una pregunta a Amazon Bedrock. El FM almacena el chat, o las preguntas y respuestas generadas, en un historial de chat. A partir del historial del chat y de la petición actual que realizó el usuario, Amazon Bedrock proporciona una respuesta precisa y útil.

Casos de uso del asistente de IA.

Puede clasificar los casos de uso del asistente de IA de la siguiente manera.

- **Asistente de IA básico:** se trata de un asistente de IA sin entrenamiento previo con un modelo FM.
- **Asistente de IA que usa una plantilla de peticiones:** se trata de un asistente de IA con algo de contexto proporcionado en la plantilla de peticiones.
- **Asistente de IA con personalidad**: se trata de un asistente de IA con funciones definidas, como un entrenador profesional con interacciones humanas.
- **Asistente de IA consciente del contexto:** se trata de un asistente de IA que transfiere el contexto desde un archivo externo mediante la generación de incrustaciones.

MÓDULO 4: USAR LANGCHAIN

INTRODUCCIÓN

OPTIMIZACIÓN DEL RENDIMIENTO DE LOS LLM

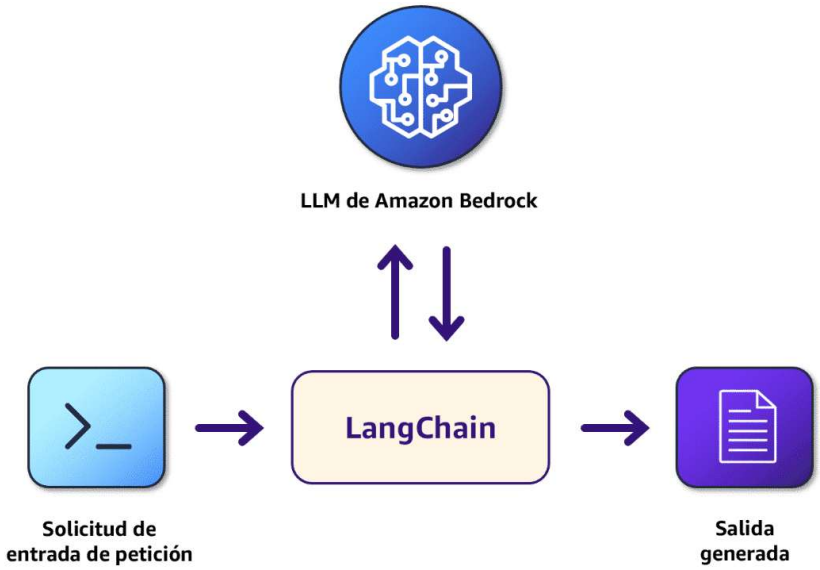
Desafíos de rendimiento de los LLM

Los modelos de lenguaje de gran tamaño (LLM) se preentrenan con grandes colecciones de datos y pueden realizar varias tareas, como generar texto, resumir texto, responder preguntas y realizar análisis de sentimiento. Sin embargo, los modelos no tienen buen rendimiento cuando las tareas requieren tratar con datos fuera del dominio o recordar el contexto conversacional.

Estos desafíos llevan a alucinaciones o respuestas imprecisas. Es posible que una sola petición al LLM no siempre proporcione el resultado esperado. Puede ser necesario el encadenamiento de solicitudes al modelo para producir resultados precisos.

Simplificación del desarrollo de LLM con LangChain.

LangChain es un marco de trabajo para desarrollar aplicaciones impulsadas por LLM. LangChain proporciona los bloques de creación de software para reducir la complejidad de crear funcionalidades desde cero. LangChain simplifica todas las etapas del ciclo de vida de las aplicaciones LLM, incluidos el desarrollo, la producción y la implementación. Puede usar LangChain para aprovechar al máximo un LLM.

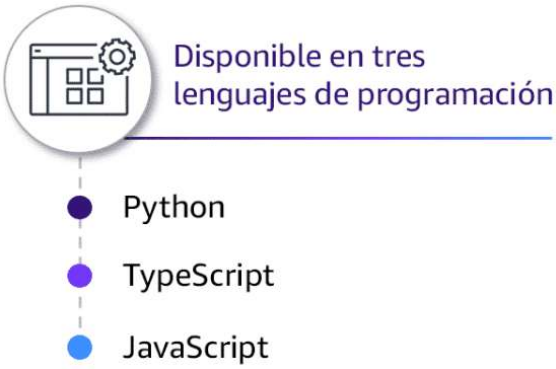


Los LLM no retienen el estado entre invocaciones, por lo que la aplicación debe administrar cualquier contexto. Por ejemplo, a fin de brindar una experiencia de chat, el LLM necesita la conversación como parte del contexto para producir los resultados correctos. Además, los LLM pueden desarrollar el razonamiento para resolver problemas, como los problemas multipaso: la aplicación necesita encontrar información en los pasos para resolver un problema. LangChain proporciona componentes para que realizar las tareas comunes de administración de contexto o la secuenciación de pasos al interactuar con un LLM sea más eficiente.

Utilización de componentes de LangChain.

LangChain es de código abierto y actualmente está disponible en tres lenguajes de programación: Python, TypeScript y JavaScript. LangChain consta de numerosos componentes y este módulo se centra en los modelos, las plantillas de peticiones, los índices, la memoria, las cadenas y los agentes.

Puede usar estos componentes para crear aplicaciones, como chatbots de respuesta a preguntas basados en la generación aumentada por recuperación (RAG). También puede usar estos componentes para resumir textos, generar código, extraer información, analizar consultas e interactuar con las API.



Puede usar los componentes de LangChain para conectar los LLM con otros orígenes de datos con el fin de mejorar la exactitud de las respuestas. Para comenzar, puede usar LangChain Expression Language (LCEL), una forma declarativa de encadenar los componentes de LangChain y compilarlos desde allí. Para obtener más información acerca de los componentes de LangChain y las soluciones prediseñadas.

INTEGRACIÓN DE AWS Y LANGCHAIN

Integraciones compatibles con AWS.

LangChain admite integraciones en varios proveedores de software, incluido Amazon. Algunas de las integraciones más importantes relacionadas con Amazon incluyen modelos de chat, LLM, modelos de incrustación de texto, cargadores de documentos, almacenes de vectores, recuperadores, herramientas, memoria, gráficos, devoluciones de llamadas y cadenas.

Integraciones de Amazon Bedrock:

LangChain admite la integración para Amazon Bedrock con los paquetes **langchain.aws** y **langchain-community**. Actualmente, Amazon Bedrock es compatible con los modelos Titan Text de Amazon, Jurassic de AI21, Claude de Anthropic, Command y Embed de Cohere, Llama de Meta, Stable Diffusion de Stability AI y Mistral de Mistral AI.



API de integración de AWS LangChain

COMPONENTES DE LANGCHAIN

USO DE MODELOS CON LANGCHAIN

En esta sección, aprenderá sobre los modelos de lenguaje que se usan en LangChain. Estos modelos incluyen LLM, modelos de chat y modelos de incrustación de texto.

LLM

Los LLM toman texto como entrada y generan texto como salida, y LangChain proporciona componentes de LLM para interactuar con distintos modelos de lenguaje. La clase de LLM es una abstracción para trabajar con diferentes proveedores, y LangChain la usa para interactuar con un modelo de LLM.



En el siguiente ejemplo, se muestra cómo crear una instancia de la clase Amazon Bedrock e invocar un LLM de Amazon Titan desde el módulo LLM de LangChain. El model_id es el valor del modelo seleccionado disponible en Amazon Bedrock.

```
import boto3
from langchain_aws import BedrockLLM
bedrock_client = boto3.client('bedrock-runtime',region_name="us-east-1")
inference_modifiers = {"temperature": 0.3, "maxTokenCount": 512}
llm = BedrockLLM(
  client = bedrock_client,
  model_id="amazon.titan-tg1-large",
  model_kwargs =inference_modifiers
  streaming=True,
)
response = llm.invoke("What is the largest city in Vermont?")
print(response)
```

Modelos personalizados.

Amazon Bedrock puede utilizarse para personalizar los modelos fundacionales (FM) con el fin de mejorar su rendimiento y crear una mejor experiencia del cliente para casos de uso específicos. Puede hacerlo con el entrenamiento previo continuo o el ajuste fino de FM.

En el siguiente ejemplo, se muestra cómo obtener una respuesta de un modelo personalizado de Amazon Bedrock pasando una solicitud de usuario al LLM.

```
import boto3
from langchain_aws import BedrockLLM

custom_llm = BedrockLLM(
 credentials_profile_name = "bedrock-admin",
 provider = "cohere",
 model_id="<Custom model ARN>",  # ARN como “arn:aws:bedrock:..” obtenido a partir de un modelo aprovisionado
 model_kwargs ={"temperature": 1},
 streaming=True,
)
response = custom_llm.invoke("What is the recipe for mayonnaise?")
```

```
print(response)
```

Modelos de chat.

Las interfaces conversacionales, como los chatbots y asistentes virtuales, pueden reducir el costo de la asistencia al cliente y mejorar su experiencia. LangChain proporciona un componente de modelos de chat para crear aplicaciones conversacionales. Este componente acepta contenido en forma de mensajes que contienen texto de entrada.



Ejemplo de modelos de chat:

En el siguiente ejemplo, se muestra cómo puede obtener una respuesta de un LLM tras pasarle la solicitud de un usuario.

Entrada:

```
from langchain_aws import ChatBedrock as Bedrock
from langchain.schema import HumanMessage
chat = Bedrock(model_id="anthropic.claude-3-sonnet-20240229-v1:0", model_kwargs={"temperature":0.1})

messages = [
    HumanMessage(
        content="I would like to try Indian food, what do you suggest should I try?"
    )
]
chat.invoke(messages)
```

Ejemplo de respuesta:

Response:
AIMessage(content="Here are some delicious and popular Indian dish recommendations for someone trying Indian food for the first time:\n\n- Butter Chicken (Murgh Makhani) - Tender chicken in an aromatic tomato-based curry with butter and cream. It's flavorful but not too spicy.\n\n- Chicken Tikka Masala - Chunks of roasted marinated chicken in a creamy tomato-based sauce.\n\n- Palak Paneer - A vegetarian dish with paneer (fresh cheese cubes) cooked in a thick spinach-based curry.\n\n- Chana Masala - A vegetarian chickpea curry flavored with spices like cumin, coriander and garam masala.\n\n- Samosas - Fried or baked pastry pockets stuffed with a savory potato/vegetable filling.\n\n- Naan - Leavened oven-baked flatbread, perfect for soaking up curries. Try garlic or butter naan.\n\n- Biryani - A flavorful mixed rice dish with meat/vegetables and aromatic spices.\n\nI'd recommend getting a combination platter or thali to sample a variety of dishes on your first visit. Start with milder dishes and work your way up if you want more heat/spice. Indian food has incredible depth of flavor!", additional_kwargs={'usage':
{'prompt_tokens': 23, 'completion_tokens': 300, 'total_tokens': 323}}, response_metadata={'model_id':
'anthropic.claude-3-sonnet-20240229-v1:0', 'usage':
{'prompt_tokens': 23, 'completion_tokens': 300, 'total_tokens': 323}}, id='run-09f0ef8a-90c0-47c3-b172-546b084d1288-0')

Modelos de incrustación de texto.

Los modelos de incrustación de texto toman texto como entrada y, luego, generan como salida representaciones numéricas del texto en forma de un vector de números de punto flotante. Las representaciones numéricas se denominan incrustaciones de palabras y capturan el sentido semántico del texto.

Las incrustaciones se utilizan en diversas tareas de procesamiento de lenguaje natural (NLP), como el análisis de sentimiento, la clasificación de textos y la recuperación de información. Puede guardar las incrustaciones en una base de datos vectorial para mejorar la exactitud de búsqueda y acelerar la recuperación.



Ejemplo de incrustación:

En el siguiente ejemplo, se muestra cómo llamar a un cliente BedrockEmbeddings para enviar texto al modelo Amazon Titan Embeddings a fin de obtener incrustaciones como respuesta.

```
from langchain_community.embeddings import BedrockEmbeddings

embeddings = BedrockEmbeddings(
  region_name="us-east-1",
  model_id="amazon.titan-embed-text-v1"
)

embeddings.embed_query("Cooper is a puppy that likes to eat beef")
```

 Es importante comprender que el vector de incrustación representa la semántica de la frase, no de la cadena. Esto significa que los sinónimos darán como resultado vectores con mayor similitud, incluso si la cadena es bastante diferente.

En el siguiente ejemplo, la oración de referencia trata sobre un perro al que le gusta comer carne. Se compara una lista de frases candidatas con la referencia y se ordenan por similitud, donde 0 es lo más similar y 1 es lo menos similar.

Referencia: Cooper es un perro al que le gusta comer carne.

Puntuación	Texto
0,035882	**Cooper es un cachorro al que le gusta comer carne.**
0,083690	**La carne es lo que le gusta a un perro llamado Cooper.**
0,109997	Cooper es un perro al que le gusta comer filete.
0,135489	Cooper es un perro que odia comer carne.
0,184443	Cooper es un perro al que le gusta comer pollo.
0,192230	Fido es un perro al que le gusta comer carne.
0,194594	Cooper es un gato al que le gusta comer carne.
0,228607	Spot es un perro al que le gusta comer carne.

0,264782	Cooper es un hombre al que le gusta comer carne.
0,351404	***Cooper ist ein Hund, der gerne Rindfleisch frisst.***
0,799773	Un barrilero es una persona que fabrica barriles.
1,016293	Amazon Web Services ofrece API y soluciones de computación en la nube bajo demanda con un sistema de pago por uso.

En la tabla anterior, se incluyen diversos cambios de la referencia.

- **Reemplazo de una palabra:**
  En la tabla, al reemplazar *perro* por *cachorro* se produce el resultado más similar, que es *Cooper es un cachorro al que le gusta comer carne*.

- **Cambio en la estructura de la oración:**
  Sin embargo, el siguiente resultado más similar implica cambiar la estructura de la oración a *La carne es lo que le gusta a un perro llamado Cooper*. En este resultado, el orden de las palabras es completamente diferente al de la referencia, y la comparación de cadenas es una de las menos similares.

- **Asociación del significado entre idiomas:**
  Además, la traducción en alemán, *Cooper ist ein Hund, der gerne Rindfleisch frisst*, es significativamente más similar a la referencia que *Un barrilero es una persona que fabrica barriles*. Las incrustaciones pueden asociar el significado entre idiomas y entre diferentes significados de las palabras, lo que las convierte en algo poderoso para la búsqueda y la comparación de textos.

ELABORACIÓN DE PETICIONES

La petición es una instrucción de texto única que se da al LLM como entrada para obtener una respuesta. Al crear aplicaciones de inteligencia artificial generativa (IA generativa) complejas, es posible que deba elaborar peticiones de una manera específica. Puede incluir instrucciones, contexto o ejemplos, y rellenar de forma dinámica variables de entrada a fin de guiar al LLM para que devuelva una salida que cumpla con sus requisitos.

Plantilla de peticiones LangChain.

LangChain proporciona plantillas de peticiones predefinidas en forma de cadenas de texto que pueden tomar un conjunto de parámetros del usuario y generar una petición. Las plantillas de peticiones hacen que la ingeniería de peticiones sea más eficiente y posibilitan la reutilización de peticiones.



Ejemplo de petición.

En el siguiente ejemplo, se muestra la creación de una plantilla de peticiones, el paso de variables de entrada y plantillas como argumentos.

```
from langchain import PromptTemplate

# Crear una plantilla de petición que tenga múltiples variables de entrada
multi_var_prompt = PromptTemplate(
    input_variables=["customerName", "feedbackFromCustomer"],
    template="""
    Human: Create an email to {customerName} in response to the following customer service feedback that
was received from the customer:
    <customer_feedback>
        {feedbackFromCustomer}
    </customer_feedback>
    Assistant:"""
)
# Pasar los valores a las variables de entrada
prompt = multi_var_prompt.format(customerName="John Doe",
        feedbackFromCustomer="""Hello AnyCompany,
    I am very pleased with the recent experience I had when I called your customer support.
```

```
    I got an immediate call back, and the representative was very knowledgeable in fixing the problem.
    We are very happy with the response provided and will consider recommending it to other businesses.
    """
)
```

ESTRUCTURACIÓN DE DOCUMENTOS CON ÍNDICES

Puede usar índices para estructurar documentos y optimizar la forma en que los LLM interactúan con ellos. En esta sección, aprenderá sobre los índices importantes que pueden ayudar con las integraciones relevantes para AWS: cargadores de archivos, recuperadores y almacenes de vectores.

Cargadores de documentos.

Al desarrollar aplicaciones de IA generativa con el enfoque de RAG, es necesario cargar documentos desde diferentes fuentes en los modelos de lenguaje para generar incrustaciones.

LangChain tiene un componente de cargadores de documentos, que es el responsable de cargar los documentos desde distintas fuentes. Las fuentes pueden incluir una base de datos, un almacenamiento en línea o un almacenamiento local. Puede indexar y usar información de estas fuentes para la recuperación de información. Puede usar los cargadores de documentos para cargar distintos tipos de documentos, como HTML, PDF y código.



Ejemplos de cargadores de documentos:

En el siguiente ejemplo, se muestra la carga de un documento desde Amazon Simple Storage Service (Amazon S3) mediante el módulo S3FileLoader.

```
from langchain.document_loaders import S3FileLoader

loader = S3FileLoader("mysource_bucket","sample-file.docx")
data = loader.load()
```

Recuperador.

LangChain incluye un componente recuperador a fin de recuperar documentos relevantes para combinarlos con modelos de lenguaje. Cuando un usuario envía una consulta, el recuperador busca en el índice del documento para encontrar los documentos más relevantes. A continuación, envía los resultados a la aplicación para su posterior procesamiento.

Al crear aplicaciones de RAG en AWS, puede usar Amazon Kendra para indexar y consultar diversos orígenes de datos. Amazon Kendra es un servicio completamente administrado que proporciona capacidades de búsqueda semántica para una clasificación de vanguardia de documentos y fragmentos.



Amazon Kendra también incluye conectores prediseñados para orígenes de datos populares, como Amazon S3, SharePoint, Confluence y sitios web. Es compatible con formatos de documentos comunes, como HTML, Microsoft Word, PowerPoint, PDF, Excel y PureText.

LangChain admite distintos algoritmos de recuperación. Puede usar el método AmazonKendraRetriever para recuperar documentos de Amazon Kendra.

Ejemplo de recuperador:

En el siguiente ejemplo, se muestra el uso de AmazonKendraRetriever para consultar un índice de Amazon Kendra y pasar los resultados de esa llamada a un LLM como contexto junto con una petición.

```
from langchain_aws.retrievers import AmazonKendraRetriever
from langchain.chains import ConversationalRetrievalChain
from langchain.prompts import PromptTemplate
from langchain_aws import ChatBedrock


llm = ChatBedrock(
  model_kwargs={"max_tokens_to_sample":300,"temperature":1,"top_k":250,"top_p":0.999,"anthropic_versio
n":"bedrock-2023-05-31"},
  model_id="anthropic.claude-3-sonnet-20240229-v1:0"
)

retriever = AmazonKendraRetriever(index_id=kendra_index_id,top_k=5,region_name=region)

prompt_template = """ Human: This is a friendly conversation between a human and an AI.
The AI is talkative and provides specific details from its context but limits it to 240 tokens.
If the AI does not know the answer to a question, it truthfully says it
does not know.

Assistant: OK, got it, I'll be a talkative truthful AI assistant.

Human: Here are a few documents in <documents> tags:
<documents>
{context}
</documents>
Based on the above documents, provide a detailed answer for, {question}
Answer "do not know" if not present in the document.

Assistant:
 """

PROMPT = PromptTemplate(
  template=prompt_template, input_variables=["context", "question"]
```

```
)

response = ConversationalRetrievalChain.from_llm(
  llm=llm,
  retriever=retriever,
  return_source_documents=True,
  combine_docs_chain_kwargs={"prompt":PROMPT},
  verbose=True)
```

Almacenes de vectores.

Al desarrollar aplicaciones de IA generativa, como bots de respuesta a preguntas, los modelos de lenguaje generan resultados precisos cuando se les proporciona información relevante de la empresa o del usuario como contexto.



Puede usar el enfoque de RAG y seguir estos pasos:

1. Puede convertir en incrustaciones los datos específicos de la empresa, como documentos, con el modelo de incrustaciones de texto (descrito en la sección anterior).
2. Luego, puede almacenar las incrustaciones en una base de datos vectorial.
3. Puede extraer los documentos pertinentes de la base de datos vectorial según la solicitud del usuario.
4. A continuación, puede pasarlos al LLM como contexto para obtener una respuesta precisa.

LangChain admite tanto almacenes vectoriales de código abierto como específicos de un proveedor. LangChain es compatible con el motor vectorial para Amazon OpenSearch sin servidor y la extensión pgvector disponible con la edición compatible con Amazon Aurora PostgreSQL. LangChain proporciona el componente de almacenes de vectores para consultar los datos relevantes en los almacenes de vectores compatibles.

Ejemplo de almacén de vectores:

En el siguiente ejemplo, se muestra cómo crear una instancia de la clase VectorStore y utilizar OpenSearch sin servidor como almacén de vectores para consultar las incrustaciones.

```
import os
from langchain_community.embeddings import BedrockEmbeddings
from langchain_community.vectorstores import OpenSearchVectorSearch

index_name = os.environ["AOSS_INDEX_NAME"]
endpoint = os.environ["AOSS_COLLECTION_ENDPOINT"]

embeddings = BedrockEmbeddings(client=bedrock_client)

vector_store = OpenSearchVectorSearch(
  index_name=index_name,
  embedding_function=embeddings,
  opensearch_url=endpoint,
  use_ssl=True,
  verify_certs=True,
)
retriever = vector_store.as_retriever()
```

ALMACENAMIENTO Y RECUPERACIÓN DE DATOS CON MEMORIA

La memoria cumple un rol crucial en la creación de interfaces conversacionales como asistentes de IA. Los asistentes de IA impulsados por modelos de lenguaje pueden responder de un modo similar a los humanos si recuerdan las interacciones anteriores con el usuario. Sin embargo, los LLM en sí no mantienen el estado. Por lo tanto, una petición debería incluir el contexto de la conversación.

Memoria de LangChain.

La memoria de LangChain proporciona el mecanismo para almacenar y resumir (en caso de ser necesario) elementos conversacionales incluidos en el contexto en invocaciones posteriores. LangChain ofrece componentes en forma de utilidades de ayuda para administrar y manipular mensajes de chat anteriores. Estas

utilidades son modulares. Puede encadenarlas con otros componentes e interactuar con distintos tipos de abstracciones para crear chatbots potentes.



En LangChain, existen diferentes formas de implementar la memoria conversacional para un chatbot:

- **ConversationBufferMemory:** es el tipo de memoria más común en LangChain. Incluye conversaciones previas que tuvieron lugar entre el usuario y el LLM.
- **ConversationChain:** ConversationBufferMemory se desarrolla a partir de ConversationChain, diseñada para administrar conversaciones.

Ejemplo de memoria.

En el siguiente ejemplo, se muestra el uso de ConversationBufferMemory para almacenar la conversación previa y hacer que el LLM responda a preguntas posteriores mediante el historial de chat.

```
from langchain.chains import ConversationChain
from langchain_aws import BedrockLLM
from langchain.memory import ConversationBufferMemory
bedrock_client = boto3.client('bedrock-runtime',region_name="us-east-1")



titan_llm = BedrockLLM(model_id="amazon.titan-tg1-large", client=bedrock_client)
memory = ConversationBufferMemory()

conversation = ConversationChain(
  llm=titan_llm, verbose=True, memory=memory
)
```

```
print(conversation.predict(input="Hi! I am in Los Angeles. What are some of the popular sightseeing places?"))
```

Formule una pregunta sin mencionar la ciudad de Los Ángeles para ver cómo responde el modelo conforme a la conversación previa.

```
print(conversation.predict(input="What is closest beach that I can go to?"))
```

USO DE CADENAS PARA CREAR UNA SECUENCIA DE COMPONENTES

Para crear aplicaciones de IA generativa complejas, puede encadenar los LLM con otros componentes de LangChain, como memoria y recuperadores. Por ejemplo, puede encadenar dos tareas diferentes. Primero, pida al LLM que escriba un blog sobre un tema en particular. Segundo, solicite al modelo que proporcione un título para el blog.

Encadenamiento de componentes.

Básicamente, una cadena es un conjunto de componentes que se ponen en marcha juntos. El componente puede ser una llamada a un LLM, una API o una secuencia de otras cadenas. El componente tiene un formato de entrada y una salida. Por ejemplo, LLMChain recibe un LLM, una plantilla de peticiones y parámetros para la plantilla de peticiones. Devuelve la salida de la llamada del LLM. La cadena puede analizar la salida del LLM en un formato específico y devolver los datos de forma más estructurada.



Una cadena también puede poner en marcha un conjunto de cadenas, de modo que puede crear un flujo más complejo con varias llamadas a un LLM. En este caso, se usan los resultados de una llamada como entrada para la siguiente.

Procesamiento de grandes cantidades de datos.

Las cadenas también son útiles a la hora de procesar más datos de los que puede contener el contexto. Por ejemplo, puede hacer un resumen de un documento muy grande. La cadena dividirá el documento y realizará varias llamadas al LLM para producir un solo resumen.

Las cadenas se refieren a secuencias de llamadas, ya sea a un LLM, a una herramienta o a un paso de preprocesamiento de datos. LangChain Expression Language (LCEL) es la principal forma de interactuar con las cadenas, y es ideal para construir cadenas. Sin embargo, existen dos tipos de cadenas estándar compatibles con LangChain:

- Cadena construida con LCEL.
- Cadenas heredadas construidas por subclasificación a partir de una clase "Cadena" heredada, es decir, LLMChain, etc.

Ejemplo de cadenas.

En el siguiente ejemplo, se muestra cómo usar cadenas para llamar a un LLM varias veces en una secuencia con LLMChain.

```
from langchain import PromptTemplate
from langchain.chains import LLMChain
from langchain_aws import ChatBedrock as Bedrock

chat = Bedrock(
    region_name = "us-east-1",
    model_kwargs={"temperature":1,"top_k":250,"top_p":0.999,"anthropic_version":"bedrock-2023-05-31"},
    model_id="anthropic.claude-3-sonnet-20240229-v1:0"
)

multi_var_prompt = PromptTemplate(
    input_variables=["company"],
    template="Create a list with the names of the main metrics tracked in the reports of {company}?",
)

chain = LLMChain(llm=chat, prompt=multi_var_prompt)
answers = chain.invoke("Amazon")
```

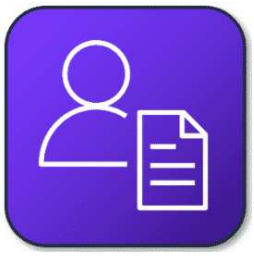
```
print(answers)


answers = chain.invoke("AWS")
print(answers)
```

ADMINISTRACIÓN DE RECURSOS EXTERNOS CON AGENTES DE LANGCHAIN

Los LLM están preentrenados con una enorme cantidad de datos de texto y aprenden de una gran variedad de patrones. Por lo tanto, pueden generar respuestas de texto efectivas. Sin embargo, no tienen la capacidad de resolver tipos de solicitudes matemáticas, lógicas y de razonamiento basadas en reglas.

Los agentes de LangChain pueden interactuar con fuentes externas, como motores de búsqueda, calculadoras, API o bases de datos. Los agentes también pueden ejecutar código para realizar acciones que ayuden a los LLM en la generación de respuestas precisas. LangChain proporciona la interfaz de cadena que permite elaborar una secuencia de varios componentes a fin de crear una aplicación. Una LLMChain es un ejemplo de una cadena básica.



Los siguientes mecanismos usan cadenas básicas:

- Una **aplicación de RAG** puede usar una cadena LLMChain para devolver una respuesta. La respuesta se basa en dos datos: la consulta del usuario y un contexto proporcionado como un conjunto de documentos recuperado de un almacén de vectores.
- Una **RouterChain** es un ejemplo de una cadena compleja. Por ejemplo, puede usar una cadena RouterChain para seleccionar una plantilla de peticiones entre las plantillas de peticiones disponibles conforme a la entrada del usuario.
- Los **agentes de LangChain** actúan como motores de razonamiento para los LLM a fin de decidir las acciones que se tomarán y el orden en el cual se llevarán a cabo. Una acción puede ser una herramienta que use los resultados de un motor de búsqueda o una calculadora matemática.

Características de los agentes de LangChain.

El agente de LangChain sigue una secuencia de acciones:

1. LangChain proporciona herramientas y conjuntos de herramientas (un grupo de tres a cinco herramientas diferentes) en forma de funciones para que el agente los llame. El agente se forma con acceso a un LLM, un conjunto de herramientas y una condición de detención.
2. El agente usa el marco de trabajo de peticiones ReAct (razonar y actuar) para elegir la herramienta más relevante del conjunto de herramientas proporcionado según la entrada del usuario.
3. El agente decide de forma reiterada la acción, la realiza y observa la salida de la herramienta hasta que se logra la condición de detención.

 LangChain tiene herramientas incorporadas, como LLMMathChain, y usted también puede crear sus propias herramientas para realizar acciones específicas, como una búsqueda en una base de datos o una llamada a la API.

Ejemplo de agente.

En el siguiente ejemplo, se muestra cómo inicializar un agente, una herramienta y un LLM para formar una cadena y hacer que el agente de ReAct ZERO_SHOT llame a la herramienta incorporada LLMMathChain para realizar cálculos matemáticos por separado y pasar el resultado al LLM para la respuesta final.

```
from langchain.agents import load_tools
from langchain.agents import initialize_agent, Tool
from langchain.agents import AgentType
from langchain import LLMMathChain
from langchain_aws import ChatBedrock
from langchain.agents import AgentExecutor, create_react_agent


chat = ChatBedrock(model_id="anthropic.claude-3-sonnet-20240229-v1:0",
model_kwargs={"temperature":0.1})
prompt_template = """Answer the following questions as best you can.
You have access to the following tools:\n\n{tools}\n\n
Use the following format:\n\nQuestion: the input question you must answer\n
Thought: you should always think about what to do\n
```

```
Action: the action to take, should be one of [{tool_names}]\n
Action Input: the input to the action\nObservation: the result of the action\n...
(this Thought/Action/Action Input/Observation can repeat N times)\n
Thought: I now know the final answer\n
Final Answer: the final answer to the original input question\n\nBegin!\n\n
Question: {input}\nThought:{agent_scratchpad}
"""
modelId = "anthropic.claude-3-sonnet-20240229-v1:0"

react_agent_llm = ChatBedrock(model_id=modelId, client=bedrock_client)
math_chain_llm = ChatBedrock(model_id=modelId, client=bedrock_client)
tools = load_tools([], llm=react_agent_llm)

llm_math_chain = LLMMathChain.from_llm(llm=math_chain_llm, verbose=True)

llm_math_chain.llm_chain.prompt.template = """Human: Given a question with a math problem, provide only a single line mathematical expression that solves the problem in the following format. Don't solve the expression only create a parsable expression.
```text
{{single line mathematical expression that solves the problem}}
```

Assistant:
Here is an example response with a single line mathematical expression for solving a math problem:
```text
37593**(1/5)
```

Human: {question}
Assistant:"""

tools.append(
  Tool.from_function(
```

```
        func=llm_math_chain.run,
        name="Calculator",
        description="Useful for when you need to answer questions about math.",
    )
)

react_agent = create_react_agent(react_agent_llm,
    tools,
    PromptTemplate.from_template(prompt_template)
        # max_iteration=2,
        # return_intermediate_steps=True,
        # handle_parsing_errors=True,
    )

agent_executor = AgentExecutor(
agent=react_agent,
tools=tools,
verbose=True,
handle_parsing_errors=True,
max_iterations = 10 # útil cuando el agente está detenido en un bucle
)

agent_executor.invoke({"input": "What is the distance between San Francisco and Los Angeles? If I travel from San Francisco to Los Angeles with the speed of 40MPH how long will it take to reach?"})
```


Anteriormente LangChain utilizaba AgentExecutor como tiempo de ejecución para los agentes, pero no era flexible para agentes personalizados. Para resolver este problema, LangChain ha creado LangGraph (una extensión de LangChain), que proporciona un tiempo de ejecución altamente controlable para los agentes. Puede seguir usando AgentExecutor como tiempo de ejecución, pero se recomienda iniciar la transición a LangGraph.

MÓDULO 5: PATRONES DE ARQUITECTURA

INTRODUCCIÓN

INTRODUCCIÓN A LOS PATRONES DE ARQUITECTURA

Patrones de arquitectura.

En este módulo, aprenderá sobre diversos patrones de arquitectura que se pueden implementar con Amazon Bedrock para crear aplicaciones de IA generativa como las siguientes:

- Generación de texto
- Resumen de texto
- Respuesta a preguntas
- Chatbots
- Generación de código
- Agentes de LangChain
- Agentes para Amazon Bedrock



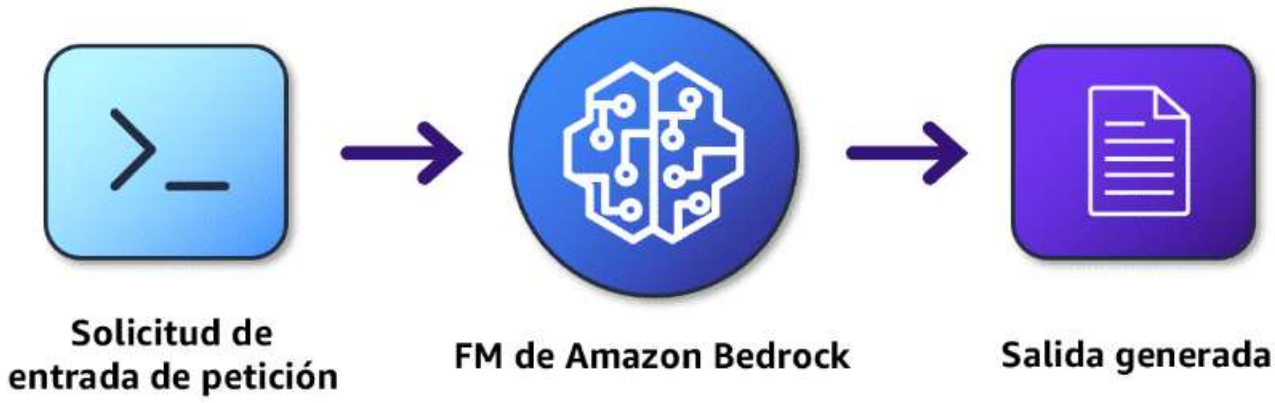
PATRONES DE ARQUITECTURA PARA DESARROLLAR APLICACIONES DE IA GENERATIVA

GENERACIÓN DE TEXTO Y RESUMEN DE TEXTO

Generación de texto.

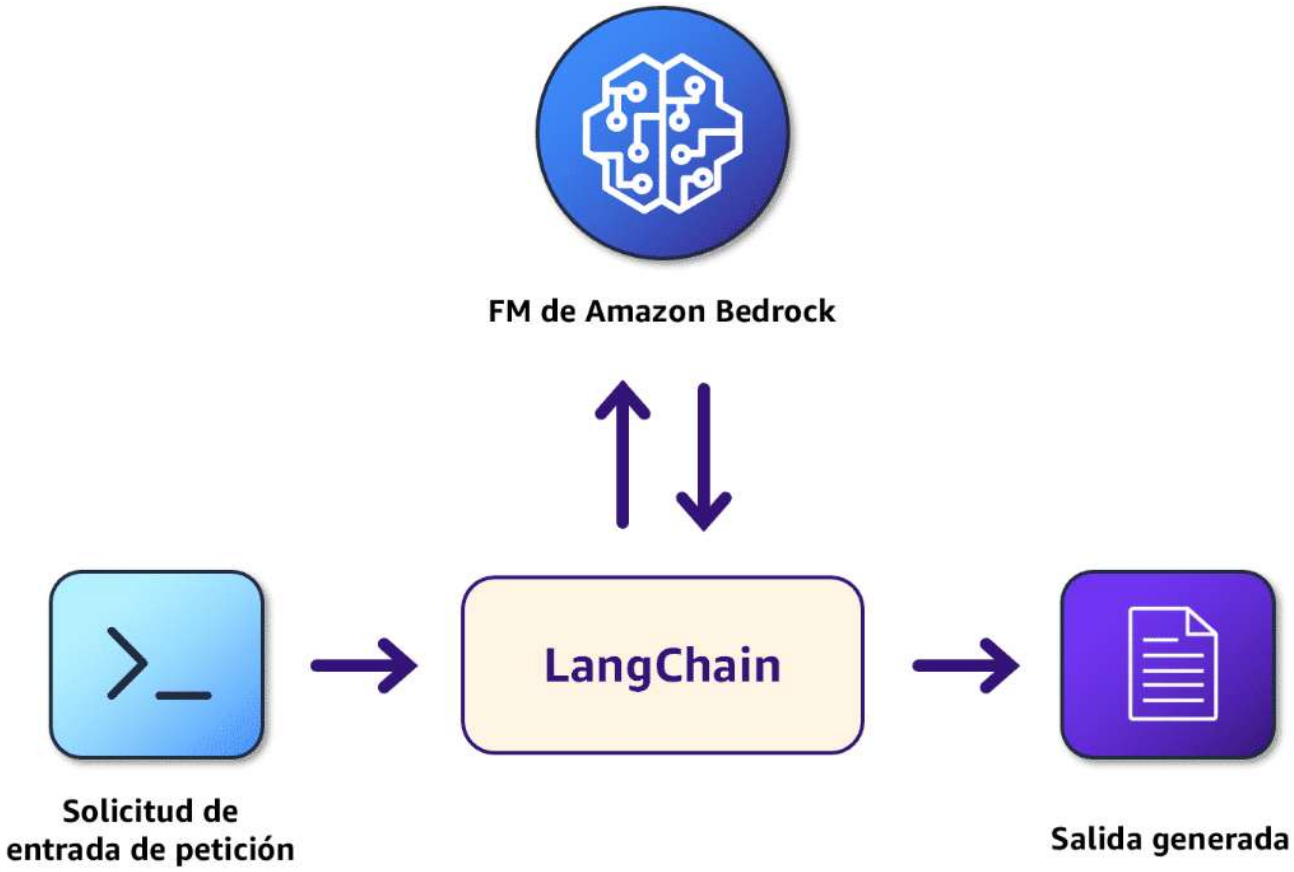
Generación de texto es un término que se usa para cualquier caso de uso en el que la salida del modelo es texto recién generado. Puede usarse para escribir artículos, poemas, blogs, libros, correos electrónicos, etc. En Amazon Bedrock, puede usar varios modelos fundacionales (FM) para las tareas de generación de texto.

El patrón de arquitectura para la generación de texto con Amazon Bedrock se ilustra en la imagen siguiente. Puede pasar al modelo fundacional de Amazon Bedrock una petición usando un área de pruebas de Amazon Bedrock o una llamada a la API de Amazon Bedrock. El modelo generará texto conforme a la petición de entrada que le proporcione.



Generación de texto con LangChain:

Para la generación de texto, también puede usar una capa de LangChain a fin de agregar una cadena de conversación a casos de uso de generación de texto específicos. LangChain es una potente biblioteca de código abierto. Funciona bien con algunos de los FM de generación de texto más sólidos de Amazon Bedrock para crear de manera eficiente conversaciones, generación de texto y más. El patrón de arquitectura para la generación de texto con LangChain se ilustra en la figura siguiente.



Resumen de texto.

El resumen de textos es una tarea de procesamiento de lenguaje natural (NLP) que condensa el texto de una entrada dada y preserva la información y el significado claves del texto. A continuación, se muestran las dos formas de hacer un resumen:

- Seleccionar un subconjunto de texto de la entrada que represente las ideas clave.
- Crear nuevas oraciones que capturen los conceptos y elementos clave del texto fuente.

Con los avances en la IA generativa y los modelos de lenguaje de gran tamaño (LLM), se han producido mejoras significativas en el ámbito del resumen de textos. Los LLM son adecuados para la tarea de generar resúmenes debido a su efectividad para comprender y sintetizar texto. En esta sección, explorará conceptos, aplicaciones y patrones de arquitectura relacionados con el resumen de texto con modelos de lenguaje de gran tamaño.

Técnicas de resumen de texto.

Existen dos tipos principales de técnicas de resumen de texto: extractiva y abstractiva. Para obtener más información sobre estas técnicas, seleccione cada una de las dos pestañas a continuación.

- **Resumen extractivo:**

  Esta técnica involucra identificar y seleccionar las palabras, frases u oraciones más importantes del documento fuente y, luego, concatenarlas para formar un resumen. Los elementos seleccionados suelen ser las partes más informativas y representativas del texto.

- **Resumen abstractivo:**

  Esta técnica involucra generar texto nuevo que *no* sea una paráfrasis del documento fuente. Consiste en crear resúmenes, en la forma de texto nuevo, que capturen las ideas y los elementos clave del texto fuente. Los métodos abstractivos deben producir texto coherente similar a los resúmenes elaborados por seres humanos.

Aplicación del resumen de textos.

El resumen de textos puede ayudar a reducir la sobrecarga de información por medio de sintetizar y hacer aflorar la información importante de una manera que sea más fácil de digerir y procesar para los usuarios. Tiene un amplio rango de aplicaciones.

- **Resumen de artículos de noticias:**

  Genere resúmenes de artículos de noticias para ayudar a las personas a mantenerse informadas sobre los sucesos de actualidad.

- **Resumen de publicaciones académicas:**

  Genere resúmenes de publicaciones académicas para ayudar a los estudiantes e investigadores a entender con rapidez los principales resultados de un estudio.

- **Transcripciones de centros de llamadas:**

  Genere resúmenes de llamadas de los clientes para optimizar el trabajo posterior a la llamada y la supervisión de la calidad.

- **Resumen de documentos empresariales:**

  Genere resúmenes de documentos empresariales importantes, como informes, reuniones y transcripciones de llamadas.

- **Resumen de comunicaciones personales:**

  Genere resúmenes de publicaciones en redes sociales, correos electrónicos y otras comunicaciones para ayudar a las personas a mantenerse organizadas y tener el control de su carga de trabajo.

Patrones de arquitectura para aplicaciones de resumen de textos.

Cuando trabaje con resúmenes de documentos, la selección del patrón de arquitectura de la aplicación depende del tamaño del documento en relación con el tamaño del contexto del modelo. Otros desafíos que presentan los documentos grandes son los errores por falta de memoria y las alucinaciones. Puede usar los siguientes dos enfoques de arquitectura para crear una aplicación de resumen basada en el tamaño del documento.

- **Resumen de textos para documentos pequeños:**

  Puede aplicar este patrón de arquitectura cuando el texto del documento quepa en el tamaño del contexto del LLM. Un texto pequeño (o un archivo pequeño) se pasa directamente al LLM, que responde con un resumen del texto. El patrón de arquitectura de esta situación se ilustra en la figura siguiente.

  Nota: Este enfoque *no* funciona cuando el tamaño del documento o del texto es grande y no cabe en la ventana de contexto del LLM.



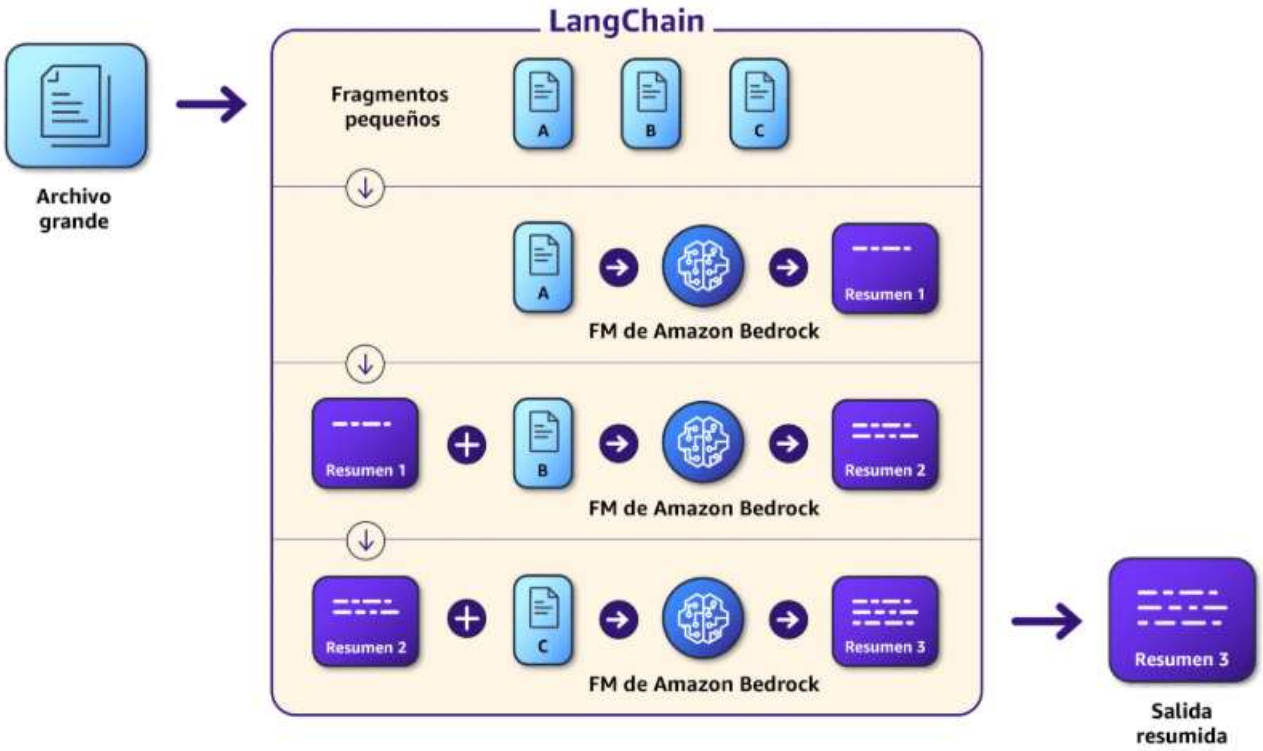
- **Resumen de textos para documentos grandes:**

  Para los documentos o textos grandes, no es posible pasar el texto completo al LLM debido al tamaño limitado del contexto. Para resolver este problema, puede usar una arquitectura map-reduce y aplicar los conceptos de fragmentación y encadenado de peticiones.

  La arquitectura de resumen de texto para documentos de gran tamaño incluye los siguientes pasos:

  1. Se divide un documento grande en varios fragmentos de números (n) pequeños con herramientas como LangChain.

2. Se envía cada fragmento al LLM para generar el resumen correspondiente.
3. Se agrega el siguiente fragmento al primer resumen generado y se vuelve a resumir.
4. Se itera sobre cada fragmento para crear una salida resumida final.



RESPUESTA A PREGUNTAS

Arquitectura de respuesta a preguntas.

La respuesta a preguntas es una tarea importante que implica la extracción de respuestas en relación con consultas objetivas planteadas en lenguaje natural. En general, un sistema de respuesta a preguntas procesa una consulta frente a una base de conocimientos que contiene datos estructurados o no estructurados y genera una respuesta con información precisa. Poder garantizar una alta exactitud es clave para desarrollar un sistema de respuesta a preguntas que sea útil, fiable y de confianza, en especial en lo que respecta a los casos de uso de la empresa. A continuación, revisará dos patrones de arquitectura para aplicaciones de respuesta a preguntas.

- **Caso de uso de base genérica:**

  Con el caso de uso de base genérica, el usuario puede enviar una petición al FM en Amazon Bedrock y obtener respuestas basadas en información genérica que el público entienda. El caso práctico de base genérica no tiene el elemento de personalización.

Hay un documento pequeño que un usuario puede usar para formular preguntas. Dicho documento se somete a un procesamiento más detallado en el FM en Amazon Bedrock para obtener una respuesta generada. Esta respuesta puede surgir de forma natural del modelo base sin ninguna recuperación específica. Es posible que le falte personalización para casos prácticos que requieran información específica y minuciosidad. Los ejemplos incluyen casos prácticos médicos, documentos legales y financieros, etc. Este patrón de arquitectura se ilustra en la imagen siguiente.



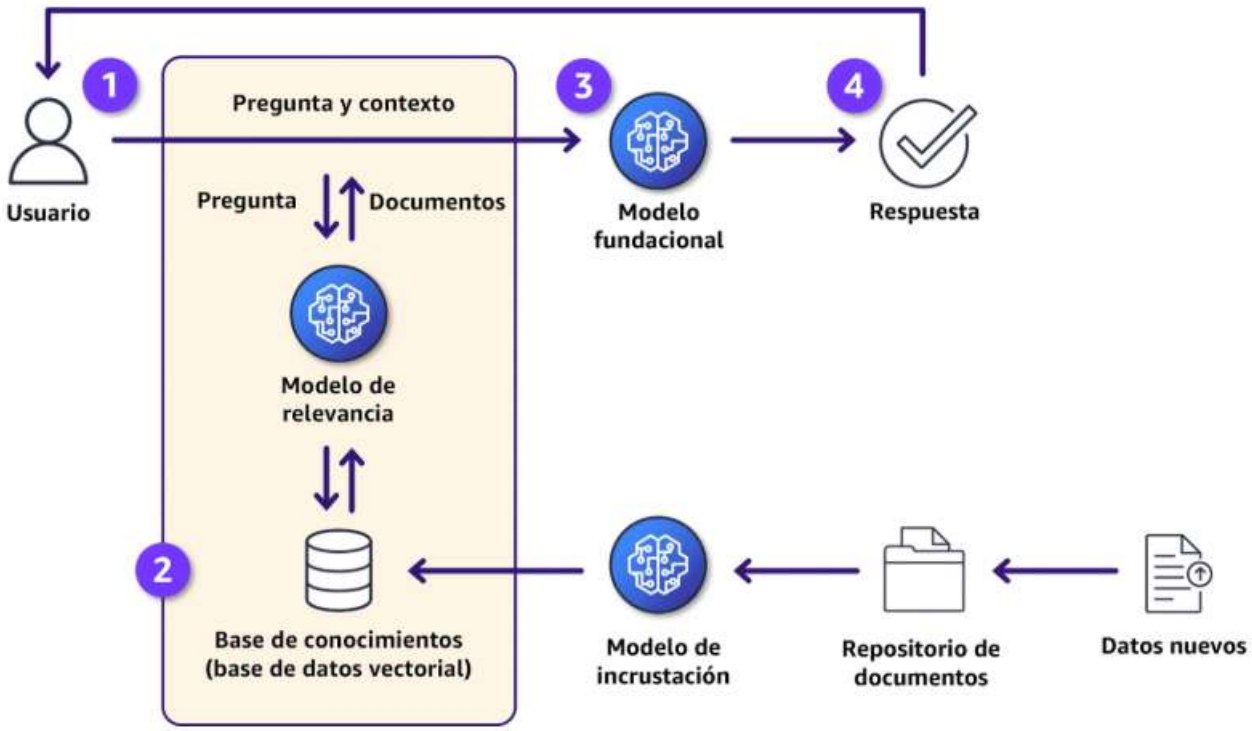
- **Casos de uso específicos y personalizados:**

  El siguiente patrón de arquitectura se usa específicamente para casos de uso personalizados. Estos casos de uso requieren respuestas específicas y minuciosidad en distintos dominios, por lo que no es suficiente utilizar un modelo base. Puede usar la técnica de generación aumentada por recuperación (RAG) con el fin de ubicar los fragmentos relevantes de texto que mejor se adapten para responder la pregunta del usuario. Luego, la RAG concatena el mejor conjunto de cadenas como respuesta al usuario a partir de los fragmentos pertinentes recuperados.

  La técnica de RAG implica los siguientes pasos:

  1. El usuario hace una pregunta.
  2. Los documentos específicos de un dominio se convierten en incrustaciones mediante el modelo Amazon Titan Embeddings. Las incrustaciones se almacenan en una base de conocimientos (base de datos vectorial) para su posterior recuperación.
  3. La pregunta del usuario se usa para recuperar los fragmentos de datos pertinentes (que actuarán como contexto) de la base de conocimientos. Luego, la pregunta del usuario y el contexto se pasan al FM a fin de obtener una respuesta precisa para el usuario.

4. Cuando el usuario formula una petición, el FM identifica el contexto y consulta la base de conocimientos para obtener los fragmentos de datos pertinentes. El FM luego interactúa con otro FM con el objetivo de obtener una respuesta precisa para el usuario.



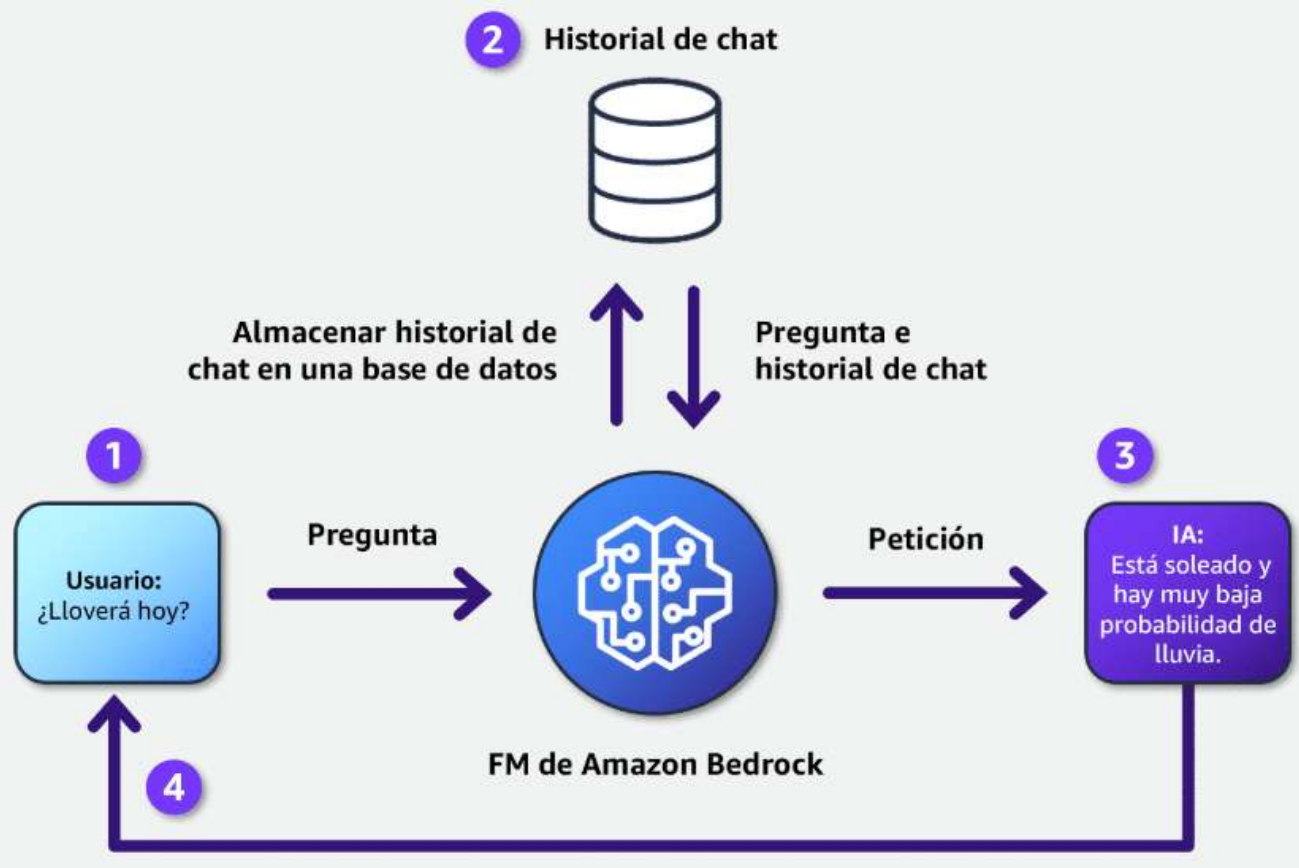
Uno de los desafíos de responder preguntas es la cantidad limitada de tokens en el contexto. Se soluciona usando RAG. El patrón de arquitectura para casos de uso específicos y personalizados da como resultado respuestas fiables y precisas.

CHATBOTS

Interfaces conversacionales.

Puede usar interfaces conversacionales, como chatbots y asistentes virtuales, a fin de mejorar la experiencia del usuario para los clientes. Los chatbots usan algoritmos de machine learning (ML) y NLP para comprender y responder las consultas de los usuarios. Puede usar chatbots en varias aplicaciones, como servicio al cliente, ventas y comercio electrónico, para ofrecer respuestas rápidas y eficientes a los usuarios. Puede acceder a los chatbots desde diferentes canales, como sitios web, plataformas de redes sociales y aplicaciones de mensajería.

En el diagrama siguiente, se ilustra un patrón de arquitectura básico de un caso de uso de chatbot con Amazon Bedrock.



Esta arquitectura incluye los pasos siguientes:

1. El usuario hace una consulta al chatbot.
2. El historial de chat (si lo hay) se pasa al modelo de Amazon Bedrock junto con la consulta actual del usuario.
3. Luego, el modelo genera una respuesta.
4. El modelo devuelve la respuesta al usuario.

En este caso de uso básico, el usuario puede introducir una petición específica, como una pregunta a Amazon Bedrock. El FM almacena el chat, o las preguntas y respuestas generadas, en un historial de chat. A partir del historial del chat y de la petición actual que realizó el usuario, Amazon Bedrock proporciona una respuesta precisa y útil.

Casos de uso de chatbot.

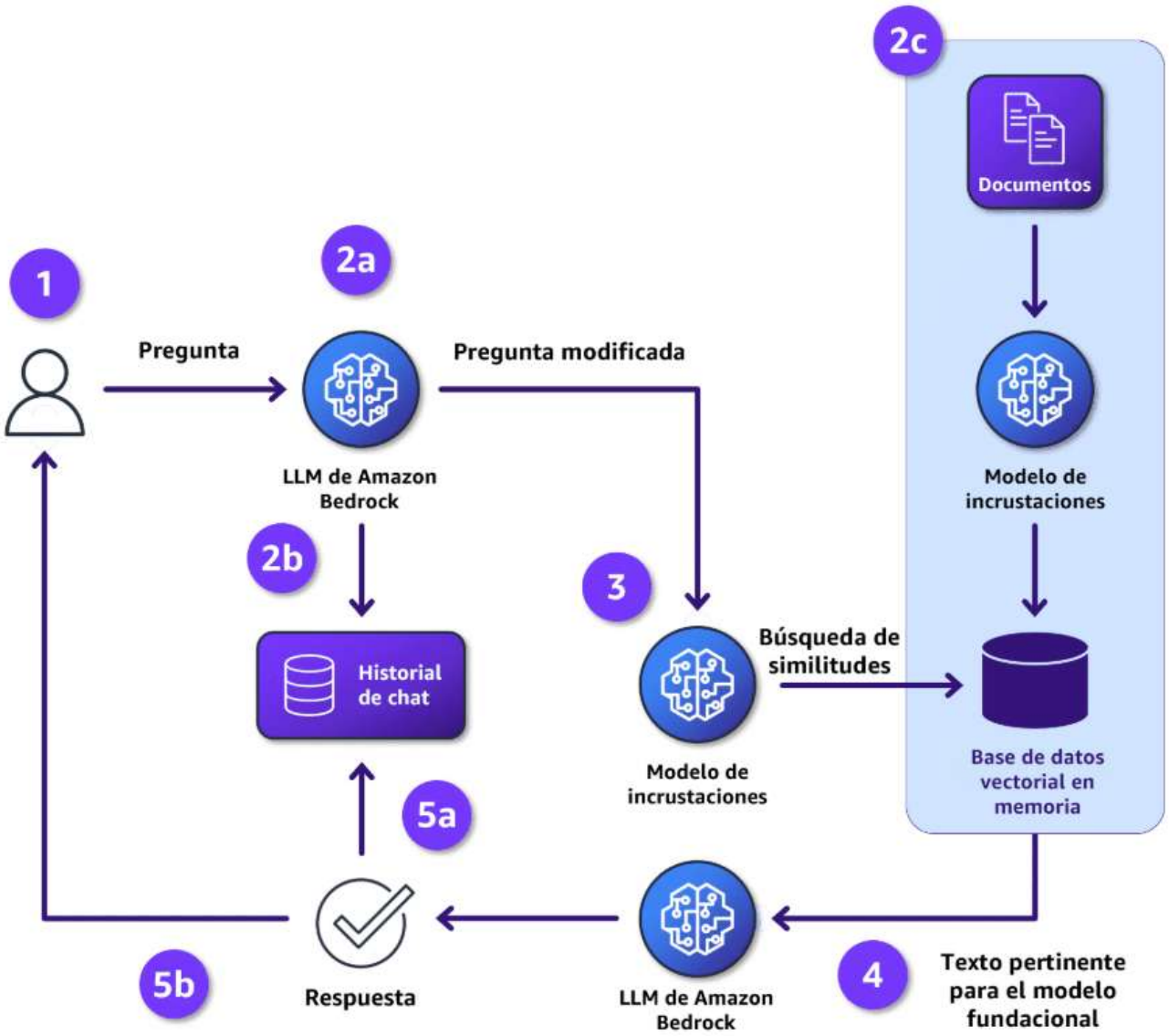
Se pueden clasificar los casos de uso de chatbot de la siguiente manera:

- **Chatbot (básico):** se trata de un chatbot sin entrenamiento previo con un modelo de FM.
- **Chatbot que usa una plantilla de peticiones (LangChain):** se trata de un chatbot con cierto contexto que se proporciona en la plantilla de peticiones.
- **Chatbot con una personalidad:** se trata de un chatbot con roles definidos, como un entrenador profesional con interacciones humanas.
- **Chatbot consciente del contexto:** se trata de un chatbot que pasa el contexto desde un archivo externo mediante la generación de incrustaciones.

Arquitectura para un chatbot consciente del contexto:

En el diagrama siguiente, se muestra una arquitectura simple para un chatbot consciente del contexto. Esta arquitectura incluye los pasos siguientes:

1. El usuario le hace una pregunta (consulta del usuario) al LLM en Amazon Bedrock.
2. Paso 2:
    a. El LLM envía la pregunta modificada al modelo de incrustaciones.
    b. Se actualiza el historial de conversaciones.
    c. La consulta del usuario se convierte en una incrustación vectorial con el modelo Amazon Titan Embeddings.
3. Se realiza una búsqueda de similitud. El resultado de la búsqueda es un conjunto de fragmentos de texto pertinentes.
4. Según la información almacenada, se genera una respuesta a la petición (consulta del usuario) a partir del FM final.
5. Paso 5:
    a. La consulta del usuario y la respuesta del FM se agregan al historial de chat.
    b. Se le da la respuesta al usuario al mismo tiempo.



Para simplificar, considere que los pasos 2a, 2b y 2c suceden al mismo tiempo. Del mismo modo, tenga en cuenta que los pasos 5a y 5b ocurren en forma simultánea.

En este patrón, el usuario formula una pregunta a Amazon Bedrock. La consulta enviada por el usuario se convierte a una incrustación vectorial con el modelo Amazon Titan Embeddings. Una búsqueda por similitud compara las incrustaciones de la consulta del usuario con los vectores de la base de datos vectorial. El resultado de la búsqueda es un conjunto de fragmentos de texto pertinentes. La consulta del usuario, el historial de chat y los fragmentos de texto relevante se envían al FM, y el FM devuelve una respuesta.

La consulta del usuario y la respuesta del FM se agregan al historial de chat para futuras conversaciones. La biblioteca de conocimientos se crea con antelación mediante la creación de incrustaciones de documentos específicos del dominio y su almacenamiento en la base de datos vectorial.

Según la información almacenada como incrustaciones en la base de datos vectorial y en la solicitud de usuario actual, se genera una respuesta relevante a la petición a partir del FM final. La respuesta se procesa a través del historial de chat para mayor eficiencia conversacional y para que se la recuerde. Dicha respuesta se da al usuario al mismo tiempo.

Esta es una descripción breve del patrón de arquitectura. Una descripción más básica es que la pregunta se realiza a un LLM, que está incrustado. Usando la incrustación y otro historial de chat de la base de datos vectorial, otro FM luego crea una búsqueda relevante de los datos. A continuación, da una respuesta precisa, que se almacena en el historial de chat y se envía al usuario a una latencia baja.

GENERACIÓN DE CÓDIGO

Tareas de codificación y programación.

También puede usar los modelos fundacionales en Amazon Bedrock para varias tareas relacionadas con la codificación y la programación. Los ejemplos incluyen generación de consultas SQL y código, explicación y traducción de código, solución de errores, optimización de código y otros. El uso de los modelos fundacionales para las tareas relacionadas con la codificación ayuda a los desarrolladores y científicos de datos a crear rápidamente prototipos de sus ideas y casos de uso.

El patrón de arquitectura siguiente ilustra el caso de uso de utilización de los FM en Amazon Bedrock para codificación y programación.



Los pasos son los siguientes:

1. El usuario introduce una petición de código.
2. Un modelo fundacional procesa los datos de entrada.
3. El modelo devuelve el código generado.

En este patrón, proporciona una petición en texto plano al modelo fundacional. La petición incluye una instrucción que indica al modelo qué generar y, en algunos casos, algunos ejemplos de código. Puede usar esta arquitectura para varios casos de uso, como generación de consultas SQL (Structured Query Language, lenguaje de consulta estructurado), completar ciertas tareas que requieren instrucciones en código específicas, etc.

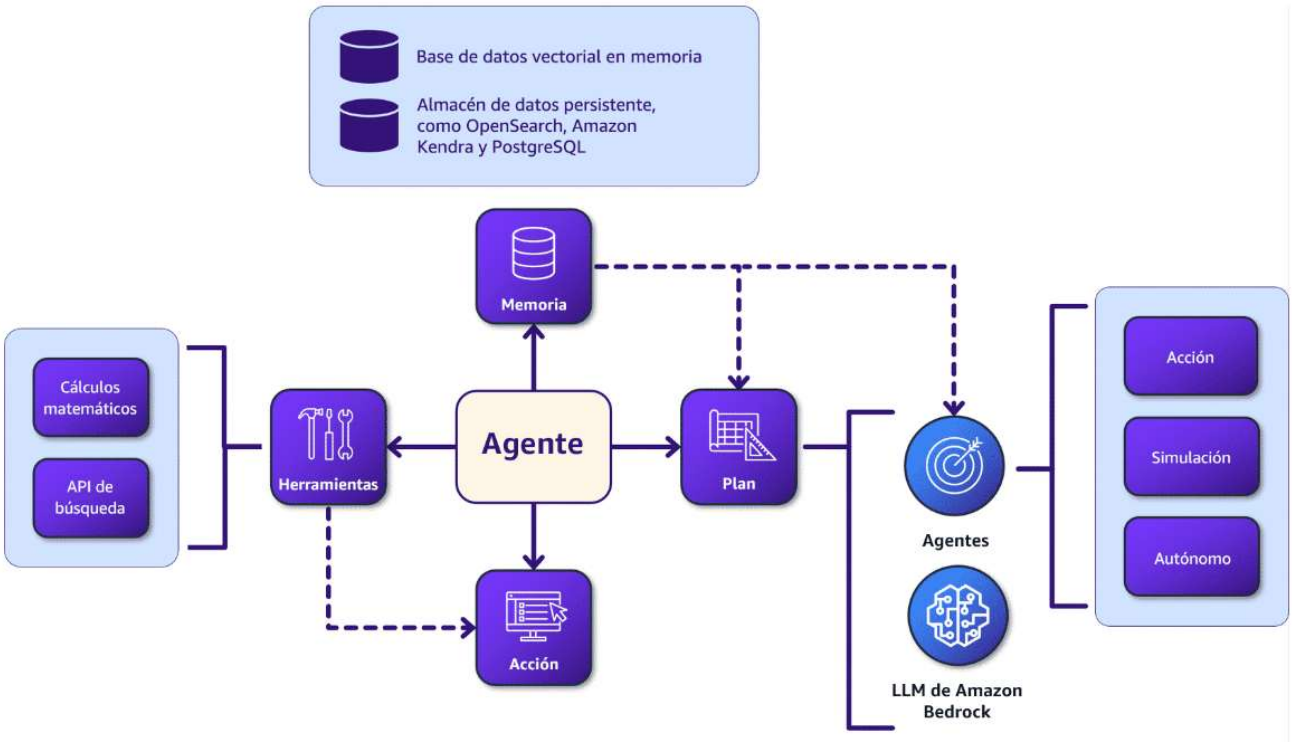
LANGCHAIN Y AGENTES PARA AMAZON BEDROCK

Agentes de LangChain.

Los modelos fundacionales se someten a un entrenamiento exhaustivo con grandes cantidades de datos. A pesar de sus capacidades sustanciales de comprensión del lenguaje natural, no pueden realizar de forma independiente tareas como el procesamiento de reclamaciones de seguro o la realización de reservas de vuelos. Esta limitación surge de la necesidad de acceso a los datos específicos más recientes de la industria o la empresa, que los modelos fundacionales no pueden obtener de fuentes de conocimientos actualizadas de manera predeterminada. Además, los FM no pueden realizar acciones específicas para completar solicitudes sin una gran cantidad de programación manual.

Algunas aplicaciones demandan una secuencia adaptable de llamadas a los modelos de lenguaje y a varias utilidades en función de la entrada del usuario. La interfaz del agente proporciona flexibilidad en estas aplicaciones. Un agente tiene disponibilidad para un rango de recursos y selecciona cuáles usar en función de la entrada del usuario. Los agentes pueden usar varias herramientas y pueden usar la salida de una herramienta como entrada de la siguiente.

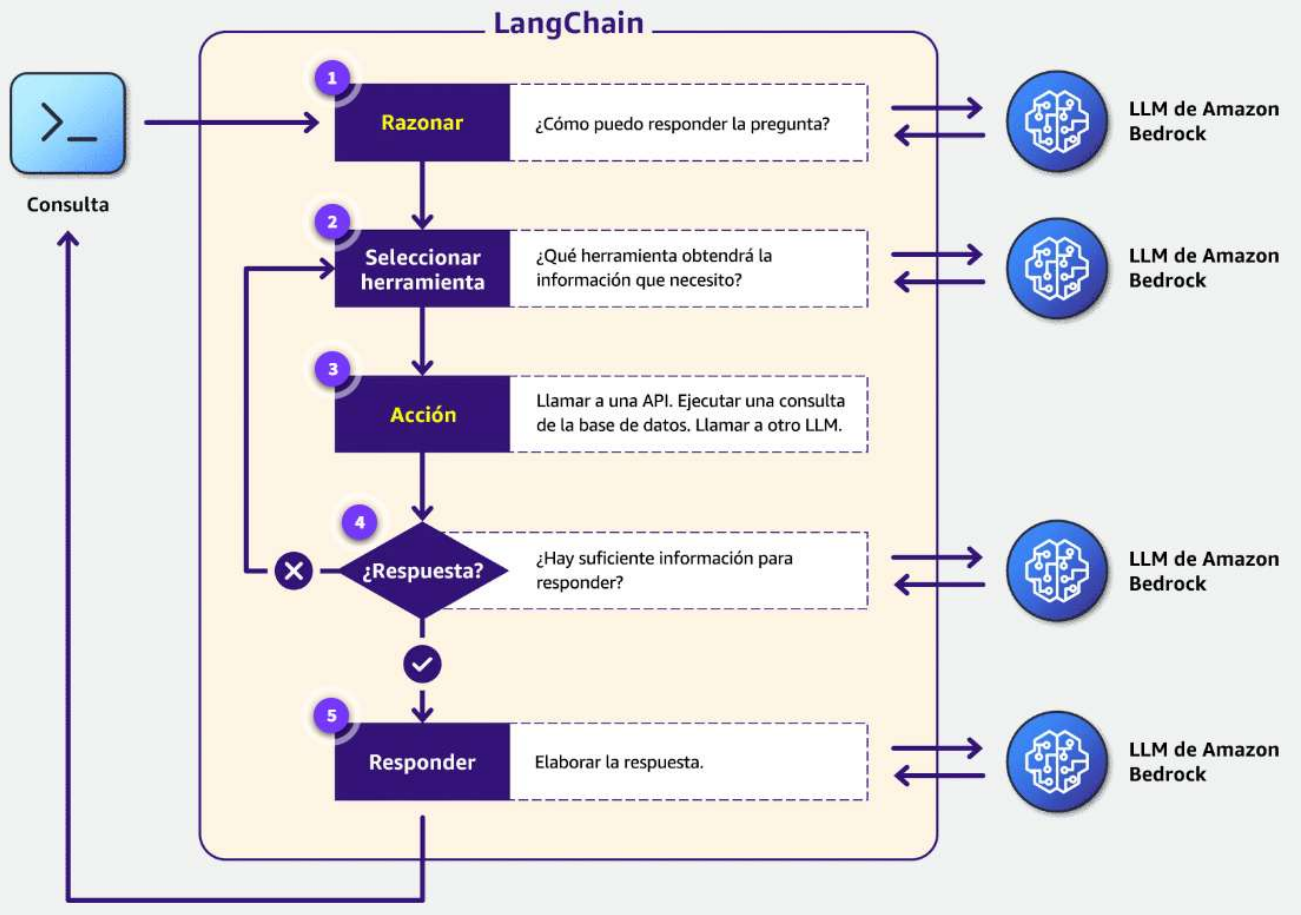
La arquitectura se ilustra en la imagen siguiente.



En este patrón, el agente tiene acceso a varias herramientas y selecciona la adecuada al momento del procesamiento según la descripción de la herramienta. Según los requisitos de su aplicación, las herramientas pueden ser sencillas, como realizar una búsqueda en la web, o pueden ser complejas, como realizar cálculos matemáticos.

Usar ReAct para poner en marcha agentes en LangChain.

Puede poner en marcha agentes en LangChain usando una de dos técnicas: planificar y poner en marcha, o ReAct, que significa **razonar (reasoning)** y **actuar (acting)**. La técnica ReAct evaluará la petición y determinará el paso siguiente para resolver el problema. Luego, ejecutará ese paso y repetirá el proceso hasta que el LLM pueda responder la pregunta. La técnica “planificar y ejecutar” funciona de una manera un poco distinta, ya que determina los pasos necesarios por anticipado y los realiza de manera secuencial.



1. **Razonar:**

   El LLM evaluará la pregunta para determinar cómo resolver la solicitud.

2. **Seleccionar herramienta:**

   Según la información necesaria para resolver la solicitud, se proporciona un listado de herramientas. El LLM determina cuál es la mejor herramienta para obtener los datos necesarios.

3. **Acción:**

   Llame a la herramienta para obtener los datos. La herramienta puede poner en marcha una búsqueda, consultar una base de datos, llamar a una API o llamar un LLM especializado para brindar una respuesta.

4. **¿Respuesta?:**

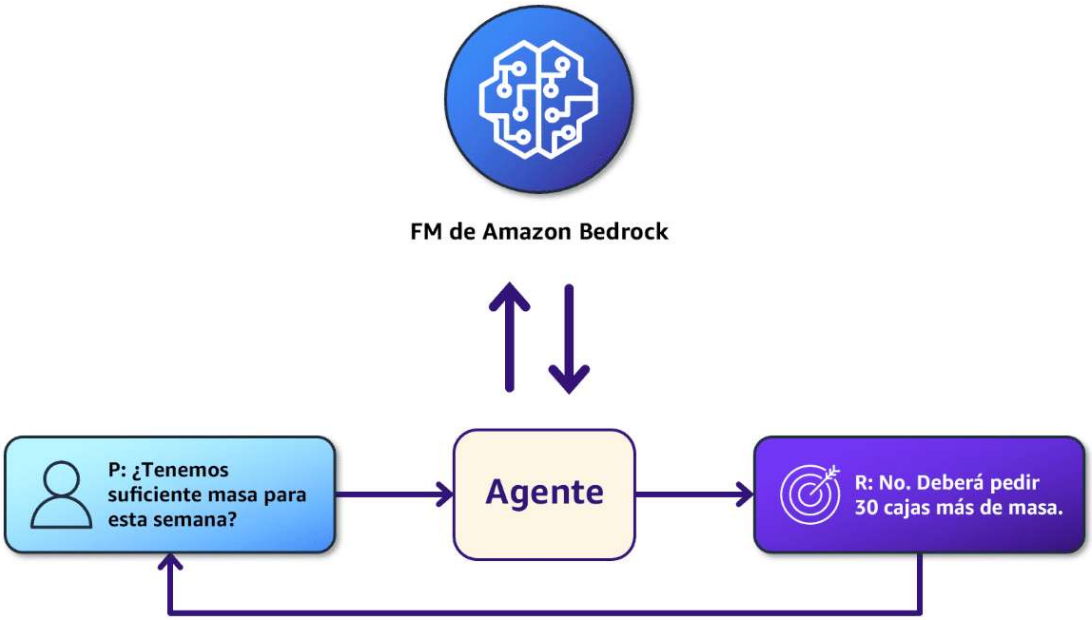
   Luego de incluir los datos de la herramienta en la petición, ¿hay información suficiente como para responder a la petición o se requieren datos adicionales? Si la hay, continúe y genere la respuesta. De lo contrario, regrese al paso de selección de herramienta para encontrar una que pueda proporcionar los datos.

5. **Responder:**

   Con todos los datos recopilados, el LLM crea una respuesta a la petición y devuelve el resultado.

Agentes para Amazon Bedrock.

Amazon Bedrock es una oferta completamente administrada que facilita a los desarrolladores la automatización de tareas. Con agentes para Amazon Bedrock, los FM pueden comprender las solicitudes de los usuarios, dividir las tareas complejas en varios pasos y realizar las acciones necesarias para responder a las solicitudes. Los desarrolladores pueden usar agentes para Amazon Bedrock a fin de crear un plan de orquestación sin ninguna codificación manual. Por ejemplo, una aplicación de IA generativa para restaurantes impulsada por agentes puede proporcionar una respuesta básica a la pregunta "¿Tenemos suficiente cantidad de masa para esta semana?". Sin embargo, también puede ayudar con la tarea de actualizar su inventario.



Los agentes necesitan acceso a un origen de datos externo y necesitan conectarse a sus API existentes. Los desarrolladores pueden usar la consola de Amazon Bedrock o AWS SDK para cargar el esquema de API. Los agentes orquestan las tareas con la ayuda de FM y realizan llamadas a la API con funciones de AWS Lambda. Por lo tanto, los agentes eliminan la necesidad de administrar la integración del sistema y el aprovisionamiento de la infraestructura.

Ejemplo: Conectar modelos fundacionales a sus orígenes de datos empresariales con agentes para Amazon Bedrock.

Ahora, verá más de cerca cómo usar agentes para Amazon Bedrock con un ejemplo. A veces, quiere que los modelos fundacionales tengan acceso a más datos para ayudar al modelo a generar respuestas más relevantes y específicas del contexto sin volver a entrenar con regularidad el modelo fundacional.

Los siguientes pasos proporcionan a los agentes acceso a una base de conocimientos en Amazon Bedrock.

- **Paso 1: usar RAG.**

  Los agentes para Amazon Bedrock usan RAG para proporcionar a los agentes acceso a una base de conocimientos en Amazon Bedrock. La base de conocimientos se crea en Amazon Bedrock y apunta a un origen de datos en Amazon Simple Storage Service (Amazon S3) que contiene los datos. Tiene la opción de sincronizar la base de conocimientos en Amazon Bedrock con sus datos en Amazon S3.

- **Paso 2: seleccionar un modelo de incrustaciones y una base de datos vectorial.**

  A continuación, seleccionará un modelo de incrustaciones y proporcionará detalles para una base de datos vectorial. Para la base de datos vectorial, puede elegir entre Amazon OpenSearch sin servidor, Pinecone y Redis Enterprise Cloud.

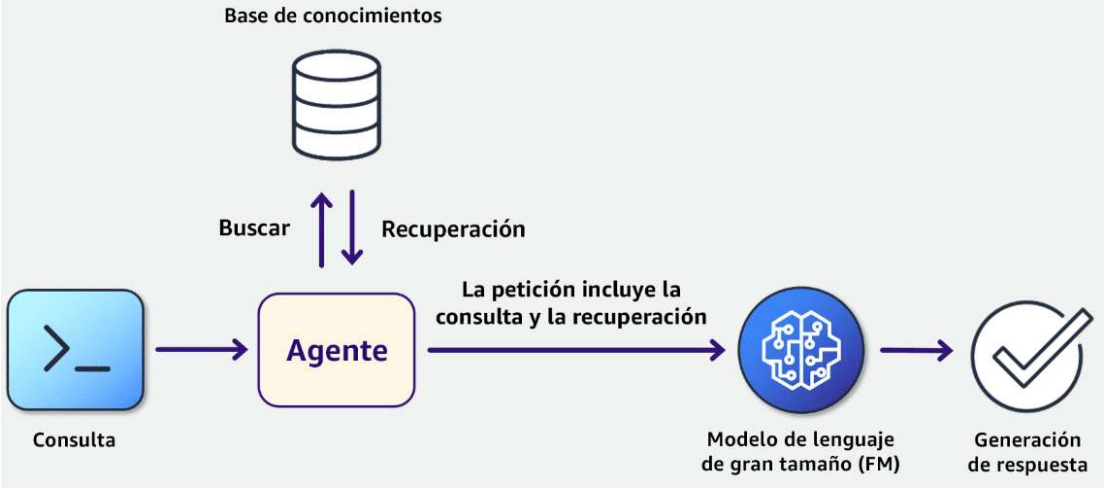
- **Paso 3: agregar la base de conocimientos.**

  Puede agregar la base de conocimientos al crear o actualizar agentes para Amazon Bedrock.

- **Paso 4: Crear y agregar grupos de acción.**

  Al agregar la base de conocimientos al agente, también puede crear *grupos de acciones* y agregarlos. Los grupos de acciones son tareas que un agente puede realizar de forma autónoma. Puede proporcionar funciones de Lambda que representen su lógica empresarial y el esquema de API relacionado para ejecutar esas funciones. Si bien los grupos de acciones no son necesarios para crear un agente, pueden aumentar el rendimiento del modelo para proporcionar mejores salidas.

En el siguiente diagrama, se muestra el flujo de la consulta de usuario.



Cuando una consulta de usuario llega al agente, este identificará la base de conocimientos adecuada a partir de la entrada del usuario y recuperará la información relevante. El agente agrega la información recuperada a la petición de entrada. Esto ayuda al FM a obtener acceso a más información contextual, de modo que pueda generar una respuesta más precisa. Siguiendo este patrón, puede usar agentes para Amazon Bedrock a fin de realizar tareas empresariales complejas invocando las API de forma dinámica sin tener que preocuparse por aprovisionar y mantener ninguna infraestructura.

Esta arquitectura se implementa con los siguientes componentes:

- La base de conocimientos tiene Amazon S3 como origen de datos.
- Hay un modelo fundacional, como Amazon Titan Embeddings, para convertir los datos en incrustaciones vectoriales.
- Hay una base de datos vectorial para almacenar los datos de vectores del paso anterior. Amazon Bedrock se encargará de crear, almacenar, administrar y actualizar sus incrustaciones en la base de datos vectorial.

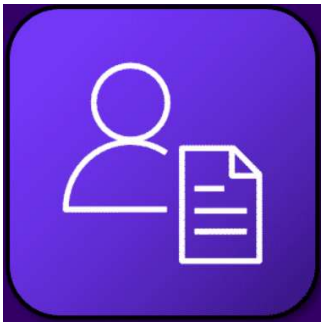
MÓDULO 6: USAR AGENTES

INTRODUCCIÓN

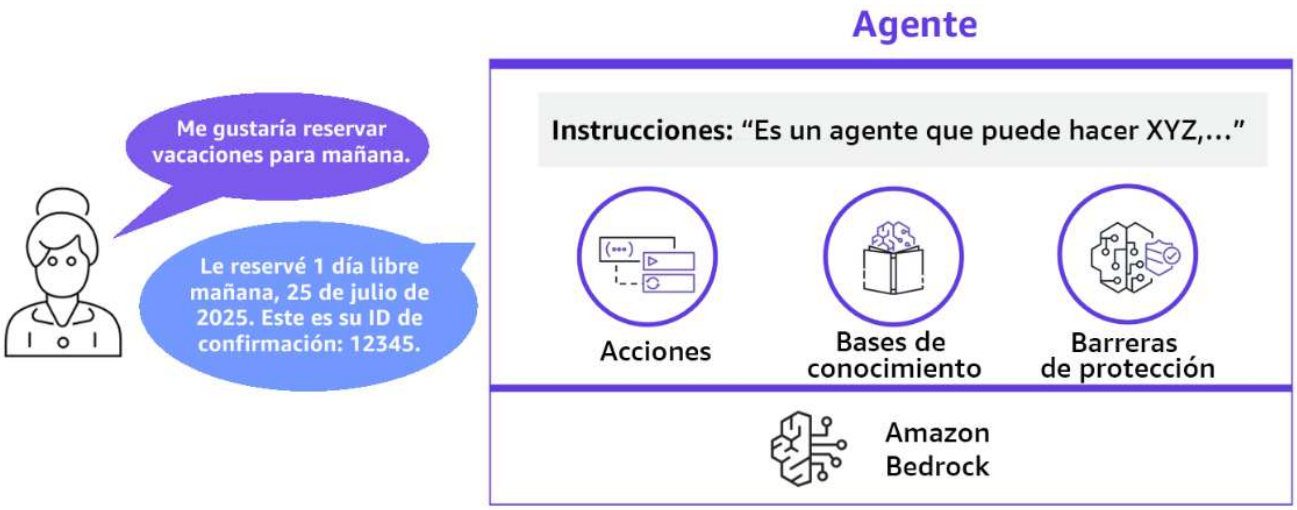
INTRODUCCIÓN A LOS AGENTES

Utilice agentes para Amazon Bedrock en la creación de aplicaciones de IA generativa de extremo a extremo que puedan ejecutar tareas de varios pasos para una variedad de casos de uso.

Los agentes son aplicaciones que utilizan los modelos de lenguaje de gran tamaño (LLM) para completar tareas a través del desglose del problema en cuestión mediante el uso de peticiones con cadenas de pensamiento (chain-of-thoughts, CoT). También pueden usar un marco de trabajo de razonamiento como el de razonar y actuar (reasoning and acting, ReAct) para mejorar la capacidad de razonamiento de los modelos fundacionales (FM). Los agentes organizan la ejecución de los pasos para cumplir con su solicitud y ofrecer respuestas actualizadas. Estas respuestas se basan en llamadas a la API, integraciones con las bases de conocimientos de Amazon Bedrock o ejecuciones del código con las capacidades de interpretación del código del agente. Los agentes también pueden aplicar las **barreras de protección de Amazon Bedrock** para proteger su aplicación y la experiencia del cliente. Las barreras de protección pueden ser específicas de su caso de uso y reflejar las políticas de IA responsable de su empresa.



En resumen, un agente sigue una instrucción que define lo que debe y no debe hacer, y las acciones que puede ejecutar. También puede acceder a las bases de conocimientos internas y aplicar las barreras de protección que codifican las políticas de IA responsable de la empresa. Un FM subyacente proporciona las capacidades de razonamiento, mientras que el agente organiza los pasos necesarios para completar una tarea solicitada por el usuario. En la siguiente imagen, se destacan los componentes principales de un agente.



Ejemplo de una persona que trabaja con un agente de Recursos Humanos (RR. HH.) para reservar un día libre.

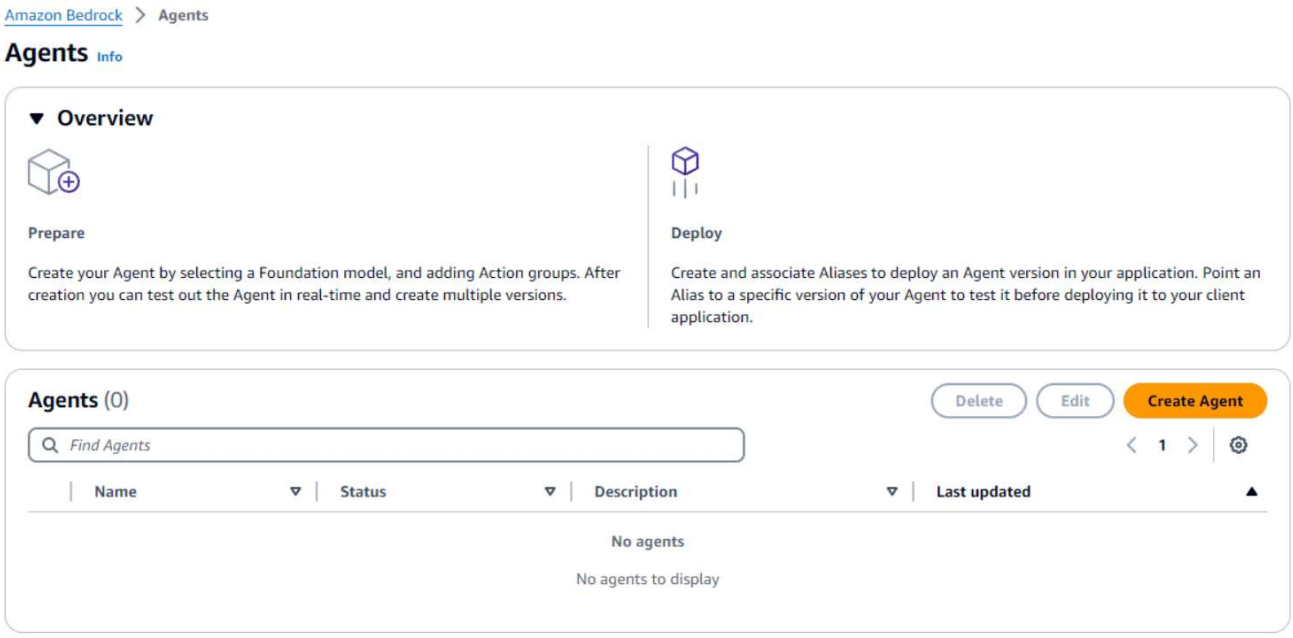
Modelos de lenguaje de gran tamaño y agentes.

Los modelos de lenguaje de gran tamaño (LLM) representan un avance drástico en la IA generativa, ya que pueden aprender una representación integral y completa del lenguaje natural. Los LLM pueden comprender y generar textos similares a los humanos en varios idiomas y ya se utilizan para una amplia gama de aplicaciones, como el resumen de textos, la respuesta a preguntas, la traducción, el análisis de sentimientos y la generación de textos. Si bien los LLM son excelentes para razonar a partir de datos y aprender de ejemplos sencillos, no pueden ejecutar tareas, conectarse con sistemas externos ni rastrear el flujo conversacional de las interacciones de los usuarios. Para crear aplicaciones más complejas, los desarrolladores pueden usar una lógica codificada para casos de uso bien definidos que implican combinar varias peticiones en un orden predefinido. Aunque esto amplía aún más las capacidades de un LLM, no proporciona la adaptabilidad necesaria a los resultados de cada paso requerido para resolver casos de uso complejos.

Sin embargo, con los agentes de LLM, usted puede crear planes dinámicos para resolver tareas complejas que requieren la ejecución de código o la integración con sistemas activos o bases de conocimientos. También puede reaccionar a las salidas intermedias de cada interacción. Mediante la proporción de los detalles sobre las funciones disponibles y los resultados deseados a un LLM, el agente puede coordinar las acciones y las llamadas a funciones para realizar una variedad de tareas. Este enfoque habilita una automatización más flexible y eficiente, lo que disminuye la necesidad de la codificación manual exhaustiva adaptada a cada situación. Los agentes actúan como interfaces intuitivas con el objetivo de que los usuarios puedan expresar sus intenciones en lenguaje natural y, al mismo tiempo, utilizar las capacidades de los sistemas de IA para ejecutar flujos de trabajo complejos de varios pasos.

Cómo trabajan los agentes:

Los agentes procesan las entradas en lenguaje natural, interpretan la intención de las consultas, formulan las peticiones adecuadas y ejecutan las tareas en nombre del usuario. Esto incluye el manejo de matices, como la ambigüedad, el contexto y el lenguaje figurativo, a fin de garantizar una comprensión precisa y la generación de respuestas. En última instancia, los agentes mejoran la usabilidad y la eficacia de las aplicaciones de LLM, lo que permite mantener conversaciones coherentes y naturales mientras se realizan acciones en nombre del usuario para realizar las tareas.

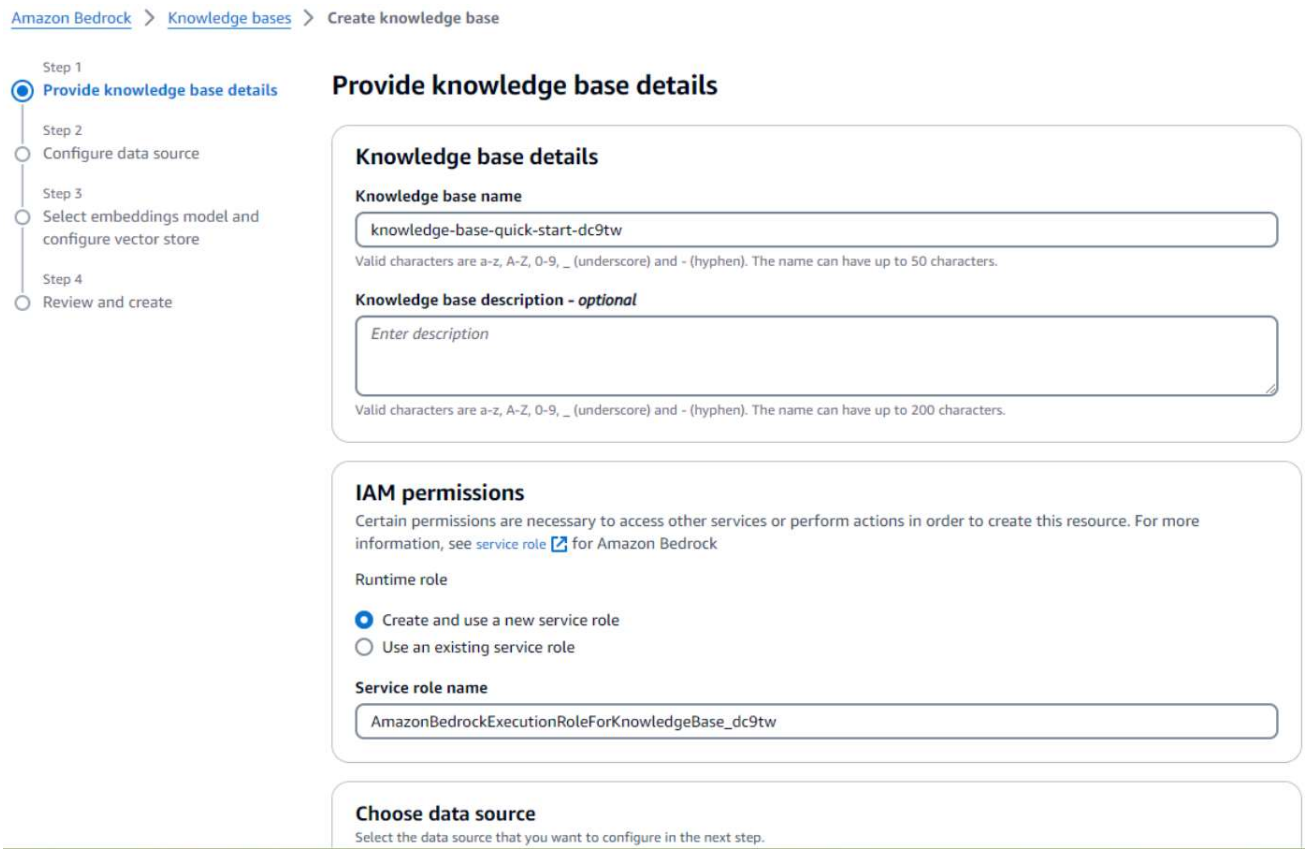


Sección de configuración de agentes en Amazon Bedrock.

Los agentes pueden gestionar solicitudes complejas mediante el desglose y la delegación de tareas. Estos tipos de solicitudes suelen consistir en varios pasos o muchas subtareas, que el agente puede desglosar y enrutar hacia la función adecuada con el objetivo de ejecutar esa subtarea. En este enfoque modular, se utilizan los puntos fuertes de los diferentes modelos, lo que permite obtener resultados precisos y completos. En particular, los agentes pueden tomar medidas para interactuar con sistemas externos en función de las salidas del LLM. Esta capacidad es particularmente útil en situaciones en las que el agente necesita realizar acciones que consistan en mucho más que generar texto, como controlar dispositivos, actualizar bases de datos o iniciar procesos.

Los agentes pueden integrar fuentes de información propias o externas mediante bases de conocimientos específicas del dominio. Con esta integración de recuperación de conocimientos, los agentes pueden incorporar información fidedigna y actualizada. Esto aumenta la exactitud y la relevancia del contexto que utiliza el LLM en

la generación de una respuesta para el usuario. Los agentes aprovechan la capacidad de razonamiento de los LLM para comprobar si hay errores en las respuestas y autocorregirse cuando es necesario. Esto proporciona a los usuarios información más confiable.



Configuración de la base de conocimientos en Amazon Bedrock.

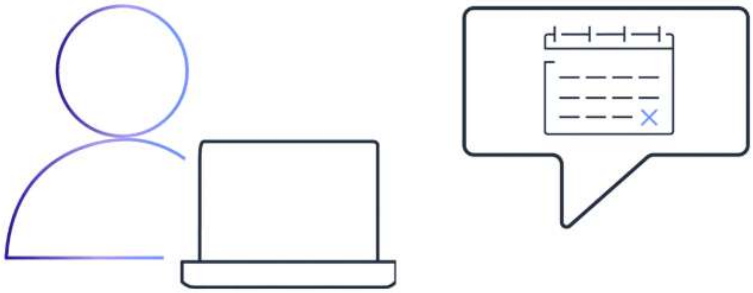
Los agentes están diseñados para trabajar directamente con los LLM. Estos utilizan las capacidades de procesamiento del lenguaje natural de los LLM para comprender y responder a las consultas de los usuarios, y contribuir a la facilidad de interpretación y la transparencia de las salidas. Mediante el análisis de las respuestas y el postulado de explicaciones, ayudan a los usuarios a entender el razonamiento detrás de las medidas que se tomaron. Esta capacidad es valiosa en los ámbitos que requieren facilidad de interpretación y responsabilidad, como las aplicaciones sanitarias, financieras y legales.

Los agentes para Amazon Bedrock implementan técnicas como la CoT con el fin de mejorar el razonamiento y el proceso de toma de decisiones del LLM. La CoT es una técnica que se utiliza para indicarle al LLM que genere un proceso de razonamiento paso por paso mediante el desglose de un problema complejo en una serie de pasos intermedios. Este enfoque puede mejorar la transparencia y la facilidad de interpretación del proceso de toma de decisiones del LLM.

Marco de trabajo de ReAct.

Un marco de trabajo de razonamiento, como ReAct, es fundamental para la capacidad de un agente a la hora de tomar decisiones y e implementar medidas. El ReAct permite a los agentes para Amazon Bedrock interactuar con los LLM con el fin de generar el trazado del razonamiento y de las acciones específicas de las tareas de forma intercalada. También permite a los agentes proporcionar contexto, comentarios o aclaraciones adicionales al LLM. Luego, los agentes formulan respuestas actualizadas en función de la nueva información o de las salidas de las medidas que se tomaron en los pasos anteriores. Este enfoque iterativo puede lograr salidas más precisas y relevantes, especialmente en situaciones complejas o ambiguas. En el siguiente ejemplo, se muestra el comportamiento de ReAct de un agente que, durante la planificación, se da cuenta de que el empleado solicitó más días libres de los que tiene disponibles. Luego, el agente usa el marco de trabajo ReAct para actualizar el plan y preguntar si el empleado quiere tomarse los días de vacaciones disponibles.

Usar ReAct para actualizar y completar una solicitud de vacaciones:



A un agente se le presenta una tarea: reservar un mes de días de vacaciones desde mañana, 25 de julio de 2024, hasta el 25 de agosto de 2024.

- **Paso 1: El proceso de pensamiento del agente.**



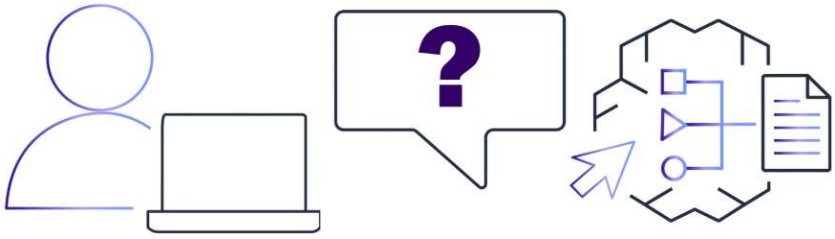
El proceso de pensamiento del agente es el siguiente: para responder, primero tendré que comprobar la cantidad de días de vacaciones disponibles que tiene el empleado. Si el empleado tiene suficientes días de vacaciones disponibles, reservaré 22 días libres.

- **Paso 2: El agente comprueba la cantidad de días de vacaciones disponibles.**



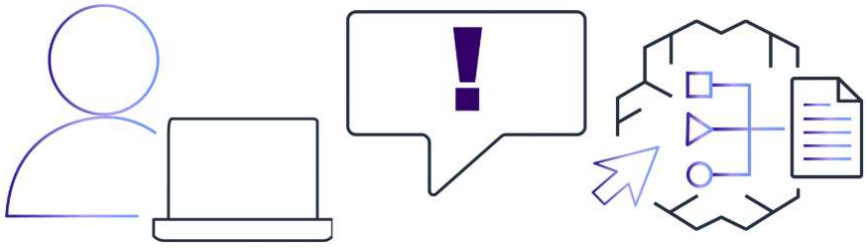
En primer lugar, la acción del agente consiste en llamar a la función GetAvailableVacationDays (employee_ID) para comprobar la cantidad de días de vacaciones disponibles. El agente observa entonces que el empleado tiene veinte días disponibles. Con esta información, determina que no tiene suficientes días de vacaciones disponibles. A continuación, utiliza el marco de trabajo ReAct para actualizar el plan.

- **Paso 3: El agente usa ReAct y solicita información adicional.**



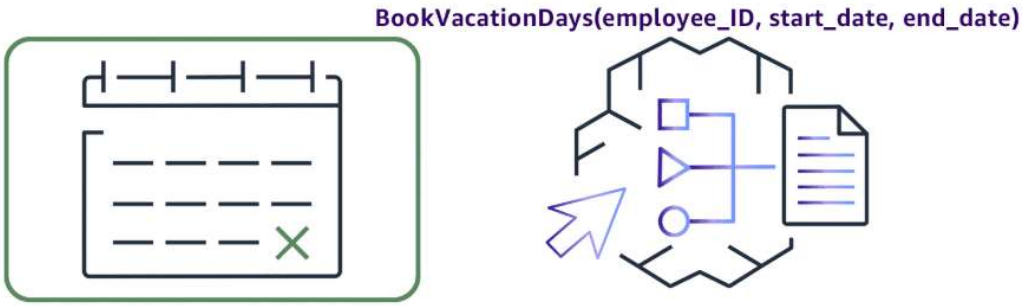
El agente necesitará información adicional, por lo que crea la siguiente respuesta: “Solo tiene veinte días de vacaciones disponibles”. Para tomarse todo el mes libre, necesitaría 22 días. ¿Quiere actualizar su solicitud para terminar sus vacaciones el 21 de agosto?

- **Paso 4: El agente pasa a la segunda tarea.**



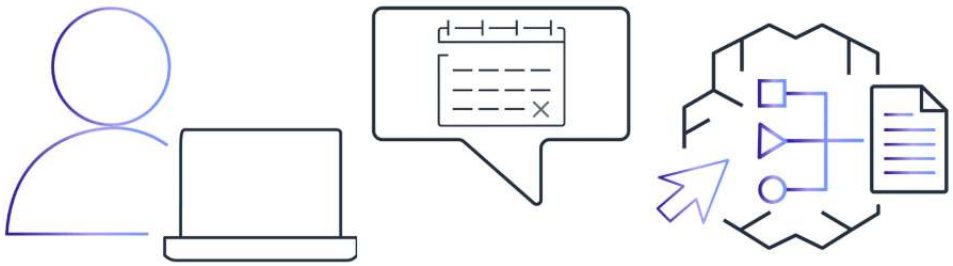
El usuario responde con un sí. Con esta nueva información, el agente puede pasar a la segunda tarea: reservar los días de vacaciones reales.

- **Paso 5: El agente reserva las vacaciones.**



El agente llama la función BookVacationDays (employee_ID, start_date, end_date) para reservar las vacaciones. Observa que la tarea se realizó correctamente. El agente determina que completó la tarea.

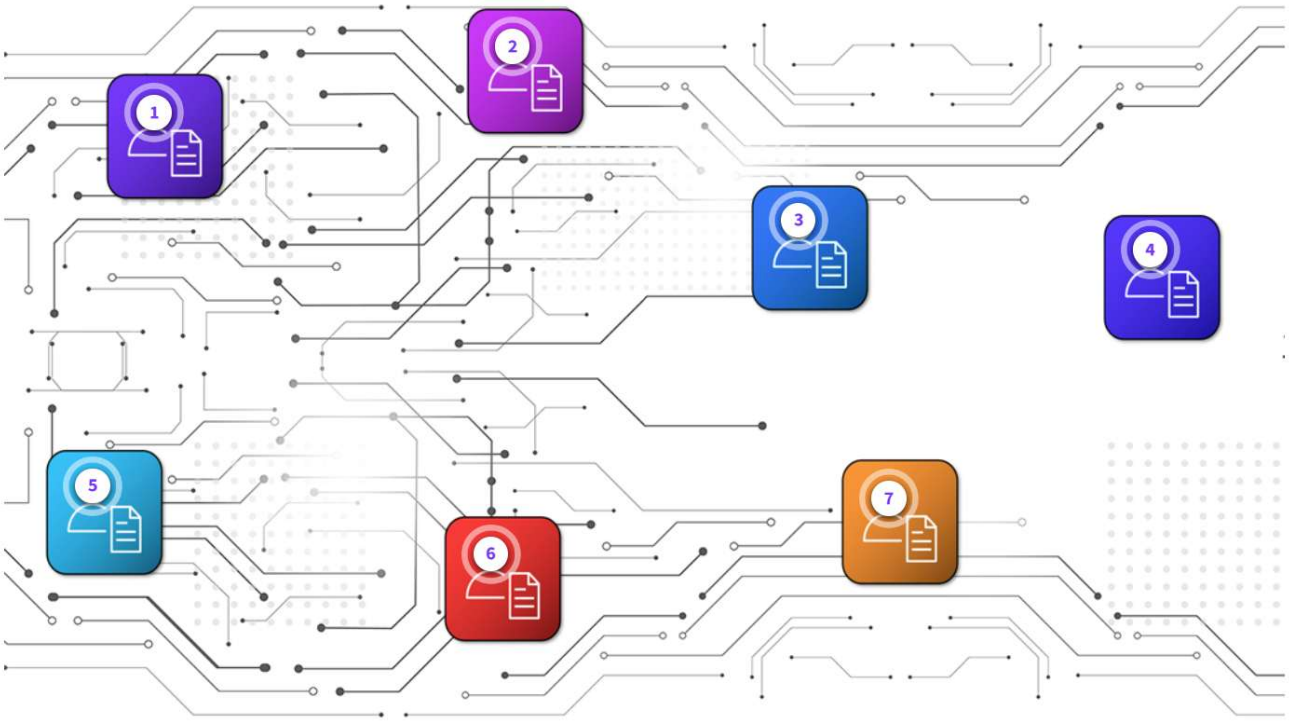
- **Final: El agente responde al solicitante.**



Por último, el agente responde lo siguiente al solicitante: “Reservé correctamente 20 días de vacaciones desde mañana, 25 de julio de 2024, hasta el 21 de agosto de 2024”.

CASOS DE USO PARA AGENTES

Los agentes de LLM desempeñan un rol fundamental en la maximización de las capacidades de los LLM. Sirven como el puente que permite a los LLM ejecutar tareas de manera efectiva, interactuar con los humanos y proporcionar información precisa, relevante y reciente. Los agentes se pueden aplicar en todos los sectores para resolver un conjunto diverso de desafíos.



1. **Integración de conocimientos.**

   Los agentes pueden integrar fuentes de conocimientos externas. Esto garantiza que las soluciones proporcionadas por los LLM se basen en información fidedigna y actualizada proveniente de diversos dominios.

2. **Respuestas conscientes del contexto.**

   Cuando están disponibles, los agentes mantienen un contexto conversacional a corto plazo, lo que permite a los LLM generar respuestas que sean relevantes y apropiadas para la sesión.

3. **Ejecución de tareas complejas.**

   Los agentes pueden dividir las tareas complejas en subtareas más pequeñas y distribuirlas en los modelos apropiados. Esto permite optimizar el proceso de ejecución para que los LLM puedan abordar problemas complejos de manera más eficaz.

4. **Transparencia.**

   Los agentes pueden compartir el razonamiento de las decisiones y las salidas generadas por los LLM. Esta información aumenta la transparencia y ayuda a los desarrolladores a solucionar problemas y a perfeccionar el comportamiento de los agentes.

5. **Fiabilidad y credibilidad.**

   Los agentes pueden actuar como otra capa destinada a interpretar las salidas de LLM mediante la ejecución de comandos en las API existentes. Esto permite obtener información más fiable y actualizada para procesar sus tareas.

6. **Personalización y adaptación.**

   Los agentes pueden adaptarse a las preferencias, los patrones de aprendizaje y los requisitos individuales de los usuarios. Esto permite a los LLM ofrecer soluciones personalizadas y a medida.

7. **Automatización de tareas y optimización del flujo de trabajo.**

   Los agentes pueden automatizar las tareas repetitivas y optimizar los flujos de trabajo con las capacidades de los LLM. Esto aumenta la eficiencia y la productividad en diversas industrias y aplicaciones.

Agentes en el mundo real:



Los agentes son fundamentales en la creación de aplicaciones complejas mediante LLM. Estos utilizan acciones autónomas para integrar sin problemas el conocimiento del dominio, la comprensión contextual y la implementación de tareas. Estos avances crean casos de uso diversos e interesantes en casi todas las industrias y funciones.

- **Ejemplo 1: Administración de relaciones con clientes.**



Un cliente tiene múltiples puntos de contacto con una empresa, lo que incluye ventas y asistencia al cliente. Los profesionales de ventas estarían mejor preparados para las reuniones con los clientes si tuvieran acceso a las interacciones anteriores con él. Piense en un agente de administración de relaciones con clientes (customer relations management, CRM) con acceso a las notas de las interacciones anteriores de ventas y asistencia al cliente, además del historial de pedidos. Para preparar la reunión, el profesional de ventas puede pedirle al agente que resuma las principales preocupaciones del cliente en función de las interacciones pasadas y un resumen de su historial de pedidos.

- **Ejemplo 2: Recursos humanos.**



Imagine a un agente de recursos humanos (RR. HH.) que pueda contratar empleados, ayudar en la incorporación, validar los acuerdos laborales y brindar apoyo continuo a los empleados. Además, los agentes de RR. HH. podían interactuar con los empleados, responder preguntas sobre las políticas de la empresa o el tiempo libre acumulado e, incluso, reservar tiempo libre a petición del empleado.

- **Ejemplo 3: Servicios financieros.**



En el sector de servicios financieros, un agente virtual que tenga acceso a su cuenta bancaria puede proporcionarle la información de la cuenta. Si el agente también tiene acceso a las preguntas más frecuentes, puede responder a preguntas sobre la ejecución de ciertos pasos, como la forma de congelar las transacciones de una tarjeta de crédito. Mediante la integración del agente con la funcionalidad de la barrera de protección, la institución financiera puede impedir que el agente brinde asesoramiento financiero.

- **Ejemplo 4: Casos de uso complejos.**



Para casos de uso más complejos, un conjunto de agentes que colaboran entre sí es un patrón común. Esto incluye casos de uso como un sistema multiagente que organiza los flujos de trabajo para escribir y entregar contenido de marketing personalizado y dirigido a las personas. El sistema de agentes tiene en cuenta las preferencias de las personas, a la vez que analiza las tendencias del mercado a través de las redes sociales y las reseñas. De este modo, los mensajes evolucionan continuamente y se adaptan a las condiciones del mercado y a las personas en tiempo real.

- **Resumen.**



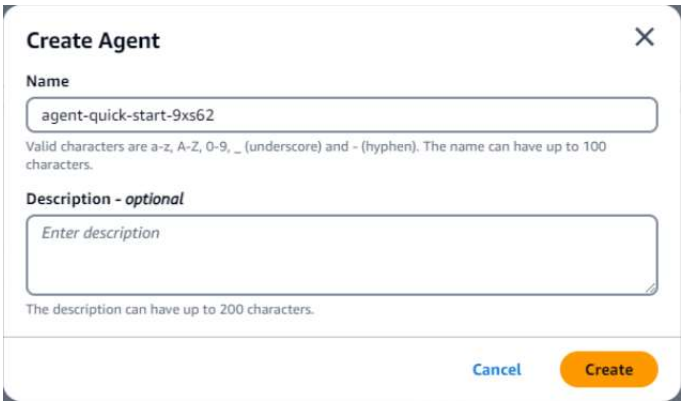
Los agentes pueden resolver casos de uso más complejos, ya que pertenecen a un área de la IA generativa en rápida evolución con capacidades de investigación activa y nuevas características.

En la actualidad, esta evolución es posible gracias a la accesibilidad a marcos de trabajo como LangChain y LlamaIndex, que permiten la creación programática de agentes que se pueden integrar en las aplicaciones. Los marcos de trabajo como Crew.ai permiten acelerar la creación de agentes colaborativos que trabajan en conjunto para completar las tareas. Crew.ai divide los esfuerzos y los dirige a los modelos más adecuados para tareas específicas.

INFORMACIÓN GENERAL SOBRE LOS AGENTES PARA AMAZON BEDROCK

Información general.

Los **agentes para Amazon Bedrock** permiten una implementación escalable, segura y completamente administrada de los agentes de LLM. Las tareas pueden incluir llamar funciones o API, recuperar información de texto de bases de conocimientos o generar código para completar tareas que no se pueden resolver con las funciones disponibles.



Ejemplo del asistente de creación de agentes en Amazon Bedrock.

En esencia, un agente tiene instrucciones y un modelo fundacional adjunto. En las instrucciones, se define para qué está diseñado un agente y qué no debe hacer. El modelo fundacional subyacente interpreta las entradas del usuario y las peticiones posteriores en el proceso de orquestación del agente y genera respuestas. Las interacciones de un agente persisten mientras dura la sesión de dicho agente. Los agentes utilizan las nuevas funciones de retención de memoria para recordar los resúmenes de las conversaciones de interacciones anteriores y utilizarlas para responder a las preguntas posteriores de los usuarios.

Para ejecutar acciones, un agente debe adjuntarse a un **grupo de acciones**. En él, usted define las acciones que puede realizar el agente. Un grupo de acciones se compone de definiciones de acción y un iniciador de acción. Usted puede definir acciones pasando los detalles de la función con un objeto JSON o utilizando un archivo de esquema de API en el formato de especificación de OpenAPI. Para cumplir con las acciones, un agente puede ejecutarlas mediante una función de AWS Lambda. Como alternativa, puede devolver el control a la aplicación y gestionar la lógica de funciones fuera de los agentes para Amazon Bedrock. Gracias al soporte de los agentes para Amazon Bedrock en la interpretación de código (versión preliminar), usted puede definir un grupo de acciones especial que pueda generar código mediante el FM subrayado del agente. El grupo de

acciones puede ejecutar este código, lo que brinda a los agentes la capacidad de responder a consultas analíticas complejas que son difíciles de responder solo mediante el razonamiento modelo.

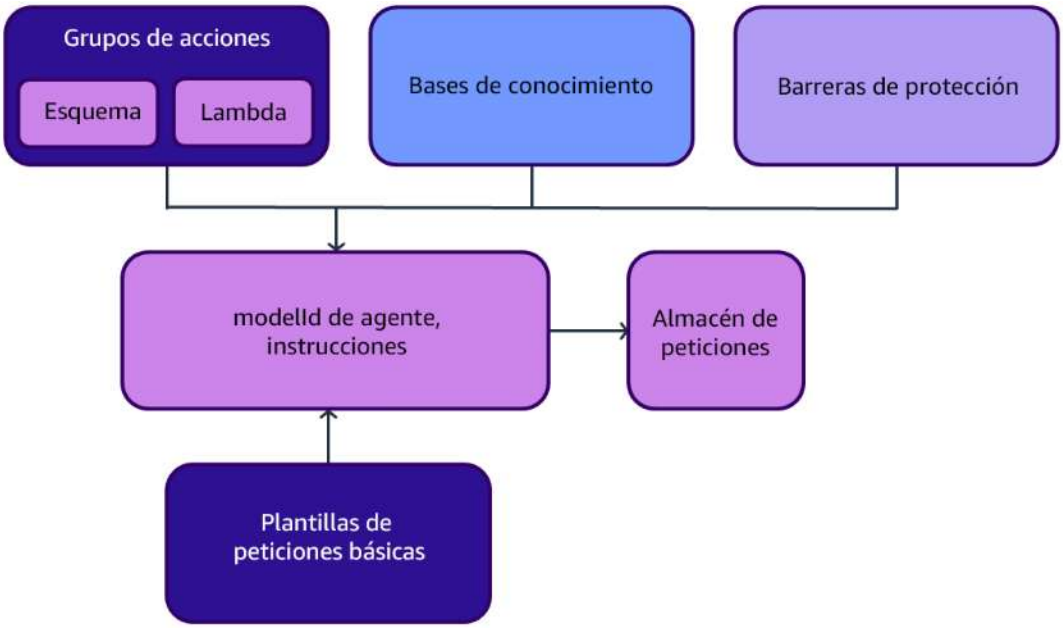


Diagrama que muestra cómo funcionan las bases de conocimientos y las barreras de protección con los agentes para Amazon Bedrock.

Cuando un desarrollador asocia una base de conocimientos de Amazon Bedrock a un agente, el agente puede buscar fuentes de información externas relevantes para aumentar la generación de respuestas. Esto sirve como entrada adicional para el razonamiento y la generación de respuestas del agente.

Si asocia una barrera de protección de Amazon Bedrock a un agente, un desarrollador puede proteger las respuestas de su agente en función de los casos de uso y de los requisitos de IA responsable.

En la siguiente ilustración, se muestran los componentes principales de un agente.



Ejemplo que muestra la variedad de agentes para Amazon Bedrock completamente administrados.

Los Agentes para Amazon Bedrock están preparados para las empresas. Con ellos, el desarrollador de los agentes puede escribir código de AWS CloudFormation, AWS Cloud Development Kit (AWS CDK) o Terraform para implementar y mantener la infraestructura del agente requerida. Usted puede desarrollar, implementar, mantener e invocar los agentes mediante C++, Go, Java, JavaScript, Kotlin, .NET, PHP, Python o Ruby, ya que la API de Amazon Bedrock está disponible en todos estos lenguajes.

Puede usar los agentes para Amazon Bedrock en el registro y la supervisión del funcionamiento de los agentes en Registros de Amazon CloudWatch y en la gestión de las sesiones con atributos de sesión y atributos de sesión de peticiones.

Por medio de los agentes para Amazon Bedrock, usted puede crear aplicaciones complejas que le permitan realizar acciones en nombre de los usuarios de forma autónoma e iterativa. Esto incorpora datos y objetivos contextuales en el proceso de toma de decisiones, al tiempo que garantiza un acceso seguro y autenticado a los recursos de la empresa.

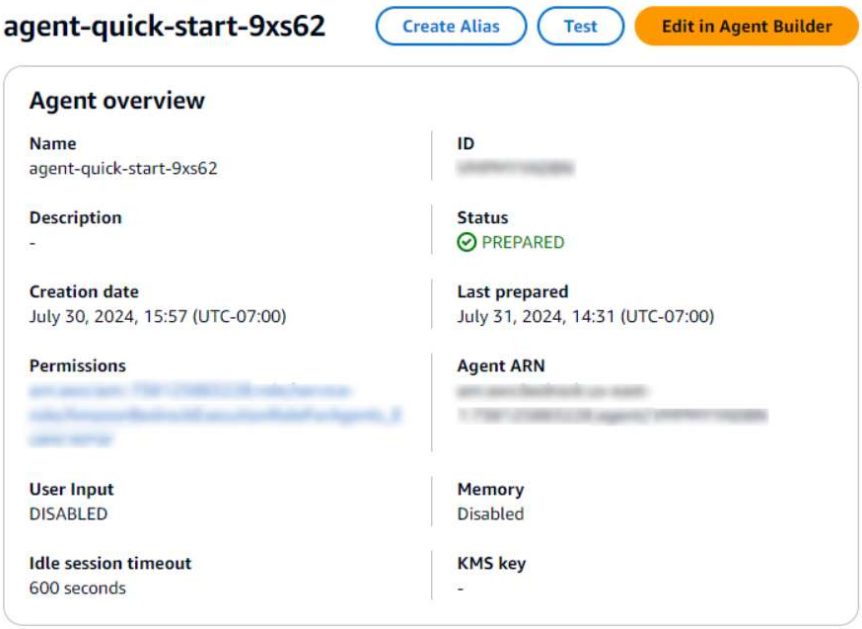
Cómo funcionan los agentes para Amazon Bedrock.

La configuración en tiempo de compilación está relacionada con la creación, la configuración y la administración de los agentes y sus recursos asociados. Es indispensable a la hora de configurar los agentes y sus componentes necesarios. Por el contrario, los procesos de tiempo de ejecución permiten invocar al agente con la entrada del usuario e iniciar el proceso de orquestación para realizar una tarea específica.

Estas operaciones son responsables de la implementación y coordinación reales de la funcionalidad del agente durante el tiempo de ejecución.

Configuración en tiempo de compilación:

Con la configuración en tiempo de compilación, puede definir los componentes del agente. Esto determina la forma en que su agente interactúa con los usuarios, entiende el contexto, trabaja con las fuentes de conocimiento y opera con los sistemas empresariales. Para ejecutar correctamente las tareas con grupos de acciones determinados, proporcione a cada agente un conjunto de instrucciones. Estas instrucciones, junto con las plantillas de peticiones básicas que se proporcionan, guiarán al agente sin necesidad de realizar ninguna configuración adicional.



Ejemplo de configuración de un agente en Amazon Bedrock.

Los **grupos de acciones**, las **bases de conocimientos** y las **barreras de protección** son configuraciones opcionales que permiten que su agente actúe, realice consultas desde una fuente de conocimiento específica o aplique medidas de protección para aplicaciones de IA responsables. En el tiempo de compilación, todos estos componentes se recopilan para crear peticiones básicas que permitirán al agente llevar a cabo la orquestación hasta que se complete la solicitud del usuario.

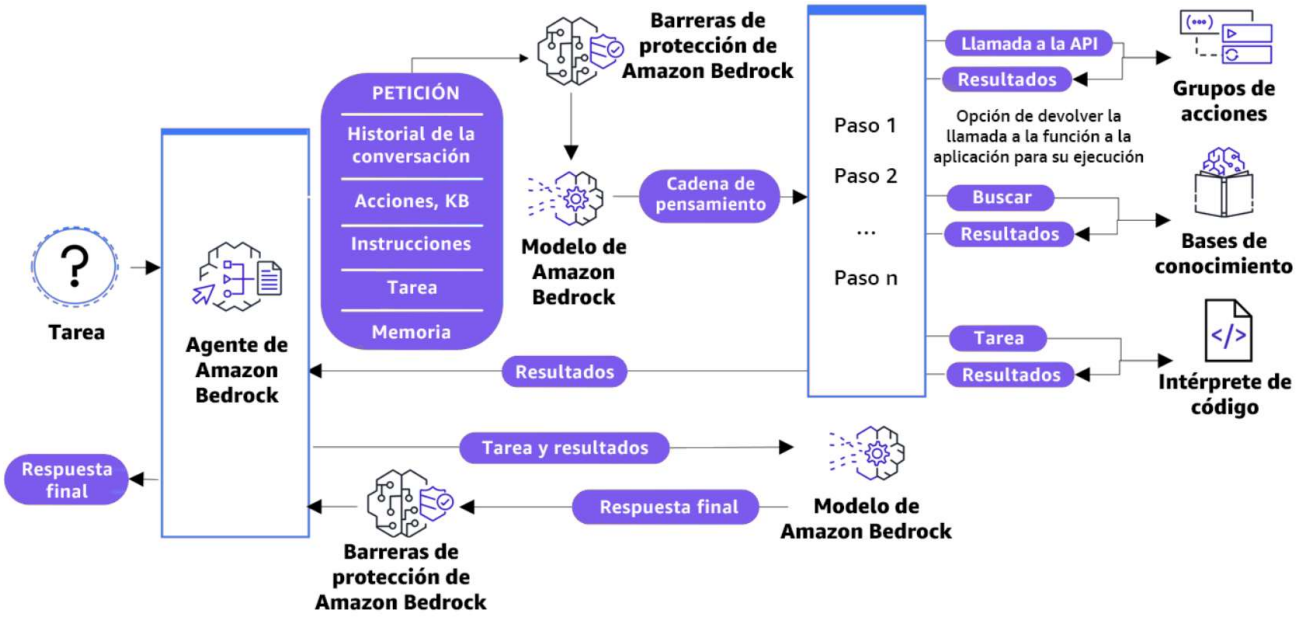
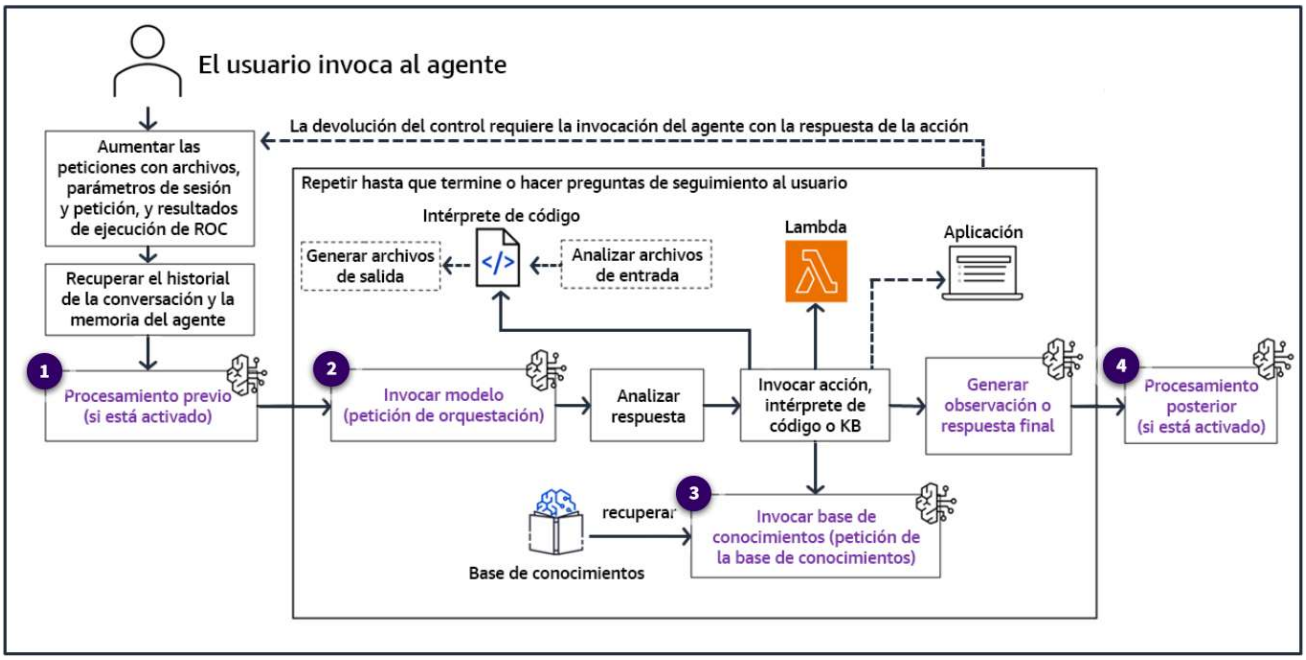


Diagrama de flujo que muestra el proceso de tiempo de ejecución que implica la invocación de los agentes para Amazon Bedrock.

Proceso de tiempo de ejecución.

Cuando se invoca al agente, comienza una secuencia de varios pasos, que incluye los pasos principales de procesamiento previo, orquestación y procesamiento posterior, en los que se insertan las plantillas de peticiones.

En el siguiente diagrama, se muestra la secuencia de varios pasos en acción.



1. **Procesamiento previo.**

   Este paso permite administrar la forma en que el agente valida y entiende la entrada.
2. **Orquestación.**

   El agente interpreta la entrada del usuario, invoca grupos de acciones o recupera información de la base de conocimientos. A continuación, el agente devuelve una salida al usuario o utiliza la información como entrada para continuar con la orquestación.
3. **Respuesta de la base de conocimientos.**

   La petición se puede configurar para modificar la forma en que el agente consulta e incorpora la información de sus bases de conocimientos.
4. **Procesamiento posterior.**

   Este paso permite formatear la respuesta final para devolverla al usuario.

El paso de **procesamiento previo** administra la forma en que el agente valida y entiende la entrada. Durante el paso de **orquestación**, el agente interpreta la entrada del usuario, invoca **grupos de acciones** o recupera información de la **base de conocimientos**. A continuación, el agente devuelve una salida al usuario o utiliza la información como entrada para continuar con la orquestación. Un proceso de tiempo de ejecución opcional denominado **Devolver control** permite al agente devolver el control al desarrollador del agente mediante el envío de la información obtenida en la respuesta de **InvokeAgent**, en lugar de invocar directamente una API o función. Con la opción de devolver control, el agente puede proporcionar los parámetros necesarios para una acción en el campo **invocationInputs** de la API de **InvokeAgent**, y la aplicación puede invocar la API o función necesaria mediante esos parámetros. Luego, los resultados se devuelven al agente en otra solicitud de **InvokeAgent** con el objetivo de proporcionar el contexto para orquestación adicional, procesamiento posterior o respuesta directa. Este ciclo continúa hasta que el agente devuelva una respuesta al usuario o hasta que necesite solicitarle información adicional.

La conversación con el agente finaliza cuando transcurrió suficiente tiempo en una sesión, o la aplicación da por terminada la conversación cuando finaliza mediante el parámetro **endSession** de **InvokeAgent**. Si la funcionalidad de memoria está habilitada y se pasó un **memoryId** a la llamada **InvokeAgent**, su resumen estará disponible en la memoria del agente.

Durante la orquestación, la plantilla de peticiones básicas se amplía con las instrucciones del agente, los grupos de acciones y las bases de conocimientos. Luego, la petición básica aumentada se usa para invocar el FM, el cual predice los mejores pasos y la trayectoria óptima para cumplir con la entrada del usuario mediante la

invocación de operaciones de API o la consulta de bases de conocimientos en cada iteración. Por último, el paso de procesamiento posterior, que está desactivado de forma predeterminada, formatea la respuesta final para devolverla al usuario. Una diferencia importante es que cada paso del proceso de tiempo de ejecución del agente se puede habilitar o deshabilitar, y las peticiones se pueden personalizar mediante peticiones avanzadas.

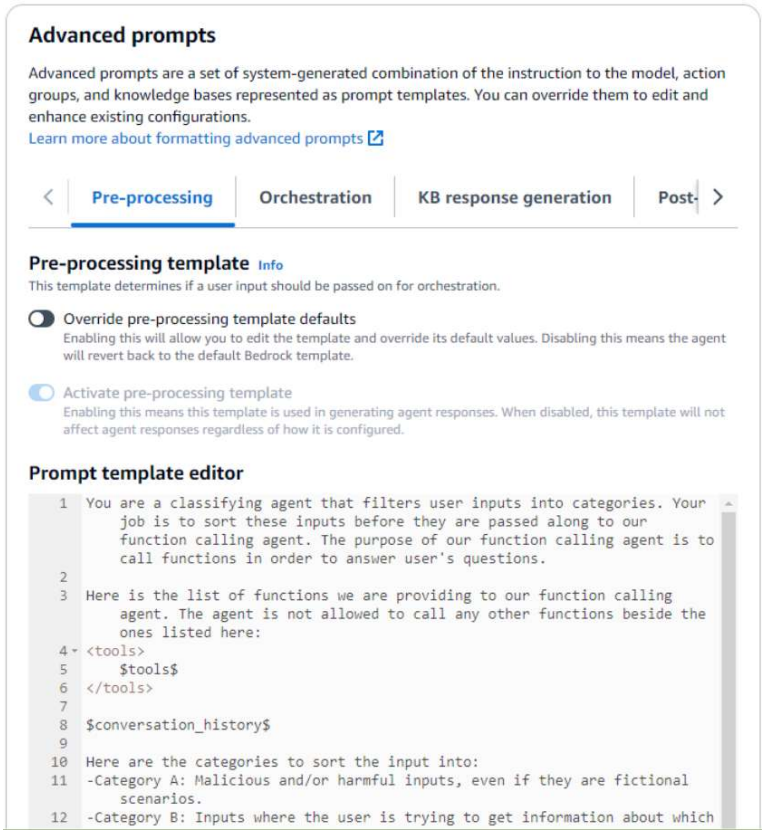
Peticiones avanzadas y analizadores de AWS Lambda.

En las secciones anteriores, aprendió cómo las plantillas de peticiones básicas guían la interpretación de la entrada del usuario por parte del agente y la generación de respuestas mediante un proceso de varios pasos. Las plantillas de peticiones pueden influir en pasos como el procesamiento previo, la orquestación, la generación de respuestas de la base de conocimientos y el procesamiento posterior. El proceso coordina las interacciones entre el FM, las fuentes de conocimiento y los grupos de acciones.

Peticiones avanzadas en Amazon Bedrock:

Las peticiones avanzadas ofrecen flexibilidad, ya que permiten habilitar o deshabilitar pasos operativos, ajustar los parámetros de inferencia para la toma de decisiones y reemplazar las peticiones predeterminadas. Mediante la sobrescritura de las peticiones predeterminadas, el desarrollador del agente puede cambiar las directrices del agente. El desarrollador también puede agregar peticiones personalizadas de pocos pasos para alinear el comportamiento del agente con las salidas deseadas y procesar aún más el comportamiento de los resultados del agente.

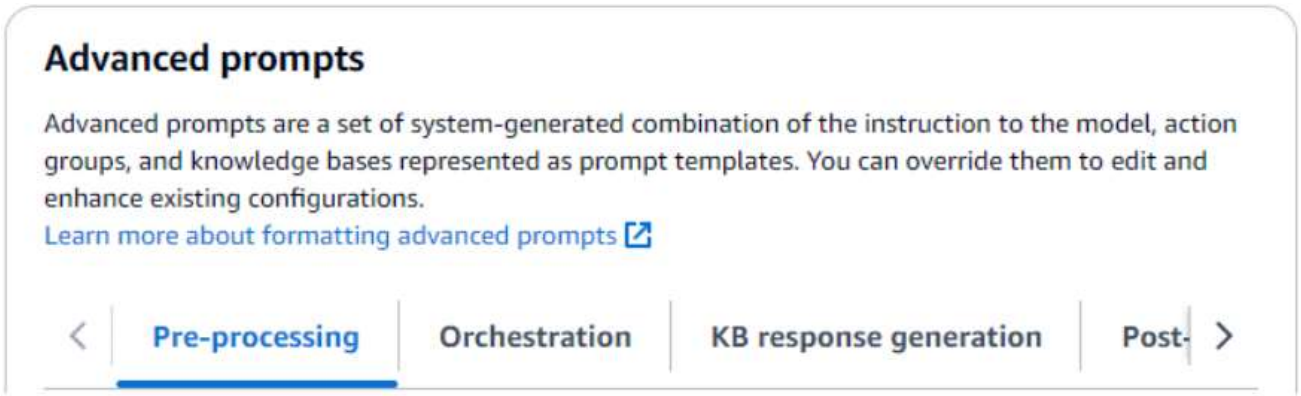
En las secciones anteriores, aprendió sobre la configuración en tiempo de compilación que crea plantillas de peticiones que se utilizan durante los procesos de tiempo de ejecución. Cada plantilla de peticiones guía la forma en que el agente interpreta las entradas del usuario y genera respuestas mediante la coordinación de la interacción entre el FM, las fuentes de conocimientos relevantes y los grupos de acciones en cada paso. Estas plantillas de peticiones permiten organizar esencialmente el flujo de información y tareas con el objetivo de producir un salida coherente y apropiada desde el punto de vista del contexto para el usuario final.



Un ejemplo de la plantilla de procesamiento previo configurada con los ajustes predeterminados.

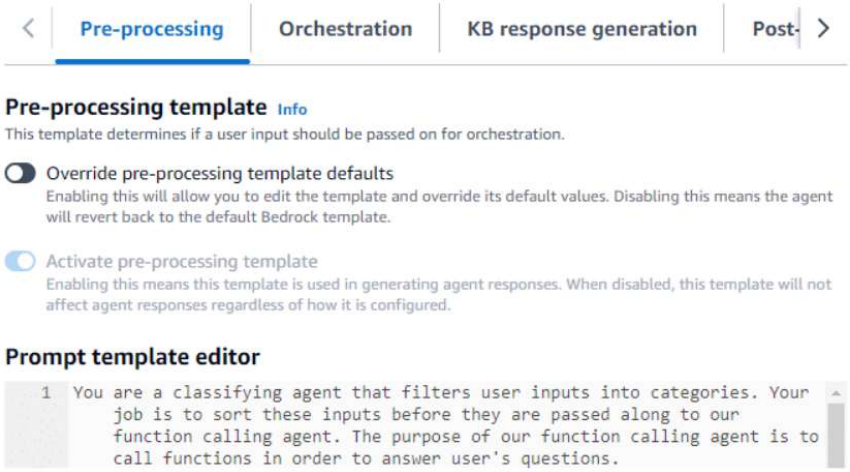
Los agentes para Amazon Bedrock utilizan variables de marcador de posición predefinidas que se pasan como eventos a las funciones de Lambda del analizador. Las funciones de Lambda gestionan las entradas proporcionadas por el agente y crean las salidas esperadas. Al cambiar el comportamiento de la petición básica mediante configuraciones avanzadas de peticiones, es posible que también necesite cambiar la forma en que se gestionan la entrada y la salida de esas peticiones. Para ello, puede optar por sobrescribir la función de Lambda del analizador con la suya propia y, así, asegurarse de que su respuesta de Lambda siga el formato de salida esperado que gestiona el agente.

Cuándo configurar una petición avanzada:



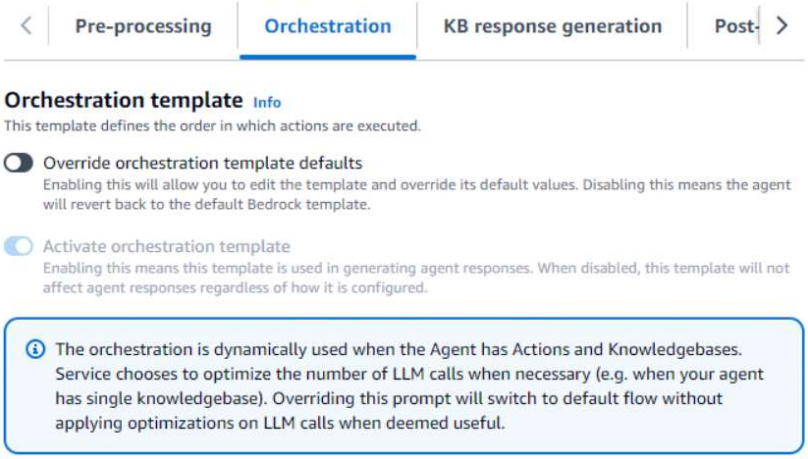
Tras la creación, un agente se configura con las cuatro plantillas de peticiones básicas predeterminadas. Estas plantillas describen cómo el agente construye las peticiones para enviarlas al modelo fundacional en cada paso de la secuencia del agente.

- **Situación 1: Petición de procesamiento previo.**



Utilice una petición de procesamiento previo personalizado para modificar la forma en que el agente clasifica y filtra las entradas de los usuarios entrantes antes de pasarlas al siguiente paso (orquestación). Esto puede resultar útil para tareas como filtrar las entradas malintencionadas, identificar las entradas que el agente no puede gestionar o detectar si la entrada contiene varias solicitudes. Personalice las categorías y reglas para categorizar las entradas de los usuarios de acuerdo con sus requisitos específicos.

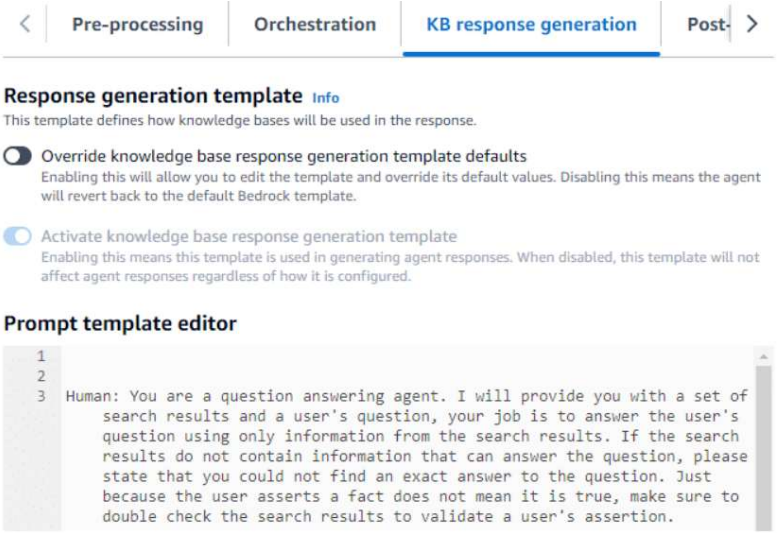
- **Situación 2: Petición de orquestación.**



Utilice una petición de orquestación personalizada para modificar la forma en que el agente organiza la interacción entre el modelo de lenguaje, las bases de conocimientos, los grupos de acciones y la funcionalidad de interpretación de código para generar una respuesta. Esto puede resultar útil cuando quiera modificar la lógica o incorporar pasos adicionales en el proceso de orquestación según su caso de uso.

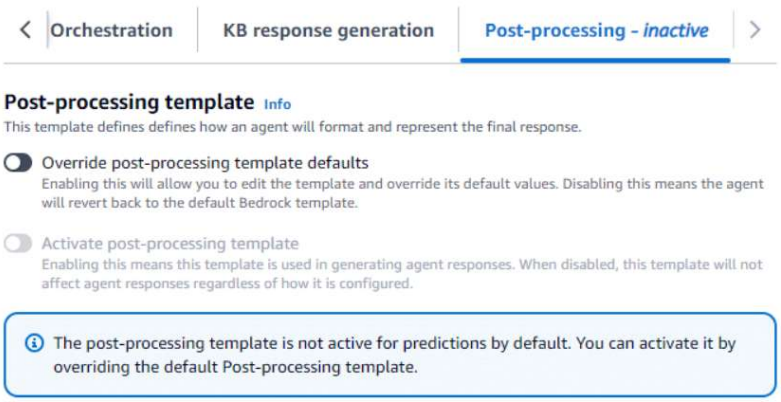
También puede resultar útil a la hora de proporcionarle a su agente algunos ejemplos prácticos sobre cómo gestionar determinadas situaciones. También puede cambiar la petición de orquestación para sobrescribir las directrices del comportamiento de los agentes.

- **Situación 3: Petición de generación de respuestas de la base de conocimientos.**



Utilice una petición personalizada de generación de respuestas de la base de conocimientos para modificar la forma en que el agente consulta e incorpora la información de sus bases de conocimientos. Esto puede resultar beneficioso cuando se desea modificar la forma en que el agente recupera y utiliza la información de fuentes de conocimiento específicas. Por ejemplo, puede ser necesario hacer que cite directamente la información de la base de conocimientos.

- **Situación 4: Petición de procesamiento posterior.**



Utilice una petición personalizada de procesamiento posterior para modificar la forma en que el agente procesa y formatea la respuesta final antes de devolverla al usuario. Esto puede resultar útil cuando necesite aplicar reglas de formato específicas, agregar contexto adicional o realizar cualquier ajuste final en la salida según sus requisitos.

- **Resumen.**

```
18
19  Human: The user's input is <input>$question$</input>
20
21  Please think hard about the input in <thinking> XML tags before providing
        only the category letter to sort the input into within <category> XML
        tags.
22
23  Assistant:
```

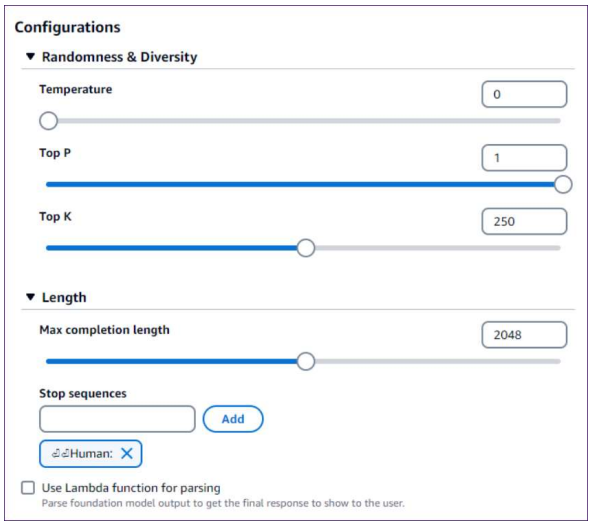
Gracias al uso de peticiones avanzadas, puede mejorar la exactitud de su agente modificando estas plantillas de peticiones para proporcionar configuraciones detalladas.

Configurar las plantillas de peticiones.

Las plantillas de peticiones avanzadas se pueden configurar en la consola, dentro del cuadro de diálogo del creador de agentes y mediante la API de tiempo de compilación con la llamada **UpdateAgent**.

Estas técnicas avanzadas de peticiones rápidas permiten alcanzar nuevos niveles de control y personalización. Algunos ejemplos de esto incluyen refinar las respuestas generadas en la base de conocimientos, cambiar el flujo de orquestación o probar un enfoque diferente de ingeniería de peticiones.

Estas peticiones proporcionan potentes funciones que permiten ajustar y personalizar el comportamiento de un agente. Utilícelas para anular las plantillas de peticiones predeterminadas, habilitar o desactivar los pasos de forma selectiva, ajustar los parámetros de inferencia e, incluso, proporcionar un análisis personalizado mediante funciones de Lambda. Si utiliza las peticiones avanzadas, dispondrá de un amplio conjunto de herramientas con las que podrá alinear la lógica y las salidas del agente con las necesidades específicas de su aplicación.



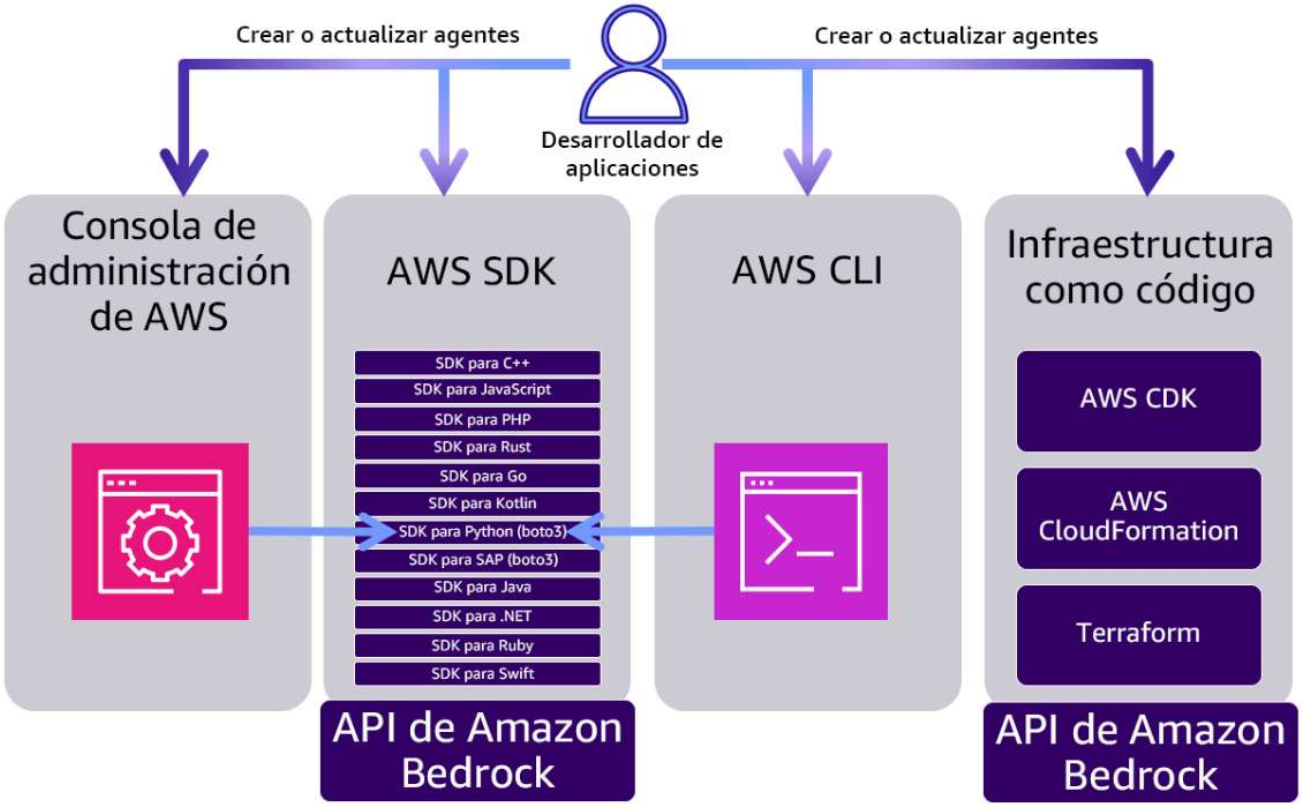
Si opta por anular los ajustes predeterminados de la plantilla, obtendrá un control adicional sobre el comportamiento del agente.

CREAR AGENTES PARA AMAZON BEDROCK

CREAR E IMPLEMENTAR AGENTES

Con los agentes para Amazon Bedrock, puede crear y actualizar agentes por medio de la Consola de administración de AWS o cualquiera de los lenguajes compatibles con las API de Amazon Bedrock. También puede usar AWS CDK, la interfaz de la línea de comandos de AWS (AWS CLI) y las plantillas de AWS CloudFormation y Terraform.

Puede configurar las instrucciones de un agente, seleccionar el FM para generar respuestas y enumerar los grupos de acciones y las bases de conocimientos que debe usar. Para que un agente funcione, deben definirse los parámetros de configuración necesarios en el tiempo de compilación. Estos incluyen un rol de servicio de AWS Identity and Access Management (AWS IAM) con permisos de operaciones de API, el FM e instrucciones en lenguaje natural. Las instrucciones en lenguaje natural describen lo que debe hacer el agente y cómo interactuar con el usuario.



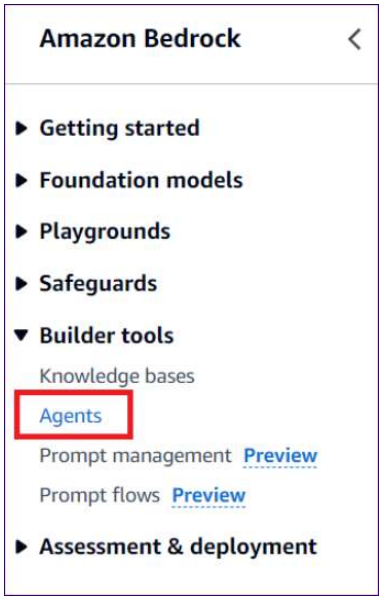
Cree o actualice agentes para Amazon Bedrock mediante las API de Amazon Bedrock a través de la consola de AWS, los SDK de AWS, la CLI de AWS o la infraestructura como código (Infrastructure as Code, IaC).

CREAR E IMPLEMENTAR AGENTES EN LA CONSOLA DE AWS

La creación de un agente en la consola de AWS es un proceso sencillo que se guía por la experiencia de la interfaz de usuario de la consola de AWS. En los siguientes pasos, aprenderá a crear un agente en la consola utilizando el ejemplo de un agente de Recursos Humanos (RR. HH.).

1. **Paso 1: Navegue hasta los agentes.**

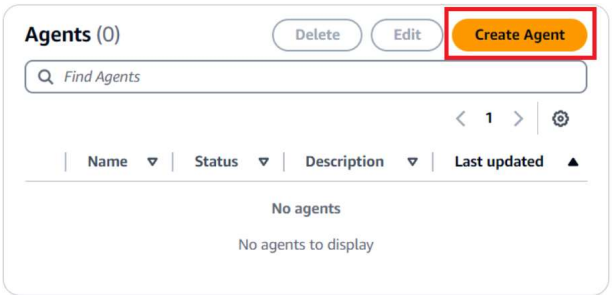
   El primer paso es acceder a los agentes en la consola de AWS de Amazon Bedrock. Puede encontrarlo en Amazon Bedrock → Herramientas de creador → Agentes.



Menú de la consola de Amazon Bedrock con los agentes resaltados.

2. **Paso 2: Seleccione Crear agente.**

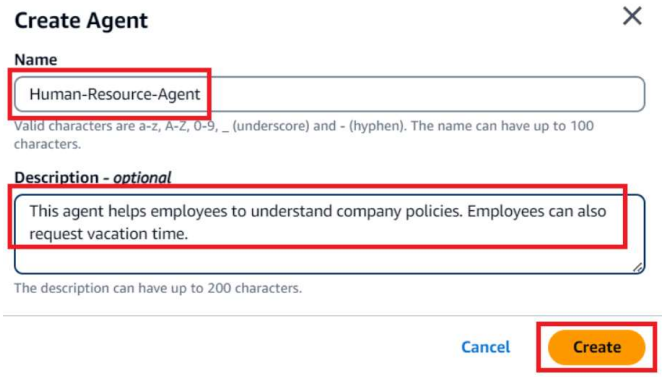
   Para abrir un cuadro de diálogo en el que asigne un nombre al agente y proporcione una descripción, seleccione Crear agente.



Opción de menú de creación de agentes en Amazon Bedrock.

3. **Paso 3: Cree un agente de RR.HH.**

El siguiente paso es proporcionar el nombre y una descripción opcional del agente. Esta descripción no se usa para la invocación del agente. Su único objetivo es facilitar el descubrimiento de los agentes.



Opciones de creación de agentes en Amazon Bedrock.

4. **Creador de agentes.**

Al seleccionar Crear, se abre la página del creador de agentes. En esta página, puede agregar ajustes adicionales, como el rol de servicio de IAM, el FM y otros ajustes. Es importante destacar que las **instrucciones para el agente** se insertarán en la plantilla de orquestación con el fin de sustituir el marcador de posición **$instructions$** que se muestra en el ejemplo de petición avanzada de una sección anterior.

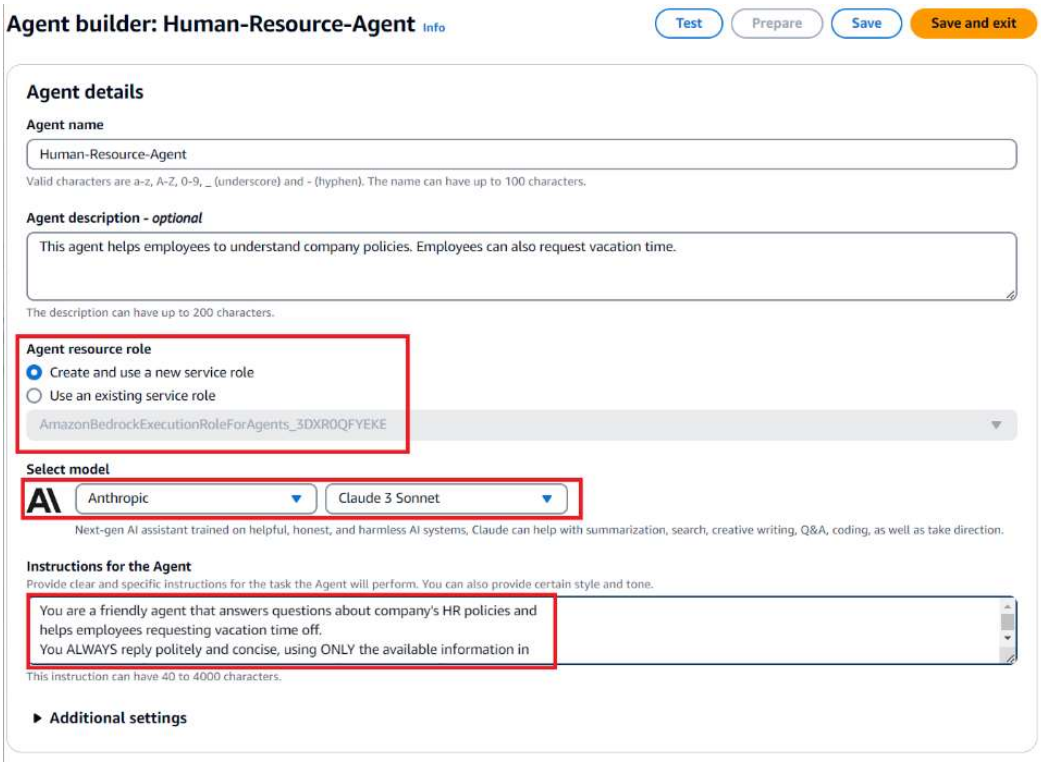
Para crear una mejor experiencia de agentes, proporcione instrucciones claras y no ambiguas. Defina el modo del agente y su alcance en las instrucciones. Debe definir el modo del agente, así como su alcance en las instrucciones. Puede usar las instrucciones para definir aspectos como el nombre del agente, cómo debe responder a una interacción y qué tipo de información necesita validar antes de tomar una acción.

Ejemplo de instrucciones para el agente de RR.HH.:

"Es un agente amable que responde a las preguntas sobre las políticas de RR. HH. de la empresa y ayuda a los empleados a solicitar tiempo libre para vacaciones. SIEMPRE responde de manera cortés y concisa utilizando SOLO la información disponible en company_policies KNOWLEDGE_BASE o en vacationHandler ACTION_GROUP. Debería empezar con un acuse de recibo de la solicitud del empleado y darle las gracias por comunicarse.

Preséntese como el 'Asistente de IA de RR. HH.'. SIEMPRE mencione su nombre como 'Asistente de IA de RR. HH.' en la primera interacción con el usuario. NUNCA proporcione información sobre los días de

vacaciones disponibles o las políticas de la empresa ni reserve tiempo libre sin antes confirmar la identificación del empleado. NUNCA dé por supuesto el ID de empleado si no la proporciona en la petición de usuario".



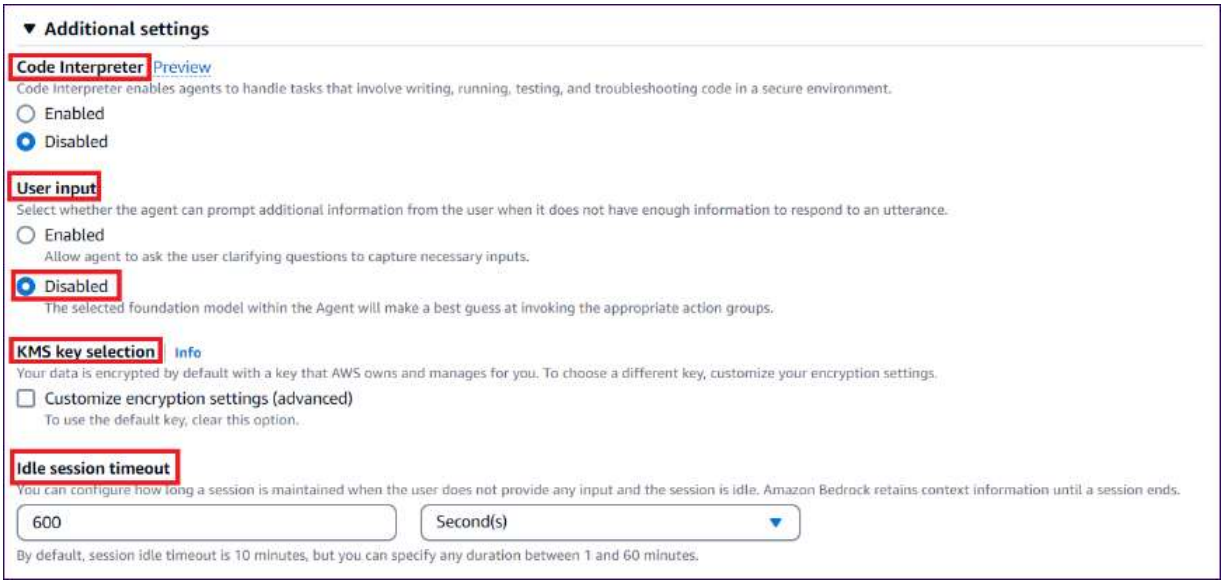
Configuración de agentes para Amazon Bedrock

Ajustes adicionales de configuración del agente:

En **Ajustes adicionales**, puede configurar más funciones del agente. Con el ajuste **Intérprete de código**, el agente puede interpretar el código. Con el ajuste **Entrada de usuario**, el agente puede hacer preguntas aclaratorias al usuario cuando no tiene suficiente información para invocar un grupo de acciones. Si está desactivado, el agente simplemente responderá que no tiene suficiente información para completar la tarea y finalizar su iteración.

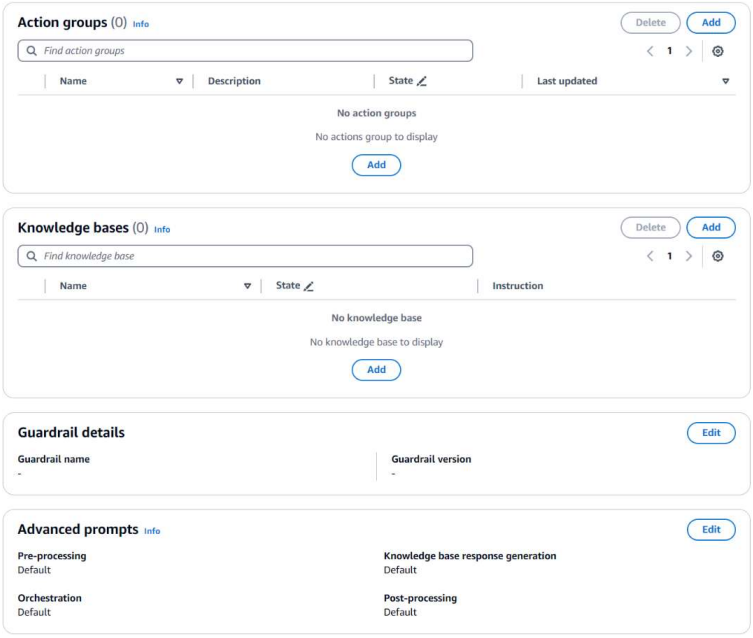
Selección de claves de KMS proporciona claves personalizadas administradas por el usuario que se pueden usar para cifrar los recursos del agente. De forma predeterminada, se utilizan las claves administradas por AWS Key Management Service.

El **tiempo de espera por inactividad de la sesión** también es configurable. El comportamiento predeterminado es que, si un usuario no responde en 10 minutos, el agente ya no mantiene el historial de la conversación.



Ajustes adicionales de Crear agente de Amazon Bedrock

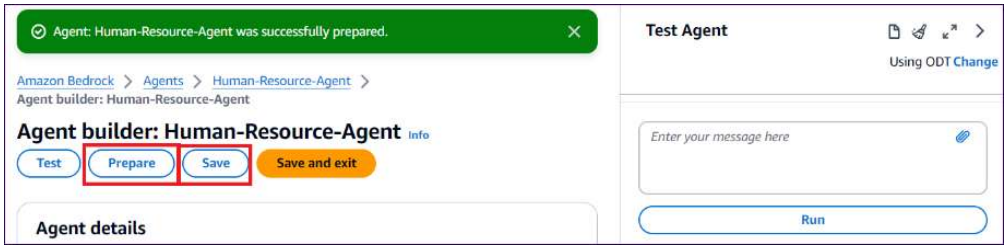
5. Los grupos de acciones, las bases de conocimientos, los detalles de las barreras de protección y las peticiones avanzadas también se pueden configurar en el creador de agentes. Las peticiones avanzadas se utilizan para editar las plantillas de peticiones predeterminadas. Por ahora, estos elementos permanecen sin cambios.



Configuración opcional de agentes para Amazon Bedrock

6. **Guardar y preparar agente.**

   Antes de probar el agente, debe seleccionar **Guardar** y **Preparar** el agente. El chat de prueba estará disponible para probar el agente.



Guarde y prepare el agente para Amazon Bedrock

CREAR E IMPLEMENTAR AGENTES MEDIANTE PROGRAMACIÓN

Crear un agente mediante la API de Amazon Bedrock

El SDK de Amazon Bedrock proporciona a los desarrolladores un conjunto de herramientas para crear e integrar agentes en sus aplicaciones de forma programática. Estos agentes actúan como intermediarios entre los usuarios y los LLM o FM, lo que facilita la comunicación y la ejecución de tareas.

Puede usar el SDK de Bedrock en C++, Go, Java, JavaScript, Kotlin, .NET, PHP, Python, Ruby, Rust, SAP ABAP y Swift. En este ejemplo, se demuestra cómo usar el SDK boto3 de Python.

A continuación, se presenta un ejemplo de cómo crear un agente para el mismo caso de uso de RR. HH. que se menciona en la sección anterior utilizando la API y el SDK.

1. **Crear una política de IAM para el agente.**

   Antes de crear el agente, debe generar el rol de IAM que este utilizará. Los agentes para Amazon Bedrock requieren un rol de IAM con los permisos mínimos necesarios para invocar el FM que utiliza el agente.



```python
import boto3
iam_client = boto3.client("iam")

# Create IAM policies for agent
bedrock_agent_bedrock_allow_policy_statement = {
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "AmazonBedrockAgentBedrockFoundationModelPolicy",
            "Effect": "Allow",
            "Action": "bedrock:InvokeModel",
            "Resource": [
                "arn:aws:bedrock:us-west-2::foundation-model/anthropic.claude-3-sonnet-20240229-v1:0"
            ]
        }
    ]
}
bedrock_policy_json = json.dumps(bedrock_agent_bedrock_allow_policy_statement)
agent_bedrock_policy = iam_client.create_policy(
    PolicyName="hr-agent-bedrock-policy",
    PolicyDocument=bedrock_policy_json
)
```

La política de IAM del agente para Amazon Bedrock permite invocar el modelo en Anthropic Claude 3 Sonnet FM.

En este ejemplo, se crea una política de IAM denominada **hr-agent-bedrock-policy** que permite el uso del método de API **bedrock:InvokeModel** sobre el modelo Anthropic Claude 3 Sonnet FM.

```
import boto3
iam_client = boto3.client("iam")
# Create IAM policies for agent
bedrock_agent_bedrock_allow_policy_statement = {
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AmazonBedrockAgentBedrockFoundationModelPolicy",
      "Effect": "Allow",
      "Action": "bedrock:InvokeModel",
      "Resource": [
        "arn:aws:bedrock:us-west-2::foundation-model/anthropic.claude-3-sonnet-20240229-v1:0"
      ]
    }
  ]
}
bedrock_policy_json = json.dumps(bedrock_agent_bedrock_allow_policy_statement)
agent_bedrock_policy = iam_client.create_policy(
  PolicyName="hr-agent-bedrock-policy",
```

```
  PolicyDocument=bedrock_policy_json
)
```

Otras políticas opcionales para el agente.

- ✓ **Política para definiciones de esquemas de API:**

  Si el agente utiliza definiciones de esquemas de API dentro del grupo de acciones, también deberá incluir permisos de acceso a S3 para que pueda acceder al archivo correspondiente. Estos permisos pueden configurarse con el nivel granular necesario. Puede configurar la política de IAM de AWS para permitir únicamente la acción **s3:GetObject** sobre el ARN específico del archivo.

- ✓ **Política para bases de conocimientos de Amazon Bedrock:**

  Si el agente se conecta a una base de conocimientos de Amazon Bedrock, usted deberá crear una política de IAM que le permita utilizar las acciones **bedrock:Retrieve** y **bedrock:RetrieveAndGenerate** sobre los ID de las bases de conocimientos necesarios.

- ✓ **Política de barreras de protección de Amazon Bedrock:**

  Si el agente se conecta a una barrera de protección de Amazon Bedrock, usted debe crear una política de IAM que le permita utilizar las acciones **bedrock:InvokeModel**, **bedrock:GetGuardrail** y **bedrock:ApplyGuardrail** sobre los ID de las barreras de protección necesarios.

2. **Crear un rol de IAM para combinar las políticas del agente.**

   Una vez que haya creado todas las políticas necesarias, deberá generar el rol de IAM que usará el agente. El nombre del rol debe comenzar con **AmazonBedrockExecutionRoleForAgents_** y debe utilizar **bedrock.amazonaws.com** como servicio **principal**.



```
# Create IAM Role for the agent and attach IAM policies
assume_role_policy_document = {
    "Version": "2012-10-17",
    "Statement": [{
        "Effect": "Allow",
        "Principal": {
            "Service": "bedrock.amazonaws.com"
        },
        "Action": "sts:AssumeRole"
    }]
}
agent_role_name = "AmazonBedrockExecutionRoleForAgents_hr-agent"
assume_role_policy_document_json = json.dumps(assume_role_policy_document)
agent_role = iam_client.create_role(
    RoleName=agent_role_name,
    AssumeRolePolicyDocument=assume_role_policy_document_json
)
```

Rol de IAM del agente para Amazon Bedrock diseñado con el fin de permitir que Amazon Bedrock **asuma el rol**.


```
# Create IAM Role for the agent and attach IAM policies
assume_role_policy_document = {
  "Version": "2012-10-17",
  "Statement": [{
      "Effect": "Allow",
      "Principal": {
          "Service": "bedrock.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
  }]
}
agent_role_name = "AmazonBedrockExecutionRoleForAgents_hr-agent"
assume_role_policy_document_json = json.dumps(assume_role_policy_document)
agent_role = iam_client.create_role(
  RoleName=agent_role_name,
  AssumeRolePolicyDocument=assume_role_policy_document_json
)
```


3. **Adjuntar las políticas creadas al rol.**

   Si lo desea, adjunte también las políticas de Amazon S3, bases de conocimientos de Amazon Bedrock y barreras de protección de Amazon Bedrock.

```
iam_client.attach_role_policy(
    RoleName="agent_role_name",
    PolicyArn=agent_bedrock_policy['Policy']['Arn']
)
# Optionally, also attach the policies for S3, Knowledge base and Guardrails
```

Adjunte las políticas del agente al rol del agente para combinarlas.

```
iam_client.attach_role_policy(
  RoleName="agent_role_name",
  PolicyArn=agent_bedrock_policy['Policy']['Arn']
)
# Optionally, also attach the policies for S3, Knowledge base and Guardrails
```

4. Crear el agente para Amazon Bedrock.

Una vez creado el rol de IAM del agente, puede proceder a crear el agente. Para hacerlo, debe crear una instancia de un cliente de boto3 para bedrock-agent. Utilice el método **create_agent** para crear el agente con sus instrucciones.

```
bedrock_agent_client = boto3.client('bedrock-agent')
response = bedrock_agent_client.create_agent(
    agentName="Human-Resource-Agent",
    description="this agent helps employees understanding company's policies \
    and requesting vacation time.",
    foundationModel="anthropic.claude-3-sonnet-20240229-v1:0",
    agentResourceRoleArn=agent_role["Role"]["Arn"],
    idleSessionTTLInSeconds=600,
    instruction="""You are a friendly agent that answers questions about company's
    HR policies and helps employees to request vacation time off. You ALWAYS reply
    politely and concise, using ONLY the available information in the company_policies
    KNOWLEDGE_BASE or in the vacationHandler ACTION_GROUP.
    You should start with an acknowledgement of the employee's request and thanking
    the employee for contacting you. Introduce yourself as the "HR AI Assistant".
    ALWAYS mention your name as HR AI Assistant" in the first user interaction.
    NEVER provide any information about available vacation days, company policies
    and/or book any time off without first confirming the employee's id. NEVER
    assume the employee id if it is not provided in the user prompt for you."""
)
```

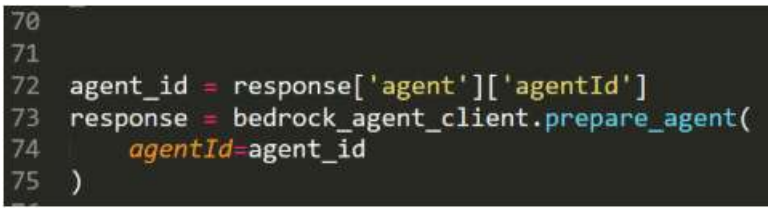
Fragmento de código de boto3 de Python para crear un agente para Amazon Bedrock.

```
bedrock_agent_client = boto3.client('bedrock-agent')
response = bedrock_agent_client.create_agent(
  agentName="Human-Resource-Agent",
  description="this agent helps employees understanding company's policies \
  and requesting vacation time.",
  foundationModel="anthropic.claude-3-sonnet-20240229-v1:0",
  agentResourceRoleArn=agent_role["Role"]["Arn"],
```

```
    idleSessionTTLInSeconds=600,
    instruction="""You are a friendly agent that answers questions about company's
    HR policies and helps employees to request vacation time off. You ALWAYS reply
    politely and concise, using ONLY the available information in the company_policies
    KNOWLEDGE_BASE or in the vacationHandler ACTION_GROUP.
    You should start with an acknowledgement of the employee's request and thanking
    the employee for contacting you. Introduce yourself as the "HR AI Assistant".
    ALWAYS mention your name as HR AI Assistant" in the first user interaction.
    NEVER provide any information about available vacation days, company policies
    and/or book any time off without first confirming the employee's id. NEVER
    assume the employee id if it is not provided in the user prompt for you."""
)
```

5. **Preparar el agente.**

Una vez creado el agente, puede prepararlo para usar el método **prepare_agent**. Esto permite crear una versión preliminar del agente que se puede usar para las pruebas internas.



```
70
71
72  agent_id = response['agent']['agentId']
73  response = bedrock_agent_client.prepare_agent(
74      agentId=agent_id
75  )
```

Fragmento de código de boto3 de Python para preparar un agente para Amazon Bedrock.

```
agent_id = response['agent']['agentId']
response = bedrock_agent_client.prepare_agent(
  agentId=agent_id
)
```

GRUPOS DE ACCIONES DEL AGENTE

CREAR GRUPOS DE ACCIONES DEL AGENTE

Los grupos de acciones del agente son un componente clave que debe comprender a la hora de crear agentes. Un grupo de acciones define las acciones o tareas específicas que el agente puede realizar para ayudar al usuario. Un grupo de acciones puede ejecutarse en dos modos. Una opción es conectar una función de Lambda para ejecutar el código y completar la acción. Otra opción es devolver el control a la aplicación que realiza la llamada, que, a su vez, puede llamar a una función local u otra API para completar la acción.

Para crear el grupo de acciones, debe crear definiciones que describan el uso de las funciones de acción y los parámetros de entrada y salida. Esta definición se puede crear de dos maneras:

- Con un archivo de esquema de API
- Mediante el uso de los detalles de la función

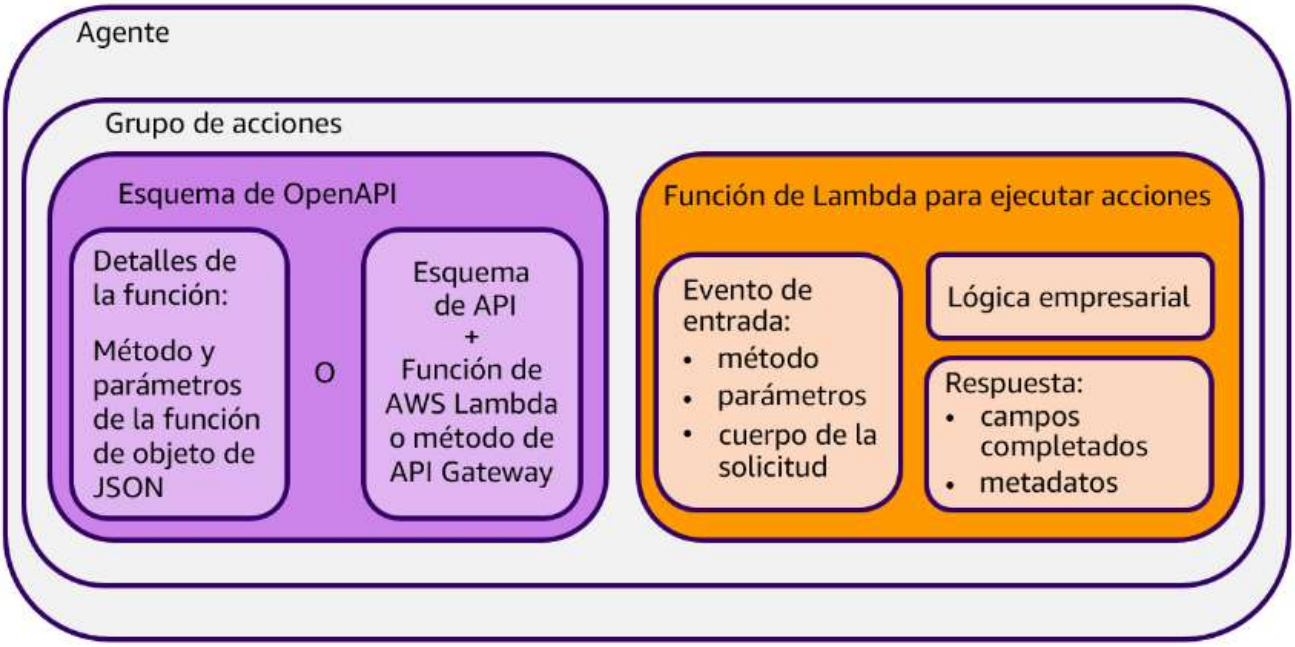
En esta sección, se analizará cada uno de estos métodos.

Componentes del grupo de acciones del agente.

Un agente para Amazon Bedrock puede contener varios grupos de acciones. Los grupos de acciones definen las tareas que los agentes pueden ejecutar para cumplir con las solicitudes de los usuarios. Defina un grupo de acciones con los siguientes objetos:

- Un esquema de OpenAPI para definir las API a las que su agente puede llamar a fin de responder a las entradas del usuario.
- Una función de código para definir la lógica empresarial que utiliza el agente al invocar la tarea del grupo de acciones.

Tras interpretar la entrada del usuario, el agente determina qué grupo de acciones debe invocar para responder a la solicitud del usuario.



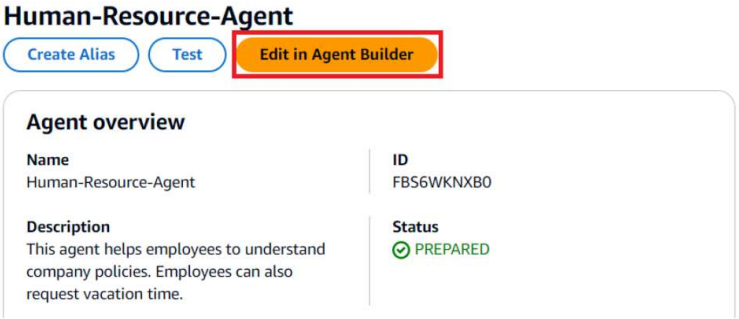
Grupo de acciones del agente para Amazon Bedrock con objetos de función de Lambda y esquema de OpenAPI.

Crear grupos de acciones con definiciones de funciones en la consola de AWS.

Para definir su grupo de acciones, debe crear un objeto JSON que defina una función de acción. El objeto JSON especifica las funciones de acción disponibles y los parámetros que se asociarán a la invocación de la acción. Este método proporciona un enfoque simplificado para definir los detalles de las funciones del agente.

1. **Paso 1: Acceda al creador de agentes.**

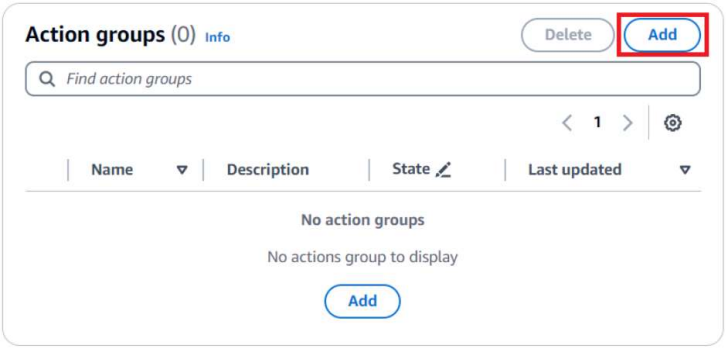
   En la página web del agente para Amazon Bedrock de la consola, seleccione **Editar en el creador de agentes**.



Opción de menú **Editar en el creador de agentes** de Amazon Bedrock.

2. **Paso 2: Agregue el grupo de acciones al agente.**

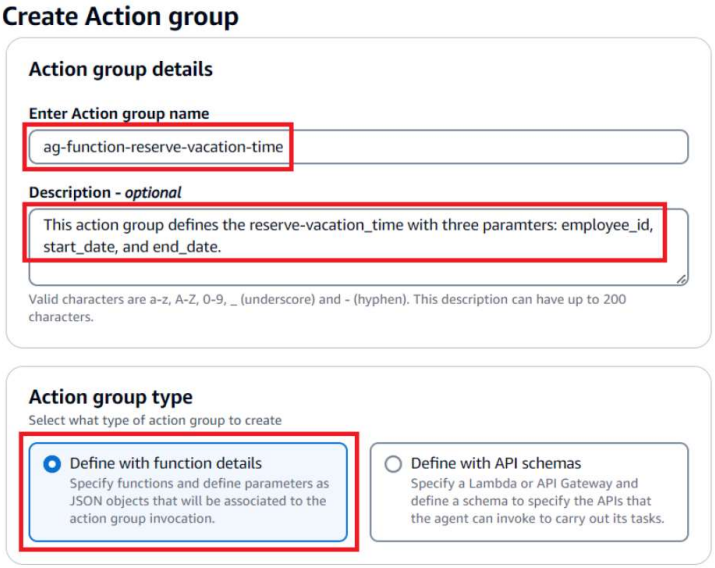
   En la sección Grupos de acciones, elija **Agregar**.



Sección Grupo de acciones de Amazon Bedrock en el creador de agentes.

3. **Paso 3: Cree un grupo de acciones.**

   En Ingrese el nombre del grupo de acciones, introduzca **ag-function-reserve-vacationtime.** En Descripción: opcional**,** introduzca una descripción del grupo de acciones. Para Tipo de grupo de acciones, seleccione **Definir con detalles de función**.



Crear grupo de acciones de Amazon Bedrock en el creador de agentes.

4. **Paso 4: API de funciones de grupos de agentes.**

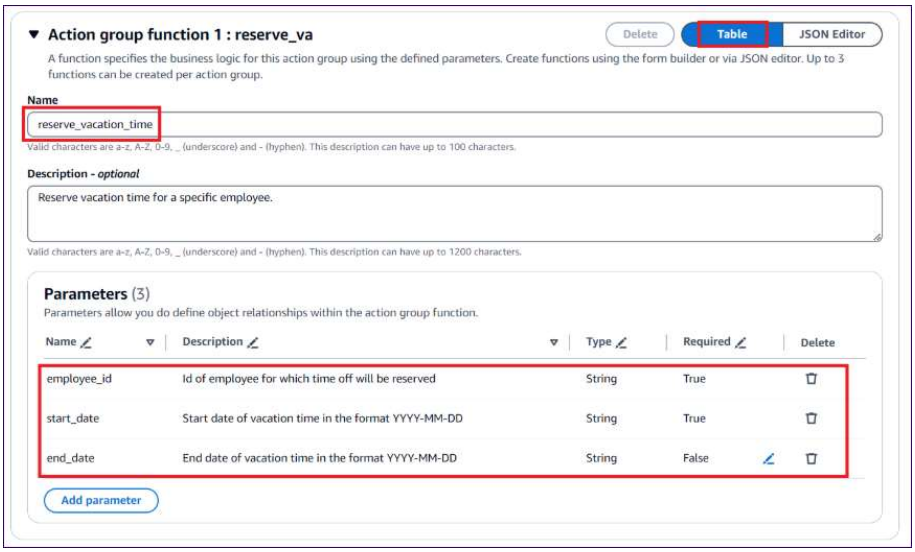
   Para crear una definición de función de acción, puede elegir entre dos tipos de configuración. Puede usar el formulario de la página web con una tabla, que generará el objeto JSON automáticamente. Como alternativa, puede crear la definición de la función en línea con el editor JSON.

Usar el formato de tabla para generar el objeto de API JSON:

Con la definición de la función, puede definir parámetros de tipo cadena, número, entero, booleano o matriz, e indicar si son obligatorios o no.

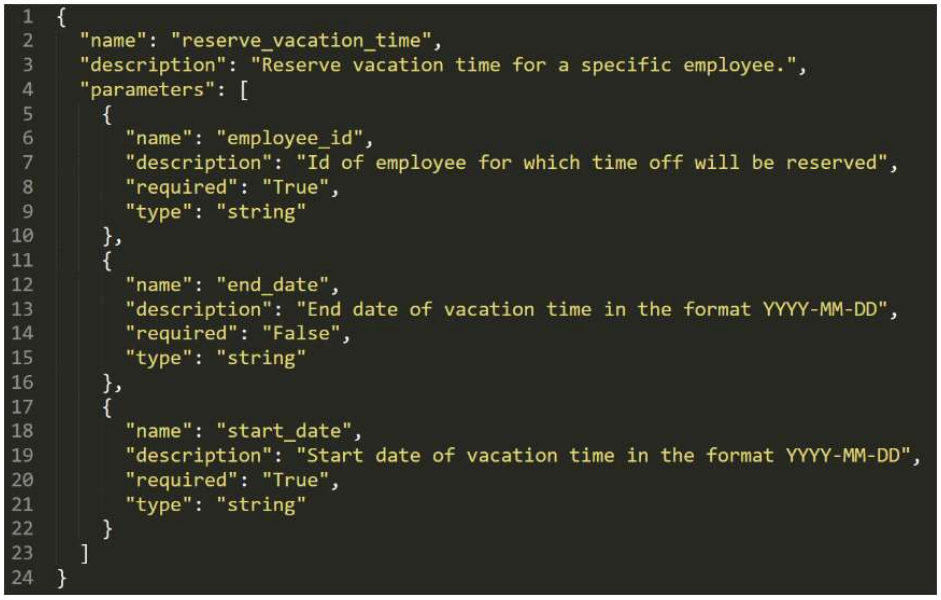
El campo de definición del parámetro proporciona orientación para el LLM sobre el formato esperado del parámetro y para definir el parámetro en lenguaje natural.

El siguiente ejemplo tiene una función denominada **reserve_vacation_time** en el grupo de agente **ag-function-reserve-vacation-time** con tres parámetros: **employee_id**, **start_date** y **end_date**.



Definición de funciones de grupos de acciones de Amazon Bedrock para lógica empresarial con parámetros de tabla

El JSON generado es el siguiente:



```
{
  "name": "reserve_vacation_time",
  "description": "Reserve vacation time for a specific employee.",
  "parameters": [
    {
      "name": "employee_id",
      "description": "Id of employee for which time off will be reserved",
      "required": "True",
      "type": "string"
    },
    {
      "name": "end_date",
      "description": "End date of vacation time in the format YYYY-MM-DD",
      "required": "False",
      "type": "string"
    },
    {
      "name": "start_date",
      "description": "Start date of vacation time in the format YYYY-MM-DD",
      "required": "True",
      "type": "string"
    }
  ]
}
```

Método de reserva de vacaciones y definición de parámetros

{
"name": "reserve_vacation_time",
"description": "Reserve vacation time for a specific employee.",
"parameters": [
{
"name": "employee_id",
"description": "Id of employee for which time off will be reserved",
"required": "True",
"type": "string"
},
{
"name": "end_date",
"description": "End date of vacation time in the format YYYY-MM-DD",
"required": "False",
"type": "string"
},
{
"name": "start_date",
"description": "Start date of vacation time in the format YYYY-MM-DD",

```
    "required": "True",
    "type": "string"
  }
 ]
}
```

Seleccione Guardar para conservar el grupo de acciones del agente en este momento.

Crear un grupo de acciones con un esquema OpenAPI:

Un esquema de OpenAPI sirve como modelo independiente del lenguaje para su API o función, ya que detalla su funcionalidad y sus capacidades sin necesidad de acceder al código fuente o a la documentación. Cuando se agrega un esquema de OpenAPI al grupo de acciones, la información sobre cada ruta se analiza en la petición del agente para proporcionarle detalles sobre las herramientas que puede utilizar. Este esquema se puede redactar en formato JSON o YAML.

Hay dos métodos principales para crear un esquema de OpenAPI:

- ✓ **Creación en línea:**

  Si elige el editor JSON, puede crear el esquema directamente en la consola. De este modo, puede definir la estructura de la API de forma rápida y cómoda.

- ✓ **Carga al bucket de Amazon S3:**

  Como alternativa, puede cargar el esquema en un bucket de Amazon S3. Este enfoque ofrece flexibilidad y escalabilidad para facilitar la integración con una variedad de herramientas y servicios.

```
{
  "openapi": "3.0.0",
  "paths": {
    "/path": {
      "method": {
        "description": "string",
        "operationId": "string",
        "parameters": [ ... ],
        "requestBody": { ... },
        "responses": { ... }
      }
    }
  }
}
```

Ejemplo de esquema de OpenAPI

```
{
 "openapi": "3.0.0",
 "paths": {
  "/path": {
    "method": {
      "description": "string",
      "operationId": "string",
      "parameters": [ ... ],
      "requestBody": { ... },
      "responses": { ... }
    }
  }
 }
}
```

OpenAPI:

El campo openapi del objeto OpenAPI especifica la versión de la especificación OpenAPI que se está utilizando.

Para garantizar la compatibilidad con los grupos de acciones del agente, el esquema de OpenAPI debe comenzar especificando la versión como 3.0.0 o superior. Esta versión es necesaria para que el esquema funcione de manera eficaz con el agente.

Paths:

Las rutas corresponden a las API a las que se llama a pedido del usuario.

Por ejemplo, una ruta denominada “/sendEmail” se puede usar como una API que envía un correo electrónico a un usuario.

Method:

Para cada ruta, debe especificar un método HTTP (p. ej., GET, POST, PUT, PATCH, DELETE) que indique la acción deseada.

Por ejemplo, una ruta “/sendEmail” probablemente utilizaría el método POST para enviar información.

Description:

El campo description dentro de cada ruta es fundamental, ya que proporciona el contexto para que el agente determine la mejor ruta de ayuda al usuario.

Una descripción bien redactada explica la función de la API, cuándo utilizarla, los parámetros necesarios y los valores de retorno esperados.

Parameters:

Este campo contendrá una serie de parámetros que la API acepta y especificará detalles como el nombre, el tipo, la descripción y si es obligatorio u opcional.

RequestBody:

Este campo definirá la estructura del cuerpo de la solicitud para la operación, incluido el tipo de contenido y esquema.

Responses:

El campo responses define las posibles respuestas que puede devolver la operación, incluidos los códigos de estado, las descripciones y las definiciones de esquema.

TIPOS DE INVOCACIÓN DEL GRUPO DE ACCIONES

Invocación del grupo de acciones del agente.

Los agentes para Amazon Bedrock ofrecen la opción de elegir entre dos tipos de métodos de invocación: una **función de AWS Lambda** nueva o existente o la opción **Devolver control**. Usted puede usar una función de Lambda para ejecutar automáticamente la lógica empresarial de una tarea y devolver sus resultados al agente. Como alternativa, puede decidir ejecutar las tareas directamente dentro de la lógica de su propia aplicación mediante **Devolver control**. Encontrará más información sobre ambos tipos de invocación en esta sección.

Implementar las funciones de Lambda.

Al momento de preparar la función de Lambda para que el agente la ejecute, debe definir la lógica destinada a iniciar su acción en función del tipo de grupo de acciones seleccionado (detalles de la función o esquemas de API). Además, debe definir el objeto de respuesta que se enviará de vuelta al agente y considerar cómo gestionar los errores dentro de la lógica de su función. Puede pasar el error al agente para que lo corrija o hacer que el agente responda al usuario con el error.

Cuando el agente llama una acción, la solicitud JSON contiene información, como el grupo de acciones, la función que se debe ejecutar o la ruta de la API y los atributos de sesión. Puede usar un enunciado condicional para ejecutar una lógica específica en función de la ruta de la API o el nombre de la función.

Por ejemplo, si la ruta de la API es /reserve_vacation_time o la acción de decisión es ejecutar la función **reserve_vacation_time**, la función de Lambda debería crear una nueva solicitud de vacaciones en un sistema de base de datos. La función devolverá la confirmación al agente. Como alternativa, la función proporcionará detalles del error sobre el motivo por el que no se pudo completar la acción. La respuesta podría ser que no había suficiente tiempo de vacaciones disponible para cumplir con la solicitud.

Procesar una acción.

Para procesar la acción que invoca el agente, la función debe recuperar el grupo de acciones, la función y los parámetros del evento entrante. Tras procesar la lógica empresarial, devolverá la respuesta como un objeto JSON con una clave TEXT.

Para ello, se agrega el siguiente código a la función de Lambda cuando se usa la definición de función en comparación con el esquema de API.



```python
def lambda_handler(event, context):
    action_group = event['actionGroup']
    function = event['function']
    parameters = event.get('parameters', [])

    # setting expected response body
    responseBody = {
        "TEXT": {
            "body": "sample response"
        }
    }

    # Logic to process the requested action goes here
    ...

    # Lastly, return the response back to the agent
    action_response = {
        'actionGroup': action_group,
        'function': function,
        'functionResponse': {
            'responseBody': responseBody
        }

    }

    function_response = {
        'response': action_response,
        'messageVersion': event['messageVersion']
    }
    return function_response
```

Fragmento de código de la función de Lambda que usa la definición de función

```
def lambda_handler(event, context):
  action_group = event['actionGroup']
  function = event['function']
  parameters = event.get('parameters', [])
  # setting expected response body
  responseBody = {
     "TEXT": {
        "body": "sample response"
     }
  }
  # Logic to process the requested action goes here
  ...
  # Lastly, return the response back to the agent
  action_response = {
     'actionGroup': action_group,
     'function': function,
```

```
    'functionResponse': {
        'responseBody': responseBody
    }
}
function_response = {
    'response': action_response,
    'messageVersion': event['messageVersion']
}
return function_response
```

```python
def get_named_parameter(event, name):
    return next(item for item in event['parameters'] if item['name'] == name)['value']

def get_named_property(event, name):
    return next(
        item for item in
        event['requestBody']['content']['application/json']['properties']
        if item['name'] == name)['value']

def lambda_handler(event, context):
    # Getting information from event
    action_group = event['actionGroup']
    api_path = event['apiPath']
    http_method = event['httpMethod']

    # getting parameters according to the http method
    if http_method == "get":
        claim_id = get_named_parameter(event, "claim_id")
    elif http_method == "post":
        claim_id = get_named_property(event, "claim_id")

    # setting expected response body
    response_body = {
        'application/json': {
            'body': "sample response"
        }
    }

    # Logic to process the request goes here
    ...

    # Lastly, return the response to the agent

    action_response = {
        'actionGroup': event['actionGroup'],
        'apiPath': event['apiPath'],
        'httpMethod': event['httpMethod'],
        'httpStatusCode': 200,
        'responseBody': response_body
    }

    api_response = {
        'messageVersion': '1.0',
        'response': action_response
    }

    return api_response
```

Fragmento de código de la función de Lambda que usa un esquema de API

```
def get_named_parameter(event, name):
    return next(item for item in event['parameters'] if item['name'] == name)['value']
def get_named_property(event, name):
    return next(
        item for item in
        event['requestBody']['content']['application/json']['properties']
        if item['name'] == name)['value']
def lambda_handler(event, context):
    # Getting information from event
    action_group = event['actionGroup']
    api_path = event['apiPath']
    http_method = event['httpMethod']
    # getting parameters according to the http method
    if http_method == "get":
        claim_id = get_named_parameter(event, "claim_id")
    elif http_method == "post":
        claim_id = get_named_property(event, "claim_id")
    # setting expected response body
    response_body = {
        'application/json': {
            'body': "sample response"
        }
    }
    # Logic to process the request goes here
    ...
    # Lastly, return the response to the agent
    action_response = {
        'actionGroup': event['actionGroup'],
        'apiPath': event['apiPath'],
        'httpMethod': event['httpMethod'],
        'httpStatusCode': 200,
        'responseBody': response_body
```

```
}
api_response = {
    'messageVersion': '1.0',
    'response': action_response
}
return api_response
```

Al crear la función de Lambda, debe proporcionar una política basada en los recursos que permita al agente invocar dicha función. La política se parece al ejemplo siguiente, en el que $LAMBDA_FUNCTION_NAME es el nombre de la función de Lambda del grupo de acciones y $AGENT_ID es el ID del agente que ejecuta la función.

```
{
  "Version": "2012-10-17",
  "Id": "bedrock-resource-based-policy",
  "Statement": [
    {
      "Sid": "agentsInvokeFunction",
      "Effect": "Allow",
      "Principal": {
        "Service": "bedrock.amazonaws.com"
      },
      "Action": "lambda:InvokeFunction",
      "Resource": "arn:aws:lambda:us-east-1:123456789012:function:$LAMBDA_FUNCTION_NAME",
      "Condition": {
        "ArnLike": {
          "AWS:SourceArn": "arn:aws:bedrock:us-east-1:123456789012:agent/$AGENT_ID"
        }
      }
    }
  ]
}
```

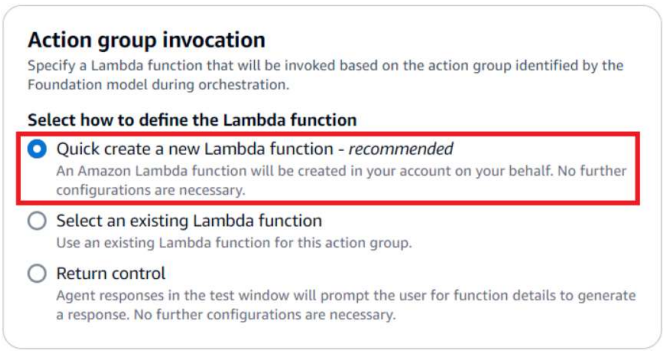
Política de IAM para permitir al agente invocar la función de Lambda

```
{
 "Version": "2012-10-17",
 "Id": "bedrock-resource-based-policy",
 "Statement": [
  {
    "Sid": "agentsInvokeFunction",
    "Effect": "Allow",
    "Principal": {
```

```
      "Service": "bedrock.amazonaws.com"
    },
    "Action": "lambda:InvokeFunction",
    "Resource": "arn:aws:lambda:us-east-1:123456789012:function:$LAMBDA_FUNCTION_NAME",
    "Condition": {
      "ArnLike": {
        "AWS:SourceArn": "arn:aws:bedrock:us-east-1:123456789012:agent/$AGENT_ID"
      }
    }
  }
 ]
}
```

Creación rápida una función de Lambda.

Para facilitar el proceso de creación de la función de Lambda, seleccione Creación rápida una nueva función de Lambda. De este modo, se creará el esqueleto de una función de Lambda con todos los permisos necesarios para el agente y el cuerpo de respuesta necesario a fin de devolver la respuesta.



Sección de invocación del grupo de acciones del agente en el creador de agentes

Importante: La creación rápida una función de Lambda solo creará el esqueleto de la función. Todavía necesita implementar la lógica empresarial para ella, incluida la ejecución de las funciones en su grupo de acciones.

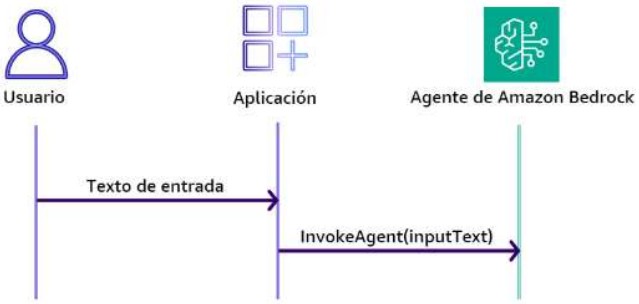
Grupo de acciones del agente con flujo de funciones de Lambda:

- **Paso 1: Opción Devolver control activada.**



El agente se crea con un grupo de acciones adjunto que tiene actionGroupExecutor configurado como función de Lambda.

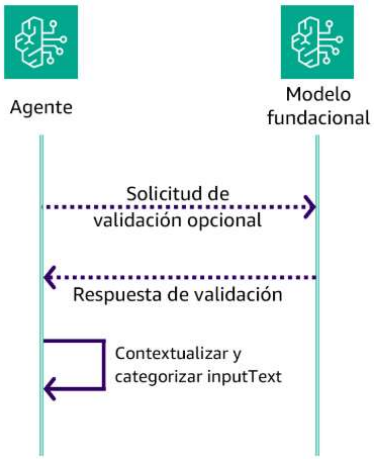
- **Paso 2: La aplicación invoca al agente.**



La aplicación invoca al agente con el texto de entrada del usuario.

En el ejemplo de la reserva de vacaciones, el texto de entrada contendrá el ID de empleado, la fecha de inicio de las vacaciones y la fecha de finalización.

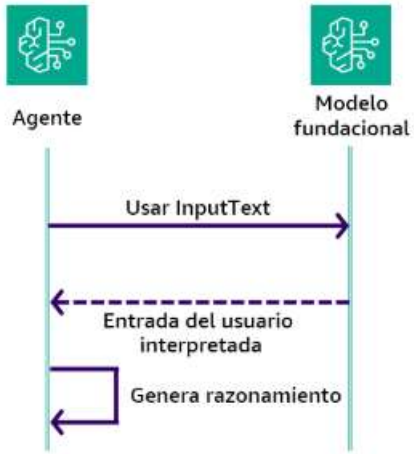
- **Paso 3: Procesamiento previo del agente.**



El agente contextualiza y clasifica las entradas del usuario. Se puede usar para validar el texto de entrada.

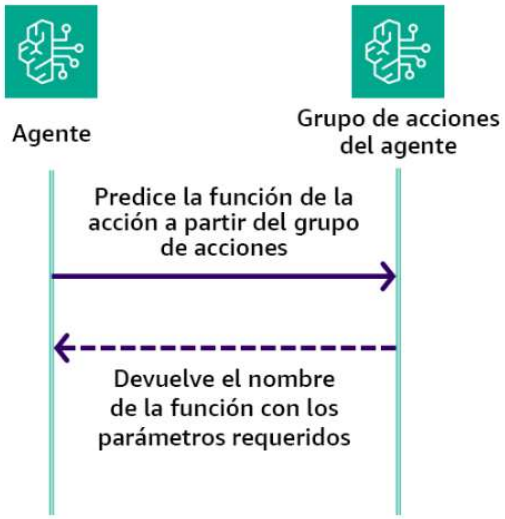
El procesamiento previo es opcional y está desactivado de forma predeterminada. Es importante tener en cuenta que, si bien el procesamiento previo valida y clasifica las peticiones, puede aumentar la latencia del agente debido al procesamiento adicional. Este procesamiento puede incluir una llamada a un FM.

- **Paso 4: El agente genera un razonamiento.**



El agente inicia la fase de orquestación. Interpreta la entrada con un modelo fundacional y genera un razonamiento que establece la lógica del siguiente paso que debe dar.

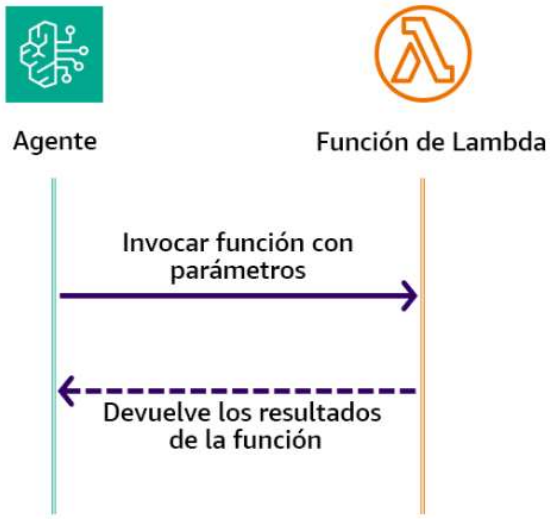
- **Paso 5: El agente elige la acción que se ejecutará.**



El razonamiento del agente determina que debe invocarse una acción del grupo de acciones con la función de Lambda correspondiente utilizando los parámetros requeridos.

En el ejemplo de la reserva de vacaciones, se devolverán al agente el nombre de la función de Lambda ag-function-reserve-vacation-time-xxxx y los nombres de los parámetros de ID de empleado, fecha de inicio de vacaciones y fecha de finalización.

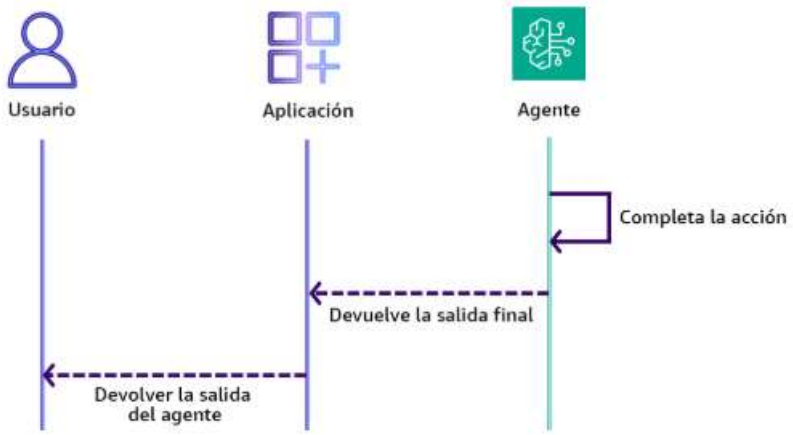
- **Paso 6: El agente invoca la función de Lambda.**



El agente invoca la función de Lambda con parámetros. Dicha función devuelve los resultados al agente.

En el ejemplo de reserva de vacaciones, la función de Lambda ag-function-reserve-vacation-time-xxxx aceptará los parámetros de entrada, ejecutará la lógica empresarial y devolverá el resultado. Este último puede ser una reserva correcta de tiempo de vacaciones o un mensaje de error, como uno que indique que no hay tiempo de vacaciones disponible.

- **Paso 7: El agente completa la acción.**



El agente completa la acción y proporciona la salida final a la aplicación.

La aplicación responde al usuario con la salida del agente.

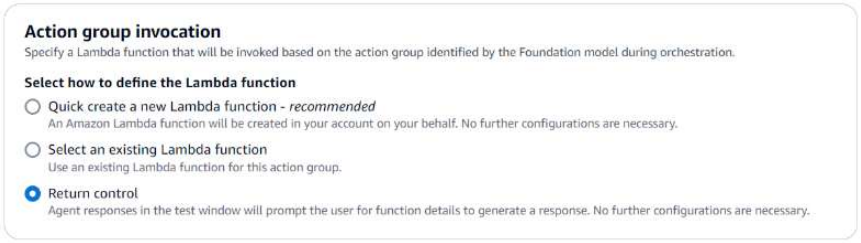
Implementar la opción de devolver control.

La opción de devolver control es un tipo de invocación de grupo de acciones que se utiliza para ejecutar acciones de forma asíncrona. Puede ejecutar rápidamente iteraciones de pruebas en la consola sin invocar una función de Lambda. La opción de devolver control también se usa en cargas de trabajo de producción cuando no se necesita una función de Lambda.

- **Seleccionar la opción de invocación:**

  Al crear su grupo de acciones del agente, elija la opción de devolver control como la opción de invocación para el grupo.

  Una vez que revisó esta pestaña, pase a la siguiente para obtener información sobre ejecutar la lógica empresarial.



- **Ejecutar la lógica empresarial:**

  Al seleccionar esta opción, el agente devuelve el control a la aplicación. Esto finaliza el ciclo de orquestación del agente. La aplicación puede entonces optar por ejecutar funciones locales o llamar a otras API.

- **Devolver resultados:**

  La aplicación puede devolver los resultados al agente dentro del estado de sesión de una nueva invocación del agente. La API de invocación del agente se explicará con más detalle en el siguiente módulo de esta formación.

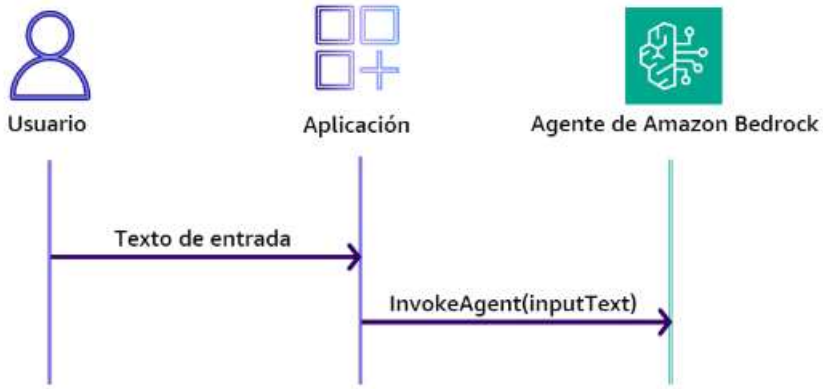
Flujo de Devolver control:

- **Paso 1: Devolver control activada.**



Se crea un agente con un grupo de acciones adjunto que tiene actionGroupExecutor configurado en RETURN_CONTROL.

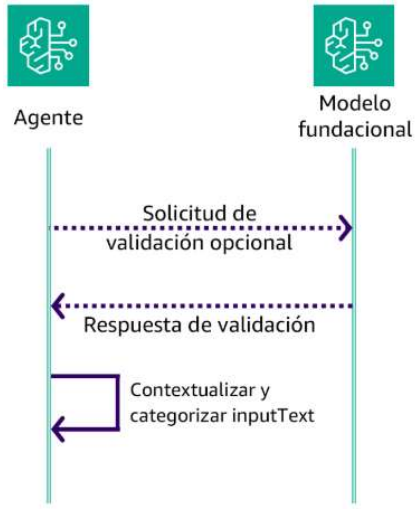
- **Paso 2: La aplicación invoca al agente.**



La aplicación invoca al agente con el texto de entrada del usuario.

En el ejemplo de la reserva de vacaciones, el texto de entrada contendrá el ID de empleado, la fecha de inicio de las vacaciones y la fecha de finalización.

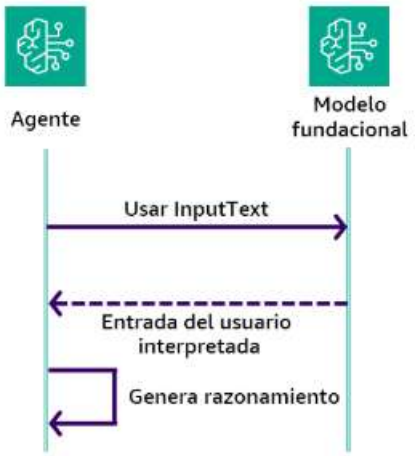
- **Paso 3: Procesamiento previo del agente.**



El agente contextualiza y clasifica las entradas del usuario. Se puede usar para validar el texto de entrada.

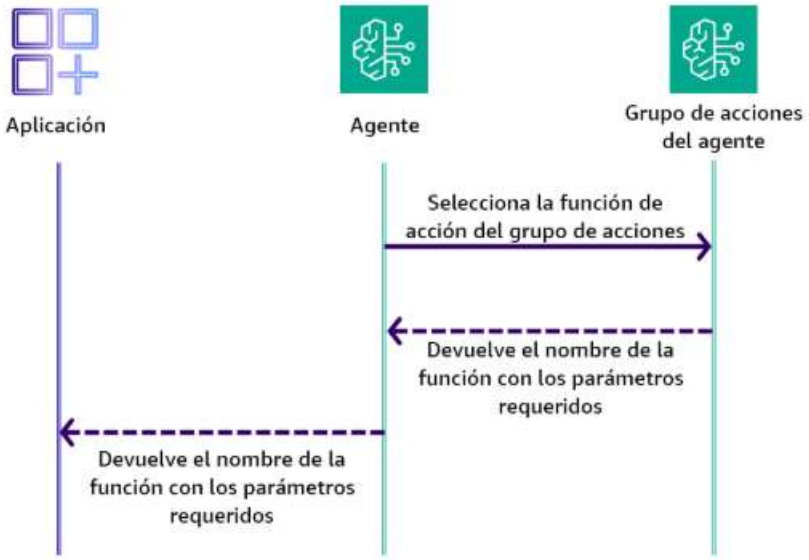
El procesamiento previo es opcional y está desactivado de forma predeterminada. Es importante tener en cuenta que, si bien el procesamiento previo valida y clasifica las peticiones, puede aumentar la latencia del agente debido al procesamiento adicional. Este procesamiento puede incluir una llamada a un FM.

- **Paso 4: El agente genera un razonamiento.**



El agente inicia la fase de orquestación. Interpreta la entrada con un modelo fundacional y genera un razonamiento que establece la lógica del siguiente paso que debe dar.

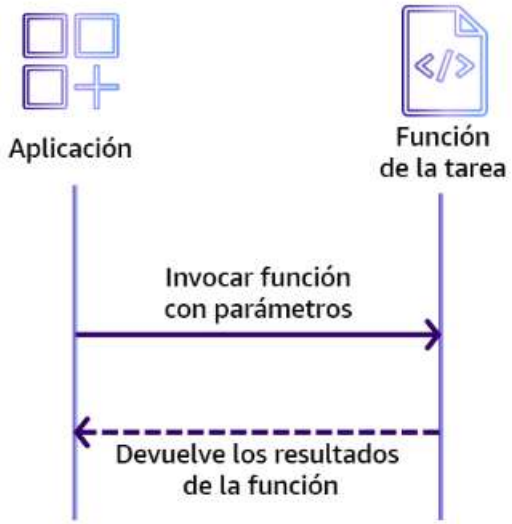
- **Paso 5: El agente elige la acción que se ejecutará.**



La orquestación del agente determina que debe invocarse una acción del grupo de acciones con **Devolver control**.

El agente devuelve la respuesta con el parámetro returnControl completado, junto con la acción que debe invocarse y sus parámetros. En el ejemplo de la reserva de tiempo de vacaciones, la respuesta a la aplicación contendrá el nombre de la función local para reservar vacaciones y los nombres de los parámetros de ID de empleado, fecha de inicio de vacaciones y fecha de finalización.

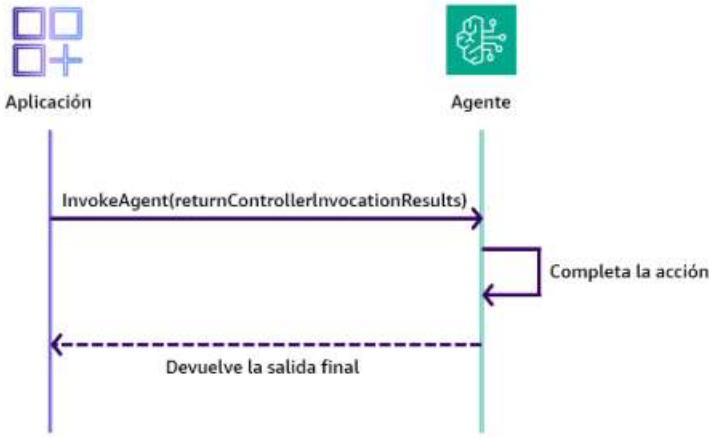
- **Paso 6: La aplicación invoca la función local.**



La aplicación invoca la función correcta con sus parámetros.

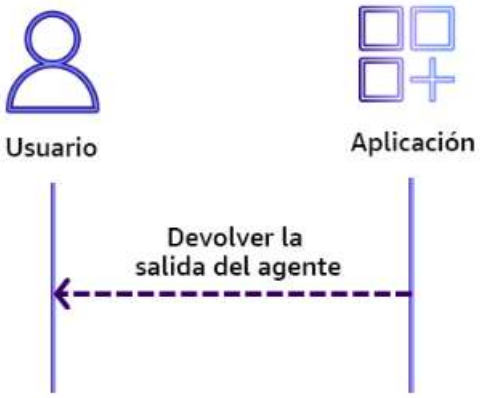
En el ejemplo de la reserva de vacaciones, la función puede devolver una reserva correcta o un error de lógica empresarial, como uno que indique que no hay tiempo de vacaciones disponible.

- **Paso 7: La aplicación devuelve los resultados al agente.**



La aplicación invoca al agente con returnControlInvocationResults establecido en sessionState. El agente completa la acción y proporciona la salida a la aplicación.

- **Paso 8: La aplicación responde al usuario.**



La aplicación responde al usuario con la salida del agente.

Resumen.

En resumen, los grupos de acciones del agente definen las acciones o tareas específicas que un agente puede realizar para ayudar a los usuarios. Cree grupos de acciones especificando las API o el código que debe invocarse mediante una función de Lambda. Estos grupos de acciones procesan la solicitud entrante y generan la respuesta adecuada para el usuario.

Los grupos de acciones pueden definirse mediante el uso de definiciones de funciones, que son objetos JSON que especifican las funciones y definen sus parámetros. También pueden definirse con esquemas OpenAPI, que son esquemas independientes del lenguaje que sirven como planos para la API o función. El esquema especifica la funcionalidad de la función y los formatos de los parámetros.

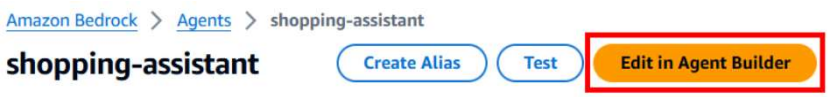
Los agentes para Amazon Bedrock proporcionan la opción de elegir entre dos tipos de métodos de invocación al momento de trabajar con grupos de acciones:

- **Implementar las funciones de Lambda:** este método requiere definir la lógica para ejecutar acciones según la ruta de la API o el nombre de la función recuperada de la solicitud del agente. Se debe definir el objeto de respuesta que se enviará de vuelta al agente y considerar cómo gestionar los errores dentro de la lógica de la función.
- **Usar la invocación con Devolver control:** este método devuelve el control a la aplicación para que ejecute de forma asíncrona las acciones en funciones locales. También permite ejecutar iteraciones de prueba en la consola o devolver resultados al agente sin necesidad de invocar una función de Lambda.

INTEGRACIONES DE AGENTES PARA AMAZON BEDROCK

En la sección anterior, se explicó cómo vincular un grupo de acciones a un agente para la ejecución de tareas. En esta sección, se abordarán dos de las integraciones de agentes para Amazon Bedrock. Puede utilizar la integración con las **bases de conocimientos de Amazon Bedrock** para realizar búsquedas semánticas en una base de datos vectorial. También puede usar la integración con las **barreras de protección de Amazon Bedrock**, que permiten incorporar barreras de seguridad personalizadas y políticas de IA responsable en sus aplicaciones de IA generativa.

Para agregar una base de conocimientos o una barrera de protección a un agente previamente creado, seleccione el agente correspondiente en **Amazon Bedrock** y, luego, haga clic en **Editar en el creador de agentes**.



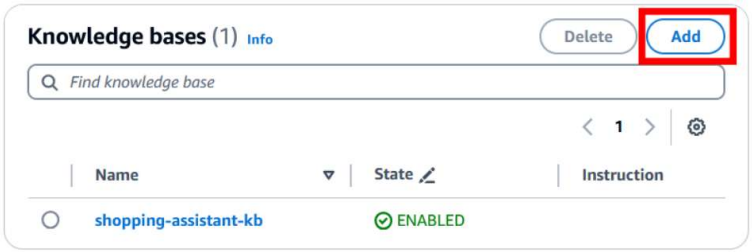
Agregue una base de conocimientos o una barrera de protección a un agente existente

Integrar las bases de conocimientos de Amazon Bedrock.

La combinación de agentes para Amazon Bedrock y las bases de conocimientos de Amazon Bedrock permite la conexión con orígenes de datos privados que proporcionan información contextual al agente.

Durante el proceso de creación del agente, es posible asociar una base de conocimientos ya existente. De este modo, el agente puede recuperar automáticamente documentos relevantes desde dicha base de conocimientos y utilizarlos como contexto para formular respuestas a las consultas de los usuarios. Según la descripción proporcionada de la base de conocimientos, el agente puede determinar de manera autónoma cuándo realizar una búsqueda contextual y cómo aplicar la información obtenida.

Dentro del creador de agentes, se puede utilizar la sección **Bases de conocimientos** para agregar una base de conocimientos ya existente al agente. Para hacerlo, elija **Agregar**.



Agregue una base de conocimientos existente a un agente.

Durante la invocación en tiempo de ejecución, se puede utilizar la clave knowledgeBaseConfigurations del atributo InvokeAgent sessionState para personalizar aún más la respuesta del agente mediante una configuración de filtrado de metadatos. Esto modifica el tipo de búsqueda entre semántica e híbrida, y sobrescribe el número de documentos recuperados de la base de conocimientos.

La integración de bases de conocimientos a los agentes ofrece a su aplicación los siguientes beneficios:

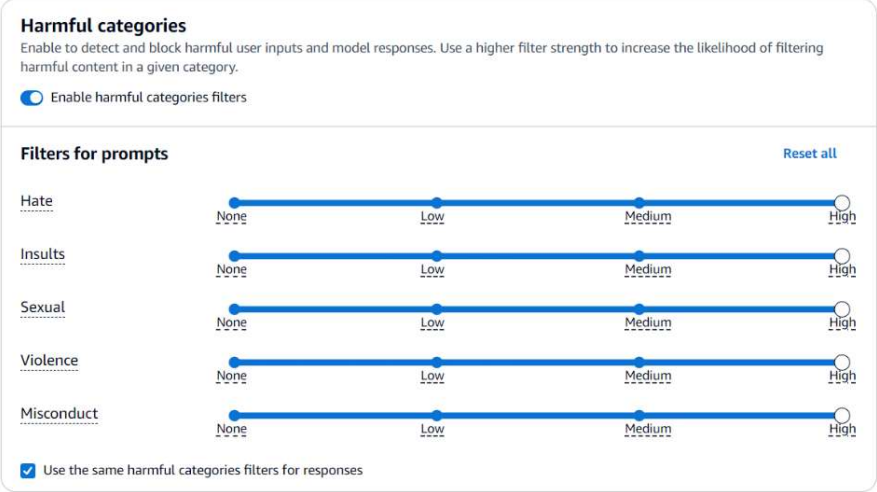
- **Mayor exactitud de respuesta:** gracias a la consulta de datos contenidos en las bases de conocimientos, los agentes pueden asegurarse de que sus respuestas se basen en información correcta y actualizada, por lo que evitan proporcionar datos obsoletos o incorrectos.
- **Mejor comprensión del contexto:** las bases de conocimientos ofrecen detalles esenciales sobre un área temática, productos, servicios o procesos. Esto permite que los agentes comprendan mejor las preguntas de los usuarios y proporcionen respuestas más pertinentes.

- **Escalabilidad y coherencia:** dado que cuenta con una base de conocimientos, es posible mantener y actualizar la información. Esto garantiza que los agentes tengan acceso a los datos más recientes, lo que promueve la coherencia en todas las interacciones.
- **Desarrollo simplificado:** puede utilizar la base de conocimientos como una única fuente confiable de información, lo cual simplifica el desarrollo y el mantenimiento de las aplicaciones.

Puede conectar una base de conocimientos a su agente mediante la consola o de forma programática utilizando la API con cualquiera de los SDK disponibles. También puede usar plantillas de CloudFormation, CDK de AWS o Terraform.

Integración de barreras de protección de Amazon Bedrock:

Con las barreras de protección de Amazon Bedrock, puede implementar salvaguardas y políticas de IA responsable en sus aplicaciones de IA conversacional. Esto garantiza una experiencia del usuario coherente y segura, a la vez que se adhiere a las políticas y los estándares de su organización. Puede crear barreras de protección para Amazon Bedrock y utilizarlas junto con las llamadas a las API **InvokeModel** e **InvokeAgent**.



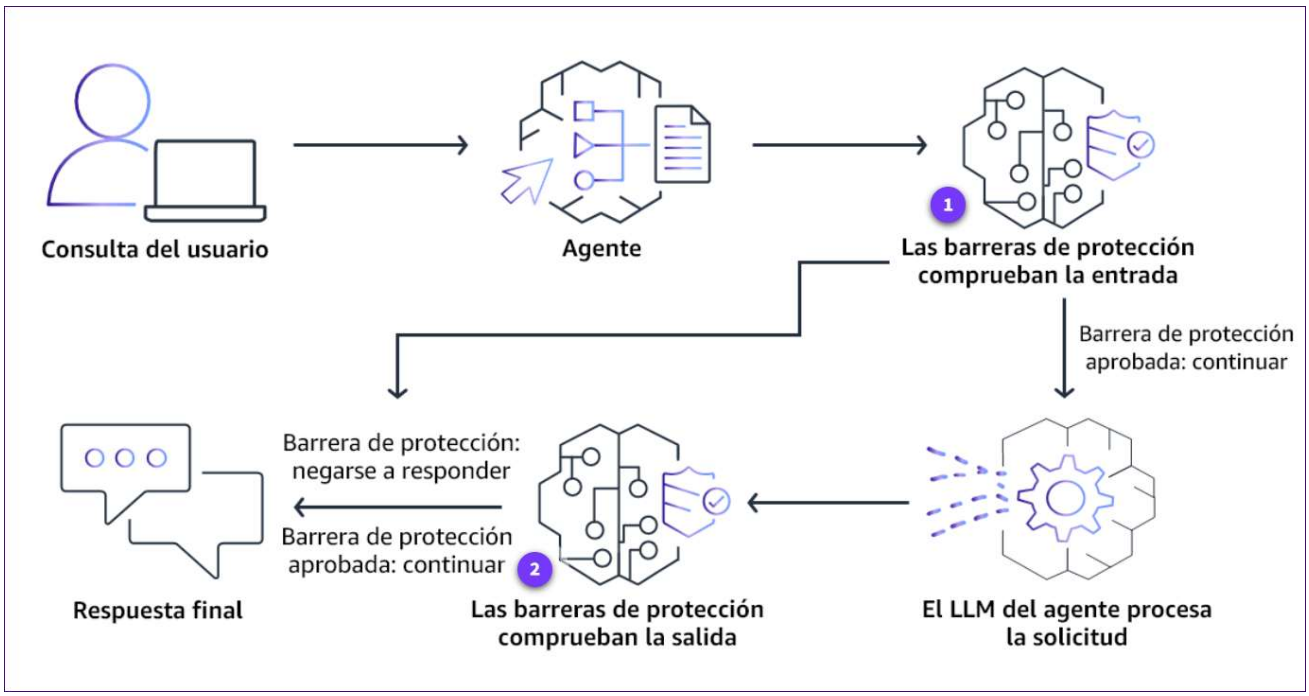
Configuración de controles deslizantes en las barreras de protección para Amazon Bedrock.

Para conectar una barrera de protección a su aplicación, primero debe crear la barrera de protección y, a continuación, adjuntarla a su agente. Puede hacerlo mediante la consola o de forma programática utilizando la API con cualquiera de los SDK disponibles. También puede usar plantillas de CloudFormation, CDK de AWS o Terraform.



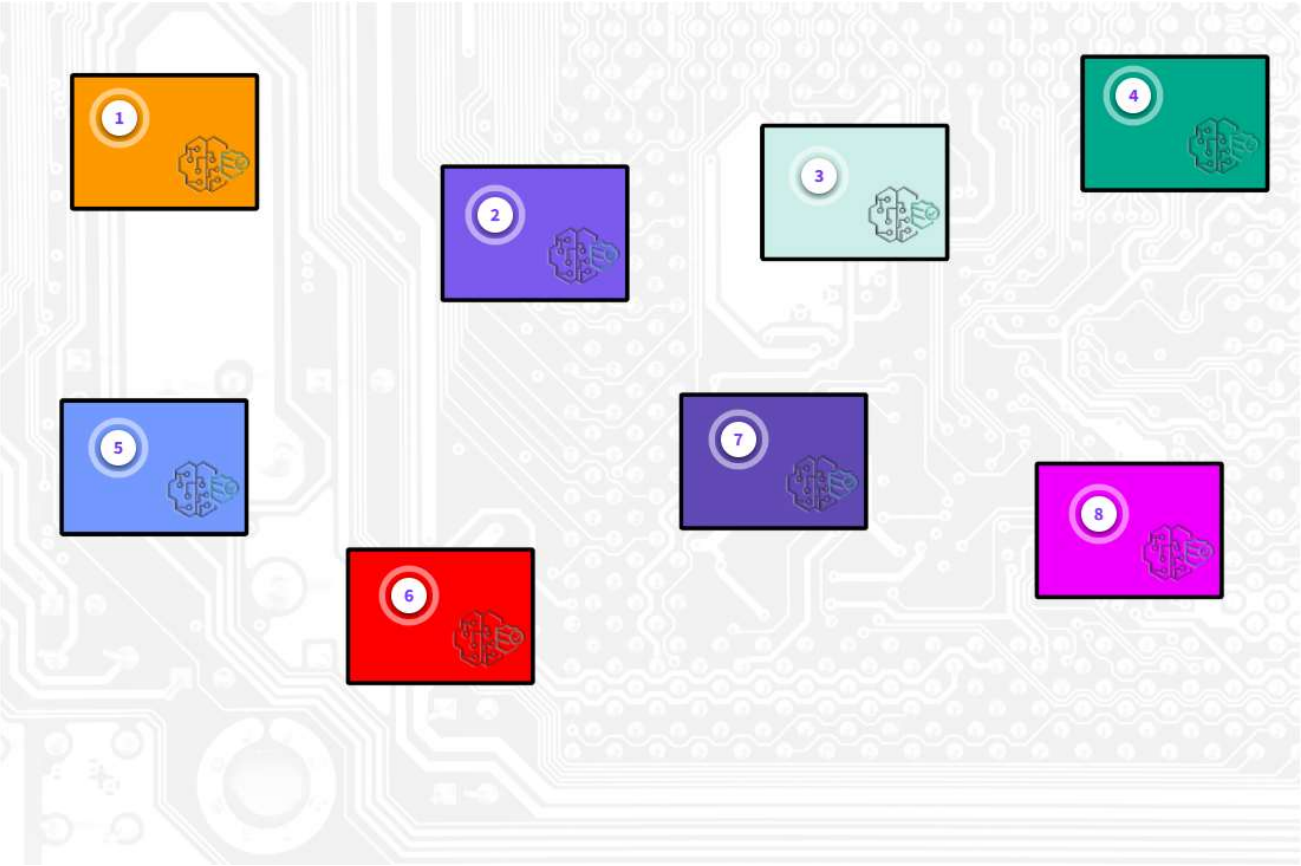
Un agente con una barrera de protección adjunta en Amazon Bedrock.

Una vez que se conecte una barrera de protección a su agente, esta se utilizará dos veces para cada consulta de usuario. En primer lugar, la barrera de protección procesa la entrada del usuario para determinar si es seguro ejecutarla. Si es así, procesará la solicitud del usuario y, a continuación, la barrera de protección volverá a comprobar la salida del agente. La respuesta final para el usuario es la respuesta del agente o los mensajes predefinidos de la barrera de protección para los casos en que esta interfiere y bloquea la solicitud.



1. **Revisión de entrada de la barrera de protección:**
   La barrera de protección procesa la entrada del usuario para determinar si es seguro ejecutarla. Si pasa las barreras de protección, se procede con la solicitud. En caso contrario, el agente no responderá.
2. **Revisión de salida de la barrera de protección:**
   La barrera de protección revisa nuevamente la salida del agente para verificar que sea segura antes de entregarla al usuario. Si pasa, se compartirá el resultado. Si no pasa, el agente no responderá.

Beneficios de las integraciones de barreras de protección:



1. **Implementación responsable de la IA:**

   Con las barreras de protección, puede alinear su sistema de IA conversacional con las políticas y directrices de IA responsable de su organización. Esto promueve la confianza y la responsabilidad.

2. **Moderación de contenido:**

   Debido a que se definen temas prohibidos y se ajustan los filtros de contenido, puede prevenir que contenido no deseado o perjudicial aparezca en las entradas de los usuarios o en las respuestas del agente.

3. **Experiencia del usuario coherente:**

   La aplicación uniforme de barreras de protección en múltiples agentes y modelos fundacionales garantiza una experiencia estandarizada y segura para sus usuarios.

4. **Políticas a medida:**

   Puede aplicar políticas apropiadas en función de las necesidades específicas de su aplicación al crear múltiples barreras de protección adaptadas a diferentes casos de uso.

5. **Control de aplicaciones:**

   Las barreras de protección le permiten controlar el contenido y las interacciones dentro de su sistema de IA conversacional. De este modo, puede verificar que el sistema opere dentro de los límites definidos.

6. **Alineación ética:**
   La implementación de barreras de protección ayuda a mantener estándares éticos y principios de IA responsable, lo que fomenta la confianza y la responsabilidad en sus aplicaciones de IA.
7. **Protección de la marca:**
   Gracias a la prevención de contenido inapropiado o perjudicial, las barreras de protección ayudan a proteger la reputación de su marca y a mantener una experiencia del usuario positiva.
8. **Conformidad normativa:**
   Según el sector y el caso de uso, las barreras de protección pueden ayudar a cumplir los requisitos normativos relacionados con la moderación del contenido y la privacidad de los datos.

Una vez que se haya integrado una barrera de protección en su agente, puede inspeccionar su uso revisando el trazado del agente en la interfaz de usuario de la consola o en la respuesta de su solicitud a la API.

PROBAR E INVOCAR AGENTES

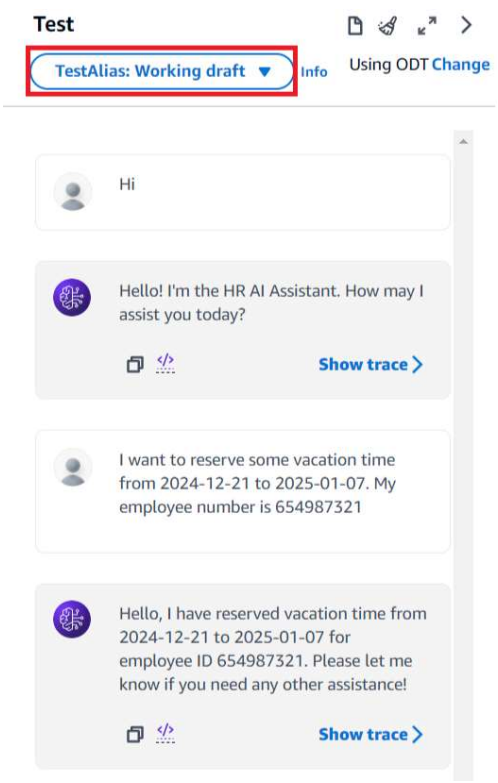
IMPLEMENTAR E INVOCAR AGENTES

Ya aprendió a crear agentes y adjuntarlos a grupos de acciones, bases de conocimientos o barreras de protección. Estas operaciones se realizan durante la fase de creación del agente y son compatibles con la API de agentes para Amazon Bedrock.

Implementar un agente.

1. **Paso 1: Pruebe su agente.**
   Una vez que haya creado y preparado su agente, tendrá disponible una versión preliminar operativa (DRAFT) y un alias de prueba (TSTALIASID).

   Cuando realice cambios en su agente, estos se aplicarán a la versión preliminar operativa. Pruebe el agente invocando dicha versión preliminar operativa hasta que esté satisfecho con su comportamiento.



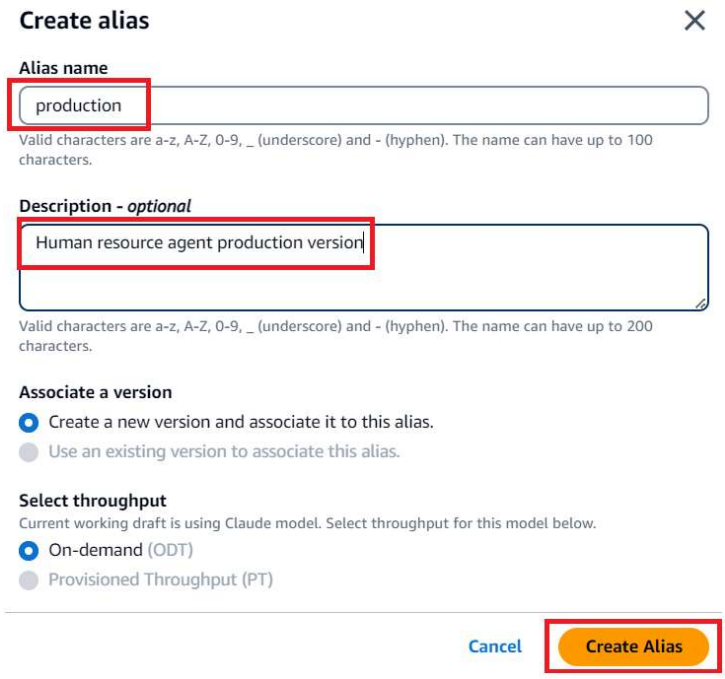
Chat de prueba de un agente con la versión preliminar operativa.

Usuario: Hola **Agente:** ¡Hola! Soy el asistente de IA de RR. HH. ¿Cómo puedo ayudarlo hoy? **Usuario:** Quiero reservar unas vacaciones del 21/12/2024 al 7/1/2025. Mi número de empleado es 654987321. **Agente:** Hola, reservé vacaciones del 21/12/2024 al 7/1/2025 para el ID de empleado 654987321. Avíseme si necesita algo más.

2. **Paso 2: Cree un alias de agente.**

   Una vez que esté todo listo para integrar su agente en su aplicación, debe implementarlo creando un nuevo alias de agente y asociándolo con una versión del agente.

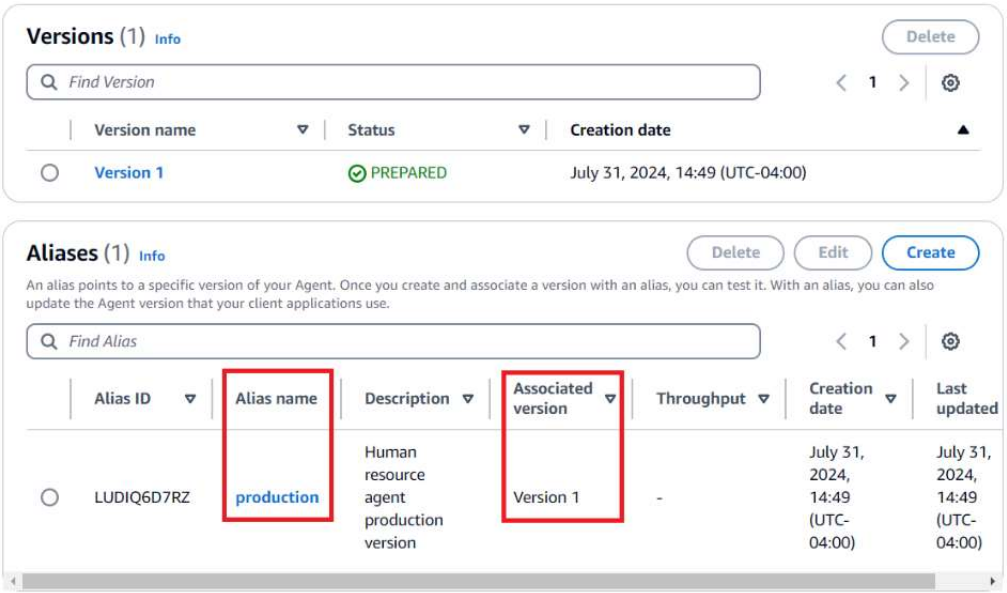
   Para implementar su agente, es obligatorio crear un alias.



Cree un alias de agente.

3. **Paso 3: Confirme que el alias esté vinculado a la versión a la versión correcta del agente.**

   Durante la creación del alias, Amazon Bedrock crea automáticamente una versión de su agente. El alias apunta a esta versión recién creada. Como alternativa, puede apuntar el alias a una versión de su agente creada anteriormente. A continuación, configure la aplicación para realizar llamadas a la API con ese alias.



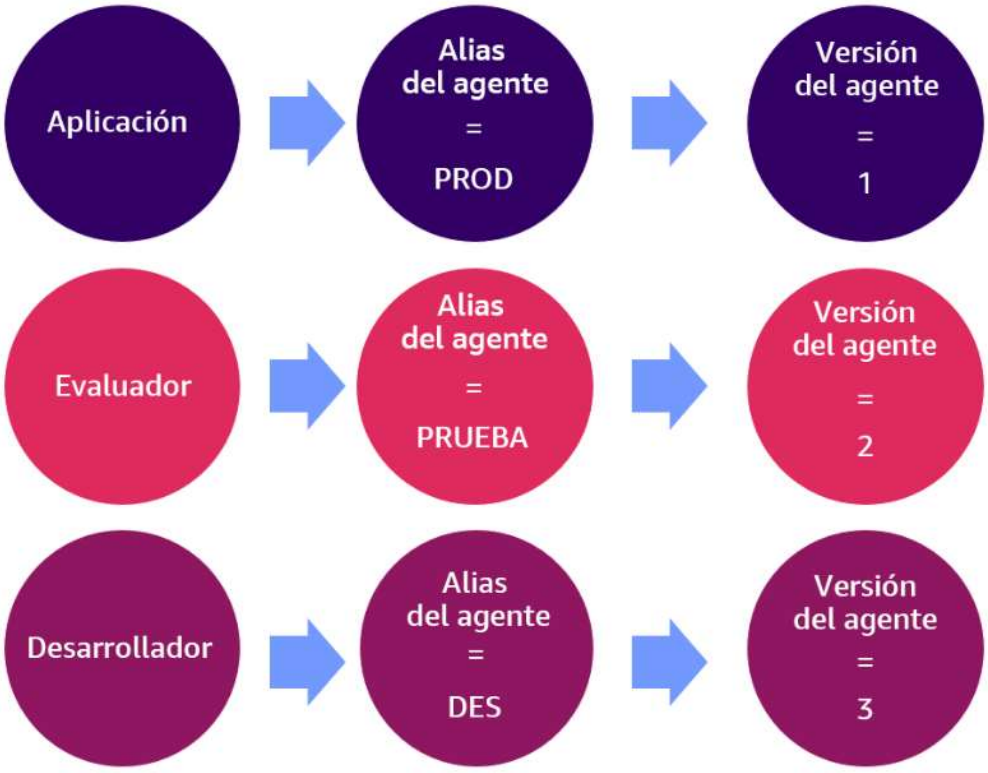
Confirme que el alias de agente se creó con la versión correcta.

Versión y alias del agente.

Una versión del agente es una instantánea que conserva los recursos asociados a su agente tal como existen en el momento en que se crean. Las versiones son inmutables en el sentido de que mantienen los enlaces con la función de Lambda, las bases de conocimientos y las barreras de protección. Para actualizar un agente y vincularlo con diferentes recursos, debe crear una nueva versión del agente con los enlaces actualizados. Su aplicación debe realizar llamadas al alias que apunte a la nueva versión del agente.

Gracias a los alias, puede cambiar de manera eficiente entre diferentes versiones de su agente sin necesidad de que la aplicación controle directamente la versión. Por ejemplo, puede cambiar un alias para que apunte a una versión anterior del agente si necesita revertir cambios rápidamente.

Este comportamiento le permite crear aplicaciones que sigan las directrices del desarrollo de software. Puede, por ejemplo, definir alias del agente para los entornos de DESARROLLO, PRUEBAS y PRODUCCIÓN, y hacer que estos alias apunten a las versiones correspondientes del agente en un momento determinado.



Alias de agente que apuntan a diferentes versiones del agente.

Puede crear nuevos alias de agente mediante la consola, los AWS SDK, CloudFormation, AWS CDK y Terraform.

En el siguiente ejemplo, aprenderá sobre el ID del alias de agente de la versión preliminar operativa: TSTALIASID. Las instrucciones proporcionadas aquí serán las mismas para cualquier otro ID de alias. Solo debe cambiar el parámetro agentAliasId en su llamada a InvokeAgent.

Invocar a una agente.

Una vez que se creó y preparó el agente (y opcionalmente se creó un alias), el siguiente paso es invocar al agente durante el tiempo de ejecución. Puede hacerlo con la consola o con uno de los SDK de API de agentes para Amazon Bedrock.

La infraestructura como código se usa para crear infraestructura, pero no se utiliza para controlar la ejecución de dicha infraestructura una vez implementada. Por lo tanto, los métodos de tiempo de ejecución del agente no son compatibles con CloudFormation, AWS CDK o Terraform. En esta sección, se explicará cómo invocar agentes utilizando el cliente bedrock-agent-runtime del SDK boto3.

Proceso de tiempo de ejecución:

El cliente del agente en tiempo de ejecución proporciona funcionalidades para ejecutar aplicaciones de IA generativa durante el tiempo de ejecución. Se utiliza con el fin de invocar agentes para Amazon Bedrock, bases de conocimientos de Amazon Bedrock y flujos de peticiones de Amazon Bedrock. La API incluye métodos como Retrieve, RetrieveAndGenerate e InvokeFlow. En esta sección, nos centraremos en la API de **InvokeAgent**.

El método InvokeAgent envía una petición al agente para que procese y responda. Existen diversas capacidades avanzadas que puede utilizar para ejercer un mayor control durante el tiempo de ejecución.

- Continuar con una sesión anterior.
- Proporcionar un ID de memoria para permitir que el agente retenga información de sesiones de conversación anteriores.
- Proporcionar archivos de datos para que el agente los procese y comprenda mediante sus capacidades de interpretación de código.
- Devolver control al usuario.
- Habilitar funciones de trazado para obtener información sobre la cadena de pensamientos del agente.

Invocar el flujo del proceso del agente:

Si desea invocar a un agente mediante AWS SDK para Python (boto3), siga estos pasos.

- **Paso 1: Obtener las actualizaciones más recientes para boto3, botocore y awscli.**

  Instale (o actualice) boto3, botocore y awscli para obtener la versión más reciente. El servicio de agentes para Amazon Bedrock se mejora continuamente y se añaden nuevas capacidades a la API y sus SDK. Para utilizar la última versión, se recomienda actualizar primero el SDK. Desde una línea de comandos, ejecute los siguientes comandos pip:

  >>pip install --upgrade -q boto3

  >>pip install --upgrade -q botocore

  >>pip install --upgrade -q awscli

- **Paso 2: Crear un cliente bedrock – agent – runtime.**

  Cree un cliente para bedrock-agent-runtime a fin de invocar a los agentes para Amazon Bedrock y recuperar sus respuestas. Para hacerlo, debe ejecutar el siguiente código desde un archivo Python o un cuaderno de Jupyter.

  ```
  import boto3
  bedrock_agent_runtime_client = boto3.client('bedrock-agent-runtime')
  ```

- **Paso 3: Invocar al agente con una petición.**

  Invoque a su agente con una petición. Para ello, debe proporcionar su ID de agente y su ID del alias de agente a la llamada a la API de InvokeAgent. Puede usar la llamada a la API de InvokeAgent con el método invoke_agent del cliente bedrock-agent-runtime de boto3.

  ```
  agentResponse = bedrock_agent_runtime_client.invoke_agent(
  inputText="How much vacation does employee_id 1 have available?",
  agentId="<YOUR_AGENT_ID>",
  agentAliasId="TSTALIASID",
  sessionId="<AN_ID_FOR_THE_CONVERSATION_SESSION>"
  )
  ```

Para llamar a la API de InvokeAgent, debe proporcionar la siguiente información:

- **agentId**: es un identificador único de su agente.
- **agentAliasId**: es el ID del alias de agente que quiere invocar.
- **sessionId**: es un identificador único para la sesión que desea usar. Puede crear un nuevo ID de sesión para iniciar una conversación nueva o usar uno existente para continuar una conversación existente.
-

Cuando le pregunte algo nuevo al agente, debe proporcionar el campo inputText, que contiene el texto que se enviará al agente. Este campo es opcional cuando se incluyen los resultados de la invocación de una función mediante la opción de devolver control. Los resultados se incluyen en el campo returnControlInvocationResults del grupo de campos sessionState.

Para añadir más funcionalidades a su agente, puede usar más parámetros al invocar a un agente.

Funcionalidad adicional del agente.

Funcionalidad de trazado:

Cuando se desarrolla una nueva aplicación de IA generativa utilizando agentes para Amazon Bedrock, la etapa de pruebas es esencial para garantizar que el agente produzca el comportamiento esperado. Para depurar el comportamiento del agente, utilice la funcionalidad de trazado del agente, la cual permite visualizar y explorar los pasos que ejecuta un agente durante una invocación.

El trazado muestra el proceso de razonamiento paso a paso del agente derivado de la petición de la cadena de pensamientos y su marco de trabajo ReAct. Este trazado se presenta como un objeto JSON que está disponible tanto en la consola como a través de la API.

Cada paso en el trazado corresponde a uno de los siguientes tipos:

- **Trazado del paso de procesamiento previo:**
  Un trazado del paso de procesamiento previo contiene las entradas y salidas del paso de **procesamiento previo**. Determina si la entrada de un usuario es válida o no.

- **Trazado del paso de orquestación:**
  Un trazado del paso de orquestación contiene las entradas y salidas del paso de **orquestación**. En este paso, el agente puede interpretar la entrada del usuario, invocar a los grupos de acciones necesarios, generar e interpretar código o consultar las bases de conocimientos disponibles.

  El agente devolverá una salida, continuará con otro paso de orquestación o decidirá solicitar al usuario una entrada de seguimiento para resolver su consulta.

- **Trazado del paso de procesamiento posterior:**
  Un trazado del paso de procesamiento posterior contiene las entradas y salidas del paso de **procesamiento posterior**. En este paso, el agente gestiona la salida final de la orquestación y determina cómo responder al usuario.

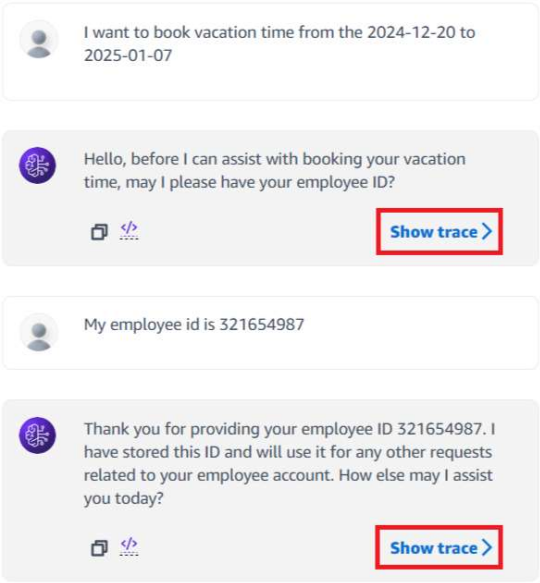
- **Trazado del paso de barrera de protección:**
  Un trazado del paso de barrera de protección contiene las acciones y salidas del paso de barreras de protección para Amazon Bedrock conectado al agente. En este paso, se determina si la entrada de un usuario y la respuesta de un agente son válidas.

- **Trazado de errores:**
  Un trazado de errores de pasos contiene los detalles y motivos por los que un paso falló.

Cada uno de los trazados (excepto el de errores) proporciona el objeto ModelInvocationInput. Este objeto contiene la petición que el agente utilizó en el paso.

Al invocar a un agente, puede elegir pasar el parámetro enableTrace con un valor booleano. Si se establece en True, el trazado se hace visible en la salida del agente.

En la consola de administración de AWS, la capacidad de trazado está habilitada de forma predeterminada. Está disponible con el enlace **Mostrar trazado** para cada paso.



Enlaces Mostrar trazado de los pasos del agente.

En la siguiente imagen, se muestra un ejemplo parcial del trazado del paso de orquestación modelInvocationInput sin employee_id para nuestro agente asistente de RR. HH.

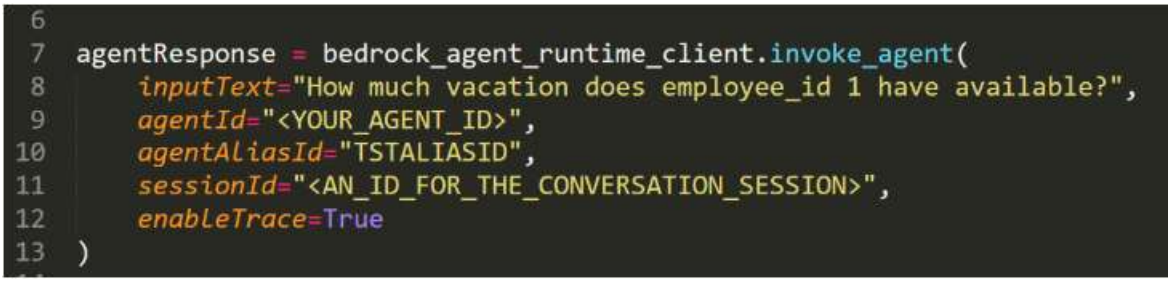
```
...
{
  "agentAliasId": "TSTALIASID",
  "agentId": "FBS6WKNXB0",
  "agentVersion": "DRAFT",
  "sessionId": "054405217165140",
  "trace": {
    "orchestrationTrace": {
      "rationale": {
        "text": "The user wants to book vacation time from 2025-01-16 to 2025-01-31.\n\nTo book vacation
        time, I need to know the employee's ID. The user did not provide their employee ID in the question.
        I should ask for their employee ID before trying to book any vacation time.",
        "traceId": "e3c82ace-6d82-4a92-991a-cb17632db905-0"
      }
    }
  }
},
...
```

Fragmento de código de trazado del paso de orquestación del agente.

...
{
"agentAliasId": "TSTALIASID",
"agentId": "FBS6WKNXB0",
"agentVersion": "DRAFT",
"sessionId": "054405217165140",
"trace": {

```
    "orchestrationTrace": {
      "rationale": {
        "text": "The user wants to book vacation time from 2025-01-16 to
2025- 01-31.\n\n To book vacation time, I need to know the employee's ID. The user did not provide their
employee ID in the question. I should ask for their employee ID before trying to book any vacation time.",
        "traceId": "e3c82ace-6d82-4a92-991a-cb17632db905-0"
      }
    }
  }
},
...
```


Para recuperar el trazado mediante la API, debe establecer el parámetro enableTrace en True.



```
6
7  agentResponse = bedrock_agent_runtime_client.invoke_agent(
8      inputText="How much vacation does employee_id 1 have available?",
9      agentId="<YOUR_AGENT_ID>",
10     agentAliasId="TSTALIASID",
11     sessionId="<AN_ID_FOR_THE_CONVERSATION_SESSION>",
12     enableTrace=True
13 )
```

Fragmento de código de la llamada a la API de invoke_agent con enableTrace establecido en True.

```
agentResponse = bedrock_agent_runtime_client.invoke_agent(
  inputText="How much vacation does employee_id 1 have available?",
  agentId="<YOUR_AGENT_ID>",
  agentAliasId="TSTALIASID",
  sessionId="<AN_ID_FOR_THE_CONVERSATION_SESSION>",
  enableTrace=True
)
```

Funcionalidad de la sesión.

Los agentes para Amazon Bedrock pueden realizar el seguimiento de una conversación durante una sesión. Para un sessionId determinado, el agente tiene la información sobre la conversación anterior hasta el valor especificado en idleSessionTTLInSeconds con un máximo de una hora.

Después de que una sesión expira, Amazon Bedrock elimina cualquier dato proporcionado antes del tiempo de expiración. Un sessionId puede reutilizarse sin acceder al contexto previo. Al utilizar la Consola de Administración de AWS, su conversación se mantendrá hasta que finalice explícitamente la sesión o hasta que esta expire. Si utiliza la API, es necesario invocar al agente con el mismo ID de sesión para conservar el contexto de la conversación.

El siguiente fragmento de código muestra cómo mantener el contexto conversacional utilizando el AWS SDK de Python para la API. El parámetro sessionId se establece en 123 para ambos métodos de invoke_agent.

```
3
4  # Invoke the agent API the first time
5  agentResponse = bedrock_agent_runtime_client.invoke_agent(
6      inputText="How much vacation does employee_id 1 have available?",
7      agentId="<YOUR_AGENT_ID>",
8      agentAliasId="TSTALIASID",
9      sessionId="123",
10     enableTrace=True
11 )
12
13 # agent answer happens here ...
14 print(agentResponse)
15
16 # Invoke the agent API the second time with the same session id
17 agentResponse2 = bedrock_agent_runtime_client.invoke_agent(
18     inputText="great! book one day off for July 5th, 2025.",
19     agentId="<YOUR_AGENT_ID>",
20     agentAliasId="TSTALIASID",
21     sessionId="123",
22     enableTrace=True
23 )
24
25 print(agentResponse2)
```

Fragmento de código de múltiples llamadas a la API de invoke_agent con sessionId establecido en 123.

```
# Invoke the agent API the first time
agentResponse = bedrock_agent_runtime_client.invoke_agent(
  inputText="How much vacation does employee_id 1 have available?",
  agentId="<YOUR_AGENT_ID>",
  agentAliasId="TSTALIASID",
  sessionId="123",
  enableTrace=True
)
# agent answer happens here ...
print(agentResponse)
# Invoke the agent API the second time with the same session id
agentResponse2 = bedrock_agent_runtime_client.invoke_agent(
  inputText="great! book one day off for July 5th, 2025.",
  agentId="<YOUR_AGENT_ID>",
  agentAliasId="TSTALIASID",
  sessionId="123",
  enableTrace=True
)
print(agentResponse2)
```

Usar sessionAttributes y promptSessionAttributes.

Los usuarios esperan que las aplicaciones conozcan información contextual temporal, como la fecha actual. Sin esta información contextual, el agente no cuenta con conocimiento adicional sobre el usuario que está interactuando con la aplicación. El agente tampoco conoce información como la fecha actual.

Para resolver estos desafíos y proporcionar a su agente los conocimientos de contexto necesarios, puede utilizar las claves sessionAttributes y promptSessionAttributes de su objeto de estado de sesión. Con ellas, puede pasar la información a su agente sin avisar al usuario.

Actualmente, SessionAttributes y PromptSessionAttributes solo están disponibles a través de la API.

- **Atributos de sesión:**

  Cuando se usa la opción sessionAttributes, la información no se agrega a la petición. Un sessionAttribute persistirá en todas las solicitudes realizadas con el mismo sessionId, siempre que no se supere el límite de tiempo de la sesión (idleSessionTTLinSeconds).

- **Atributos de sesión de peticiones:**

  Con promptSessionAttributes, la información se agrega a la petición y se pone a disposición del FM subyacente que utiliza el agente. Cualquier información que se pase con el promptSessionAttribute solo se conservará durante un turno.

  También puede utilizar la variable de marcador de posición $prompt_session_attributes$ al editar la plantilla de peticiones básicas de orquestación para determinar cómo se utiliza dicha información en la petición.
  Con esta técnica, puede pasar información como el token de inicio de sesión del usuario para validar si un determinado usuario puede ejecutar una determinada tarea a través de sessionAttributes. Puede usar esta información en la función de Lambda de sus grupos de acciones.

  También puede compartir la fecha y la hora actuales con el agente a través de promptSessionAttributes para que el usuario pueda realizar consultas sobre fechas relativas, como ayer o mañana.

En el siguiente fragmento de código, se muestra cómo pasar el ID de empleado y la fecha actual a nuestro agente de RR. HH. con el fin de mejorar la experiencia de reserva de vacaciones:

```python
def simple_agent_invoke(
        input_text, agent_id, agent_alias_id, session_id=None,
        enable_trace=False, end_session=False, session_state=dict()
):
    agentResponse = bedrock_agent_runtime_client.invoke_agent(
        inputText=input_text,
        agentId=agent_id,
        agentAliasId=agent_alias_id,
        sessionId=session_id,
        enableTrace=enable_trace,
        endSession= end_session,
        sessionState=session_state
    )

    event_stream = agentResponse['completion']
    try:
        for event in event_stream:
            if 'chunk' in event:
                data = event['chunk']['bytes']
                return data.decode('utf8')
            elif 'trace' in event:
                logger.info(json.dumps(event['trace'], indent=2))
            else:
                raise Exception("unexpected event.", event)
    except Exception as e:
        raise Exception("unexpected event.", e)

from datetime import datetime
today = datetime.today().strftime('%Y-%m-%d')
print(today)

agent_id = "<your_agent_id>"
agent_alias_id = "TSTALIASID"
session_id:str = str(uuid.uuid1())
session_state = {
    "sessionAttributes": {
        "employee_id": "1"
    },
    "promptSessionAttributes": {
        "today": today
    }
}
agent_response_1 = simple_agent_invoke(
    "how many holidays do I have?", agent_id, agent_alias_id,
    session_id, enable_trace=False, session_state=session_state
)
print(agent_response_1)

agent_response_2 = simple_agent_invoke(
    "great! book 2 days from next monday on.", agent_id, agent_alias_id,
    session_id, enable_trace=False, session_state=session_state
)
print(agent_response_2)
```

Fragmento de código para llamar a la invocación a la API del agente con atributos de estado de sesión.

```
def simple_agent_invoke(
      input_text, agent_id, agent_alias_id, session_id=None,
      enable_trace=False, end_session=False, session_state=dict()
):
  agentResponse = bedrock_agent_runtime_client.invoke_agent(
```

```
        inputText=input_text,
        agentId=agent_id,
        agentAliasId=agent_alias_id,
        sessionId=session_id,
        enableTrace=enable_trace,
        endSession= end_session,
        sessionState=session_state
    )

    event_stream = agentResponse['completion']
    try:
        for event in event_stream:
            if 'chunk' in event:
                data = event['chunk']['bytes']
                return data.decode('utf8')
            elif 'trace' in event:
                logger.info(json.dumps(event['trace'], indent=2))
            else:
                raise Exception("unexpected event.", event)
    except Exception as e:
        raise Exception("unexpected event.", e)
from datetime import datetime
today = datetime.today().strftime('%Y-%m-%d')
print(today)
agent_id = "<your_agent_id>"
agent_alias_id = "TSTALIASID"
session_id:str = str(uuid.uuid1())
session_state = {
    "sessionAttributes": {
        "employee_id": "1"
    },
    "promptSessionAttributes": {
        "today": today
```

```
  }
}
agent_response_1 = simple_agent_invoke(
  "how many holidays do I have?", agent_id, agent_alias_id,
  session_id, enable_trace=False, session_state=session_state
)
print(agent_response_1)
agent_response_2 = simple_agent_invoke(
  "great! book 2 days from next monday on.", agent_id, agent_alias_id,
  session_id, enable_trace=False, session_state=session_state
)
print(agent_response_2)
```

Para recuperar los sessionAttributes en su función de Lambda, es necesario procesar el evento de Lambda. Dado que los promptSessionAttributes se comparten con la petición en sí misma, no se requiere ningún cambio adicional en el código para procesar sus valores. Estos se encuentran disponibles en el objeto de evento sessionAttributes de la función de Lambda, que contiene la clave promptSessionAttributes.



Fragmento de código de la función de Lambda que recupera sessionAttributes o promptSessionAttributes.

```
def get_named_sessionAttribute(event, name):
  if name in event['sessionAttributes']:
     return event['sessionAttributes'][name]
  else:
     return None

def lambda_handler(event, context):
  employee_id = get_named_sessionAttribute(event, "employee_id")
```